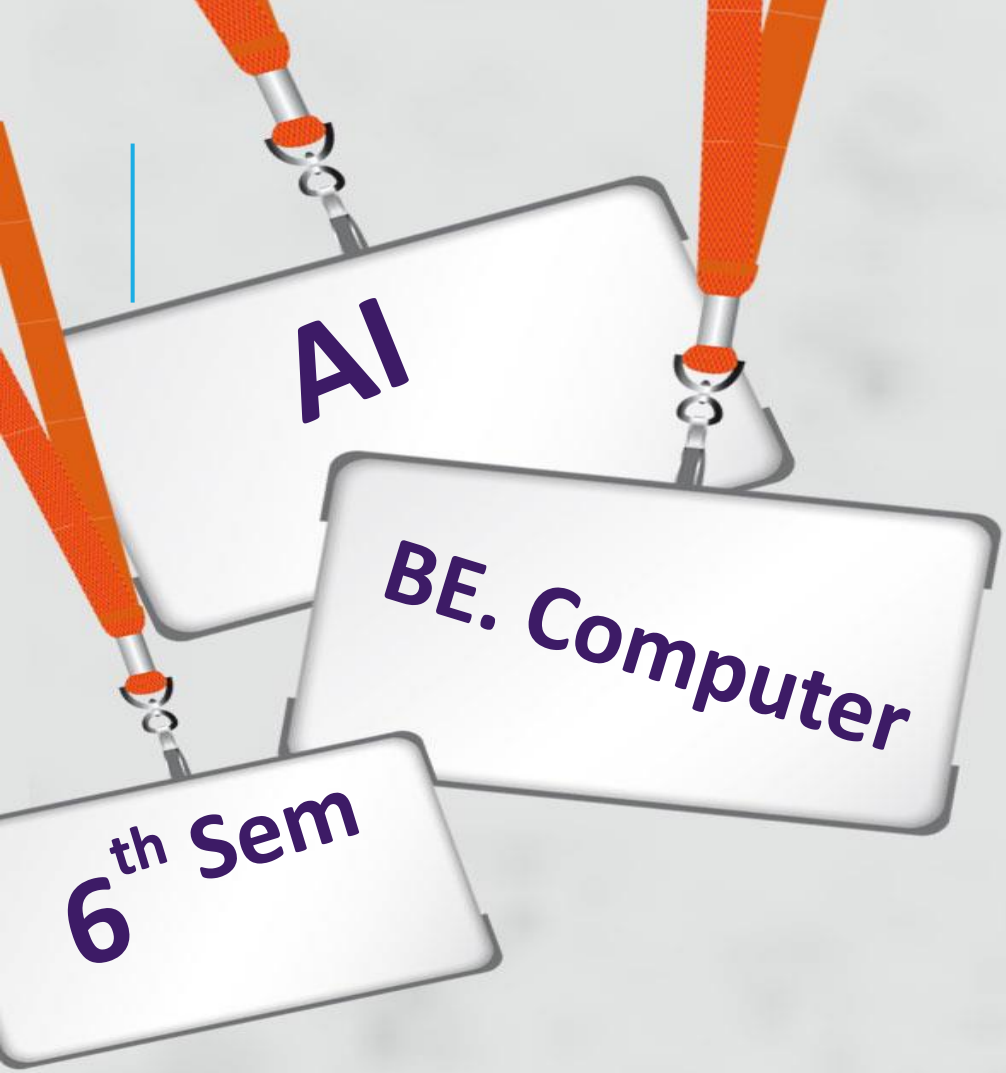


Chapter 3

Search Techniques

SEARCHING

- The general concept of ‘searching’ is about looking for something.
- In computer science, searching techniques are strategies that look for solutions to a problem in a search space.
- The solutions or ‘goal states’ could sometimes be an object, a goal, a sub-goal or a path to the searched item.
- Search is a universal problem-solving technique.

SEARCHING

- In general, a computer search problem has the following characteristics:
  - **A goal state** – the definition of a need
  - **An initial state** – a current position or a set of conditions
  - **A set of actions -** strategies and actions to select and evaluate an option in the search space
  - **Goal test criteria** – criteria used to test if an option is a solution
  - **A path cost -** the cost of the actions in a search path.

SEARCH SPACE

- The search space can be represented by a ‘graph’ which is based on graph theory.
- A computer ‘tree’ data structure is a special kind of graph. A tree has a **root node** on the top of the structure and it has **at most one path to each node**. Each node may be connected to a lower level of neighbors which are called **child nodes** (successors). Nodes that have no children are called **leaf nodes**.

Why is search important in Artificial Intelligence?

QUICK REVISION…….

1. Define problem. Explain steps of problem solving.

2.What do you mean by problem formulation.

3. What are well-defined problems. What are the components of well defined problems.

4. What are state space of the problem. Explain problem as a state space with an example.

5. Formulate the TOH problem with its state space and show solution for 4 number of disks.

6.What are CSP. How constraints helps during problem solving.

7. Formulate the Eight Queen Problem as CSP.

SEARCH TERMINOLOGY

- **Problem Space**: Environment in which the search takes place.
- **Problem Instance:** It is Initial state + Goal state
- **Problem Space Graph:** It represents problem state. States are shown by nodes and operators are shown by edges.
- **Depth of a problem:** Length of a shortest path or shortest sequence of operators from Initial State to goal state.

- **Space Complexity:** The maximum number of nodes that are stored in memory.
- **Time Complexity:** The maximum number of nodes that are created.
- **Admissibility:** A property of an algorithm to always find an optimal solution.
- **Branching Factor:** The average number of child nodes in the problem space graph.

SEARCH STRATEGIES

- A search strategy is defined by picking the order of node expansion
- Strategies are evaluated along the following dimensions:

Completeness:

does it generate to find a solution if there is any?

Optimality:

does it always find the highest quality (least-cost) solution?

Time complexity:

How long does it take to find a solution?

Space complexity:

How much memory does it need to perform the search?

Time and space complexity are measured in terms of

- *b:* maximum branching factor of the search tree
- *d:* depth of the least-cost solution
- *m*: maximum depth of the state space (may be ∞)

CLASSIFICATION

Uninformed Search (Blind Search/Brute force search)

The search algorithms that do not use any extra information regarding the problem

- ✓ Depth First Search
- ✓ Breath First Search
- ✓ Depth Limit Search
- ✓ Iterative deepening
- ✓ Uniform Cost
- ✓ Bidirectional Search

Informed search or Heuristic search

Informed search have problem specific knowledge apart from problem definition

- ✓Hill climbing Search
- ✓Best first Search
  - ▪Greedy Best First Search
  - ▪A* Search
- ✓Simulated Annealing

DEPTH FIRST SEARCH (DFS)

- Proceeds down a single branch of the tree at a time
- Expands the root node, then the leftmost child of the root node
- Always expands a node at the deepest level of the tree
- Only when the search hits a dead end (a partial solution which can't be extended), the search backtrack and expand nodes at higher levels.

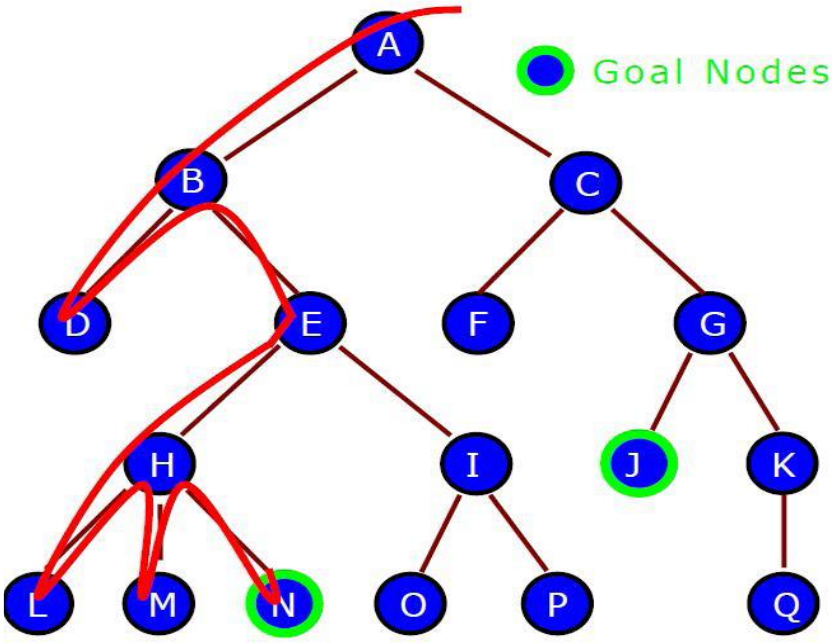


Fig. Depth-first search (DFS)

DEPTH FIRST SEARCH (DFS)

- ***Completeness***:
  - ✓ Can get stuck going down the wrong path when a different choice would lead to a solution near the root of the search tree
  - ✓ Not complete
- ***Optimality***:
  - ✓ The strategy might return a solution path that is longer than the optimal solution, if it starts with an unlucky path
  - ✓ Not optimal

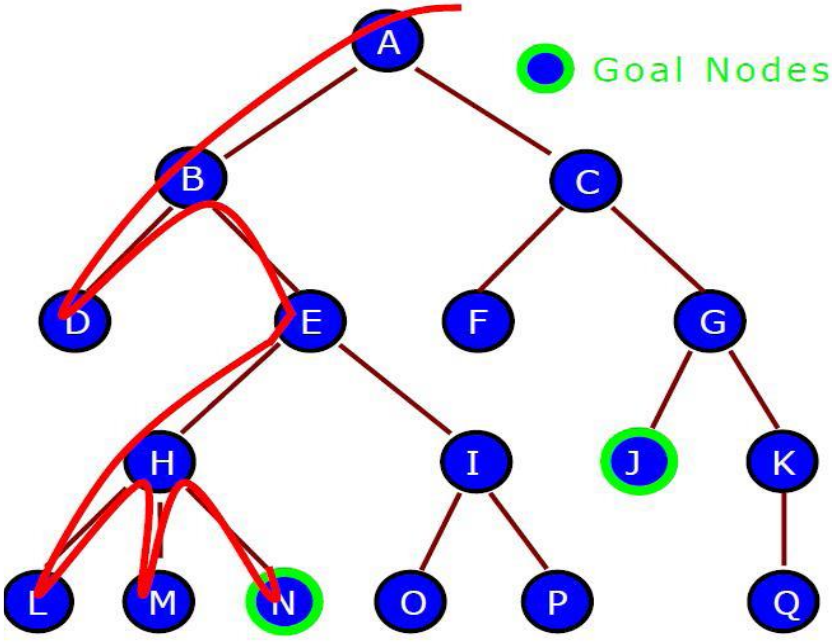


Fig. Depth-first search (DFS)

DEPTH FIRST SEARCH (DFS)

- ***Space complexity***:
  - It needs to store a single path from root to a leaf node and the remaining unexpanded sibling nodes for each node in the path
  - Once a node has been expanded, it can be removed from memory as soon as all its decedents have been fully explored
  - For a search tree of branching factor ‘b’ and maximum tree depth ‘m’, only the storage of bm+1 node is required
  - Hence,

$$\text{SpaceComplexity} = O(bm+1) = O(bm)$$

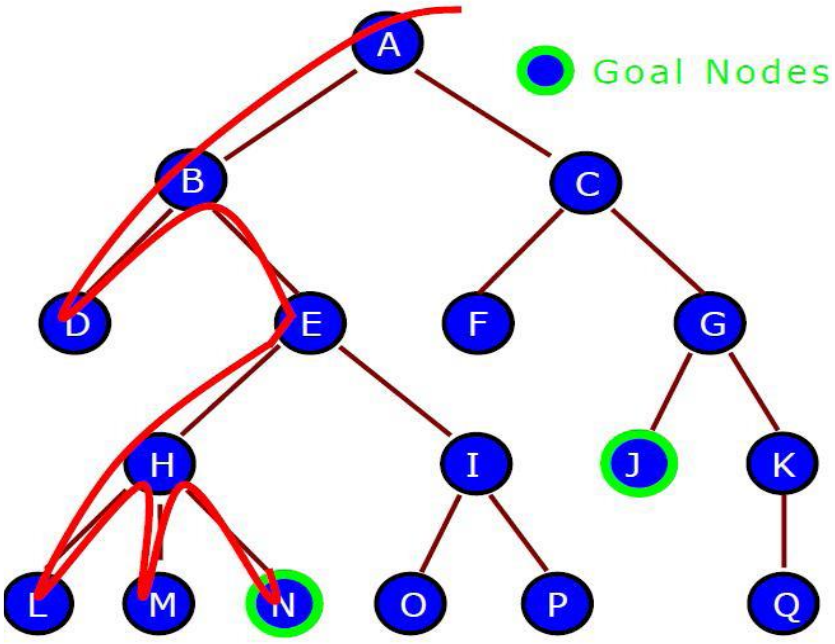


Fig. Depth-first search (DFS)

DEPTH FIRST SEARCH (DFS)

- ***Time Complexity*:**

  If you can access each node in O(1) time, then with branching factor of b and max depth of m, the total number of nodes in this tree would be = b * b *b … m times = b^m, resulting in total time to visit each node proportional to b^m. Hence the complexity = $O(b^m)$

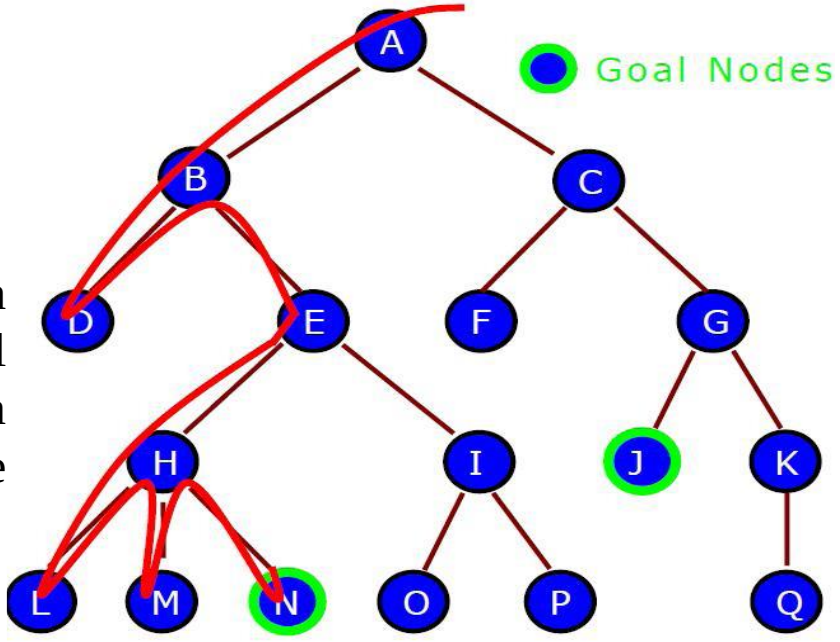
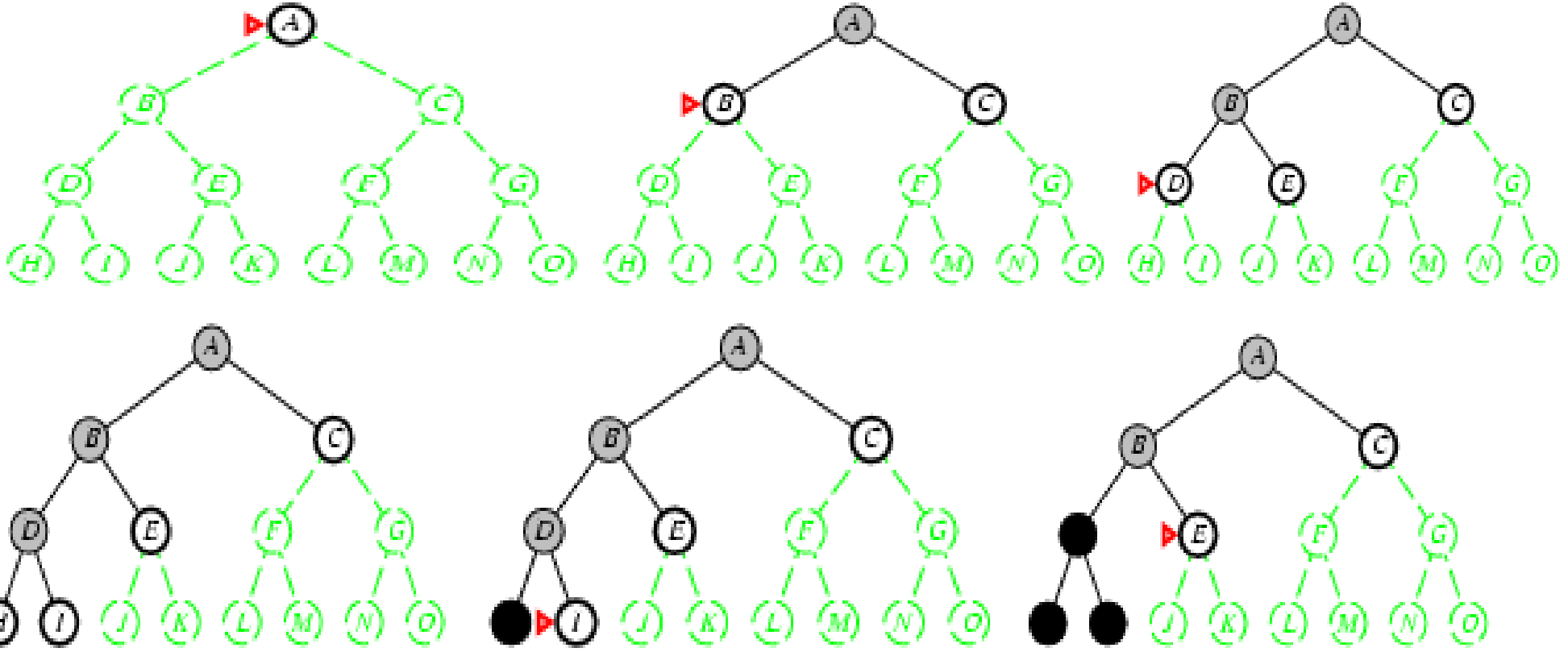
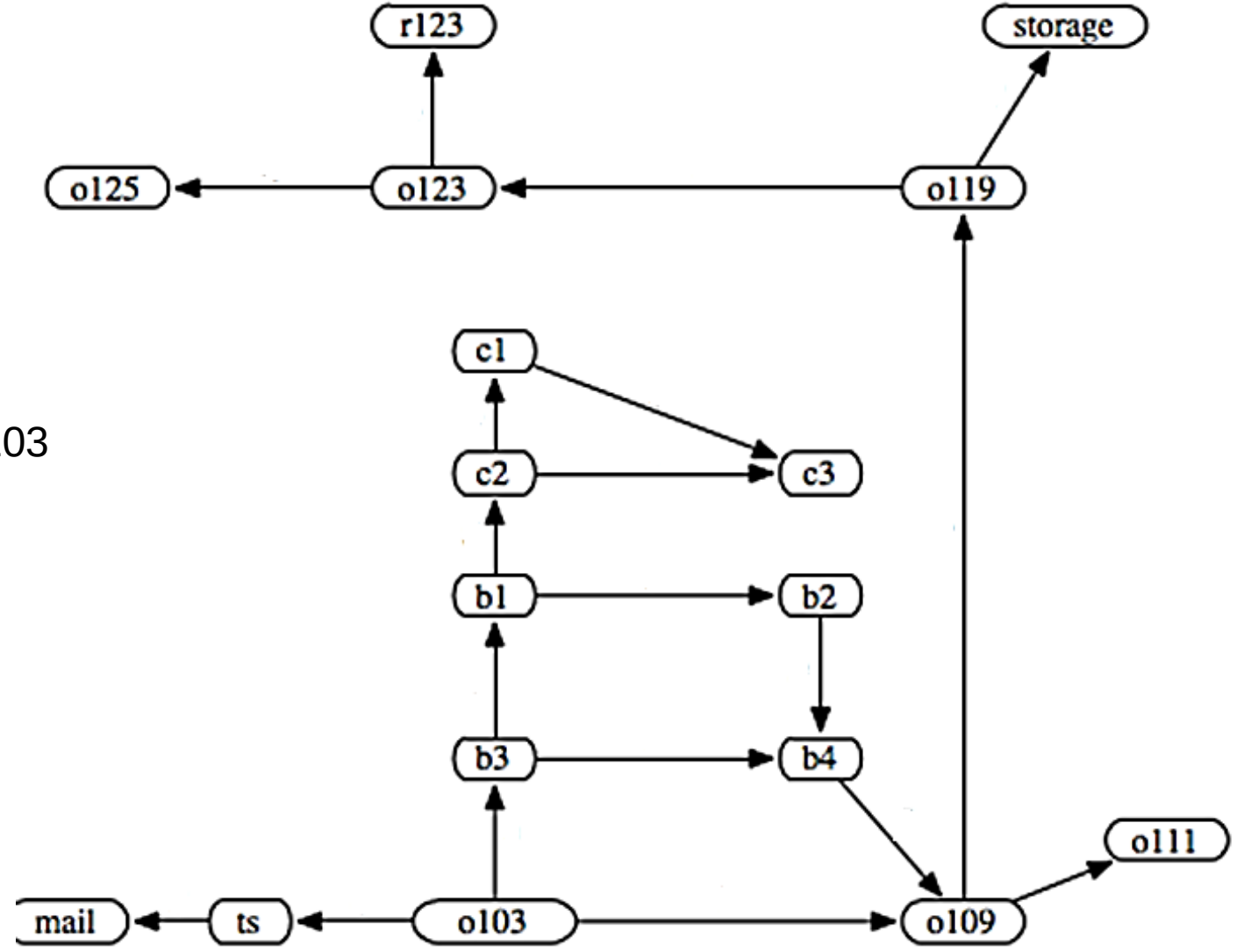


Fig. Depth-first search (DFS)

DFS example (find path from A to E)



Q. Find the Path from o103 to r123 using DFS



Breadth-First Search (BFS)

- Proceeds level by level down the search tree
- Starting from the root node (initial state) explores all children of the root node, left to right
- If no solution is found, expands the first (leftmost) child of the root node, then expands the second node and so on

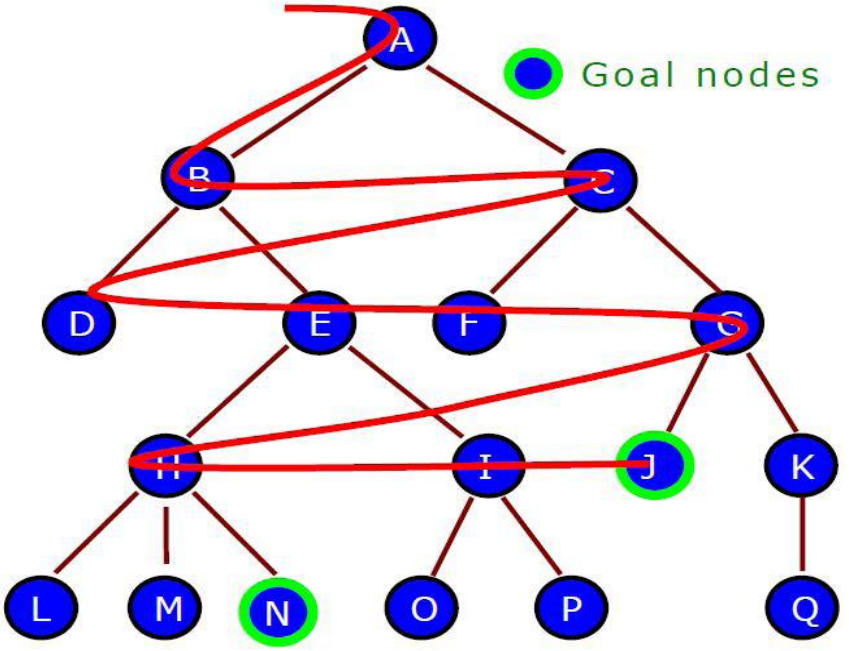


Fig. Breadth-first search (BFS)

Breadth-First Search (BFS)

- ***Completeness:***
  - This search strategy finds the shallowest goal first
  - Complete, if the shallowest goal is at some finite depth
- ***Optimality***
  - The shallowest goal node is not necessarily the optimal one
  - Optimal, if the path cost is a non-decreasing function of the path of the node (For example: when all the actions have the same cost)

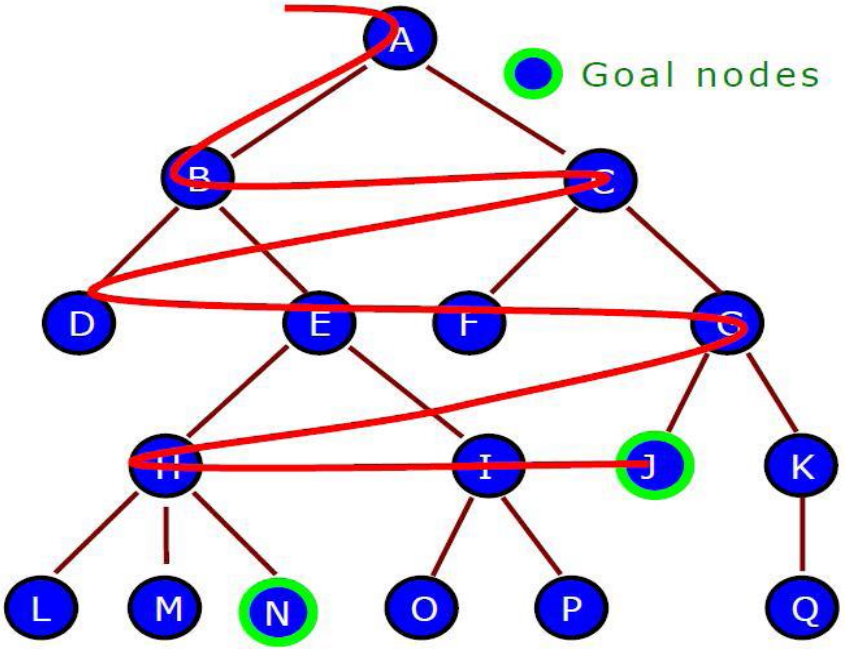


Fig. Breadth-first search (BFS)

Breadth-First Search (BFS)

❑ *Time Complexity:*

- For a search tree a branching factor ‘b’ expanding the root yields ‘b’ nodes at the first level.
- Expanding ‘b’ nodes at first level yields b^2 nodes at the second level.
- Similarly, expanding the nodes at d^{th} level yields b^{d+1} node at $(d+1)^{th}$ level

Hence, time complexity is $O(b^{d+1})$
where,

b= branching factor and
d= level of goal node in the search table

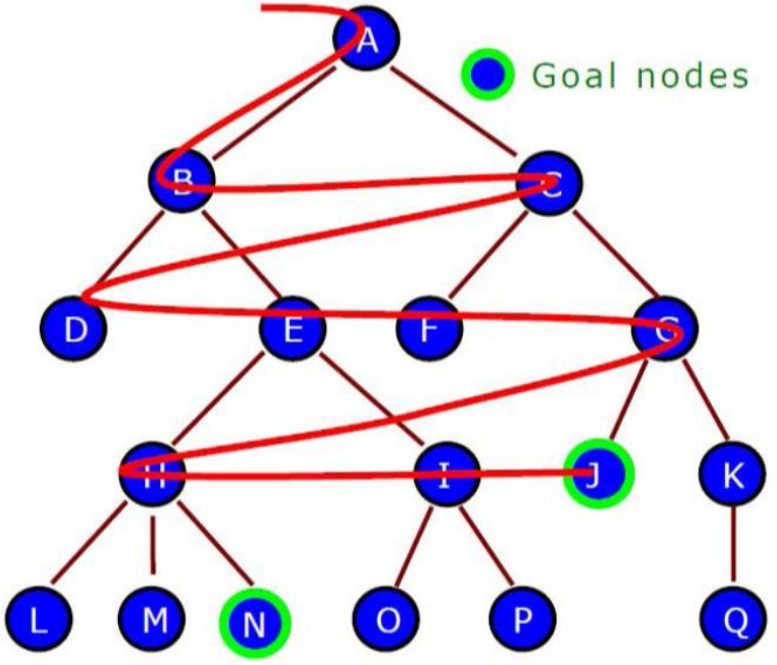


Fig. Breadth-first search (BFS)

Breadth-First Search (BFS)

- ***Space Complexity***
  - Same as time complexity
  - i.e. $O(b^{d+1})$

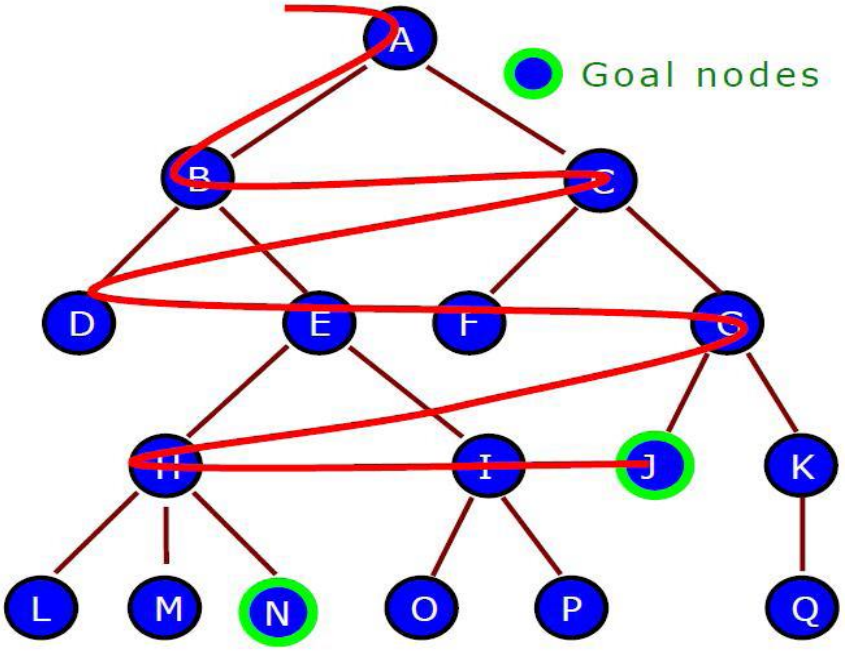
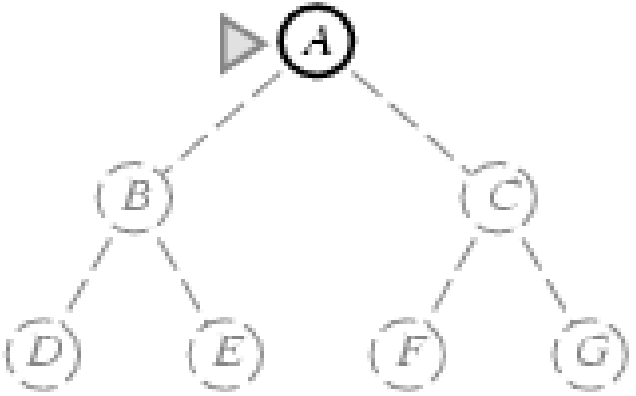
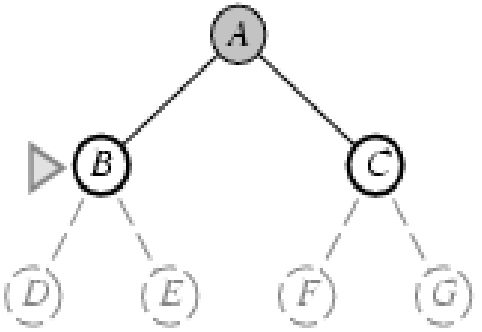
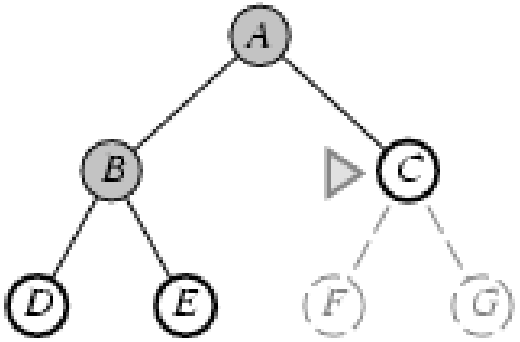


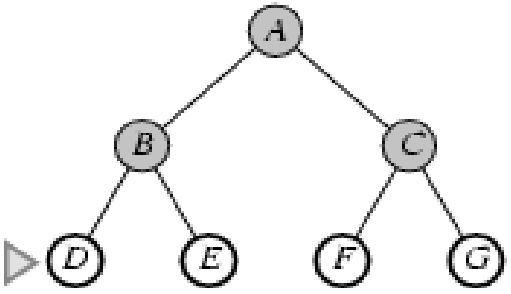
Fig. Breadth-first search (BFS)

BFS example (Find path from A to D)

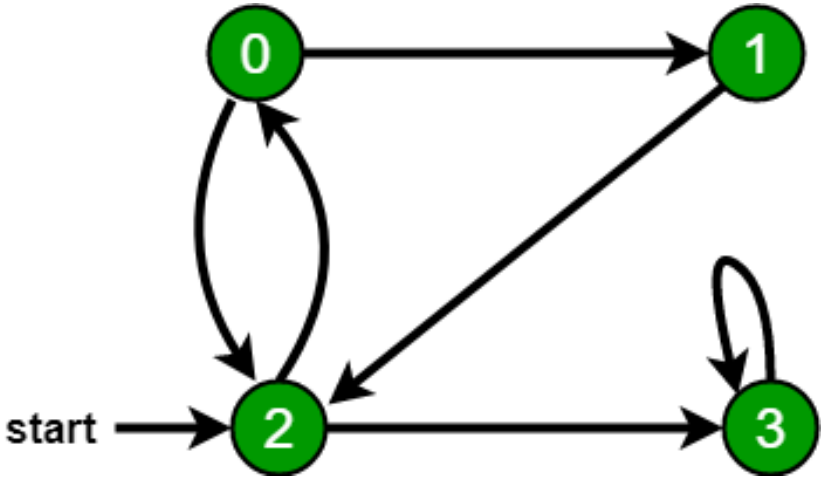






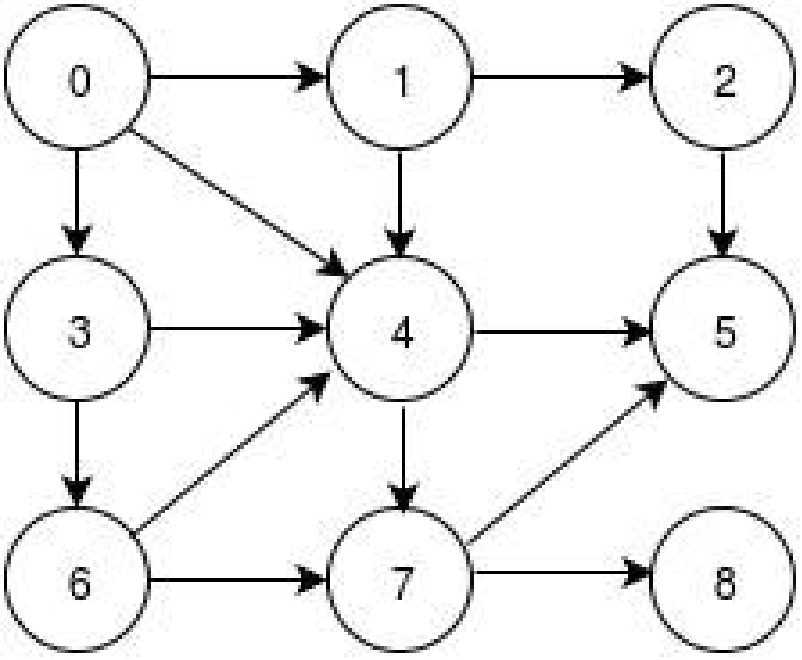


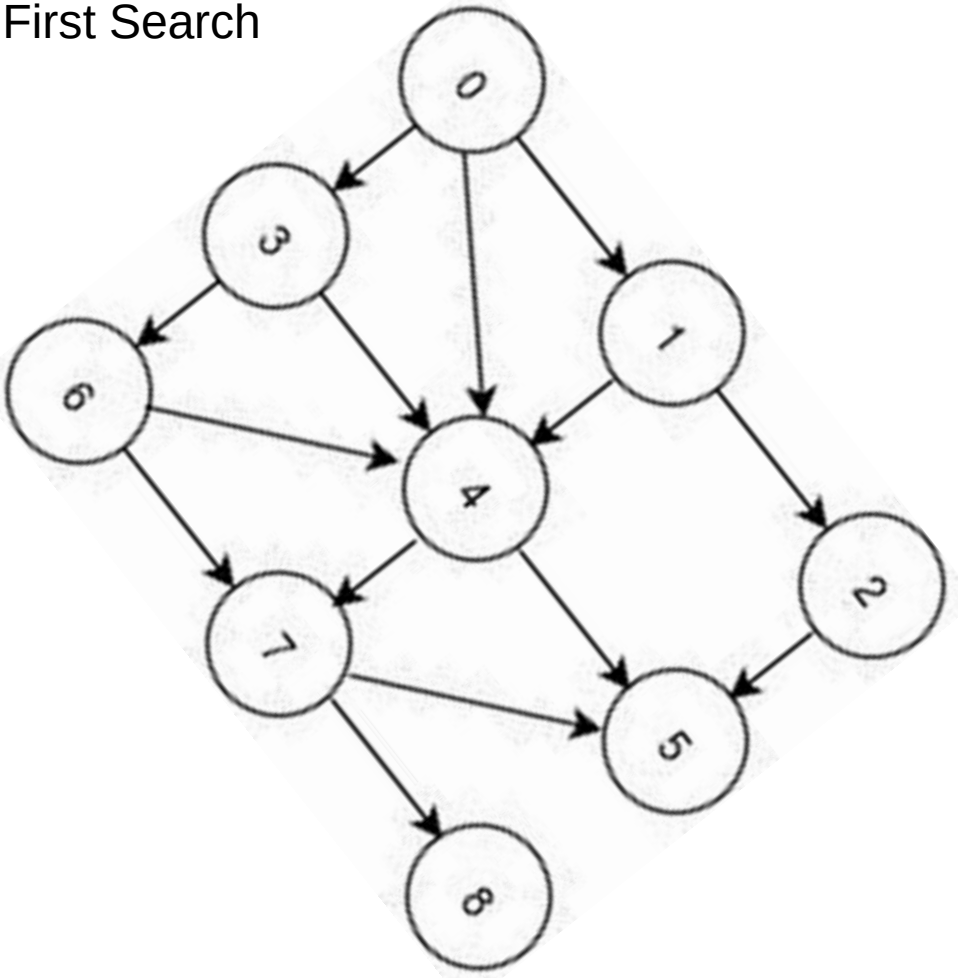
Perform the Breadth First Search



2, 0, 3, 1.

Q. Find the Path from 0 to 8 using Breadth First Search



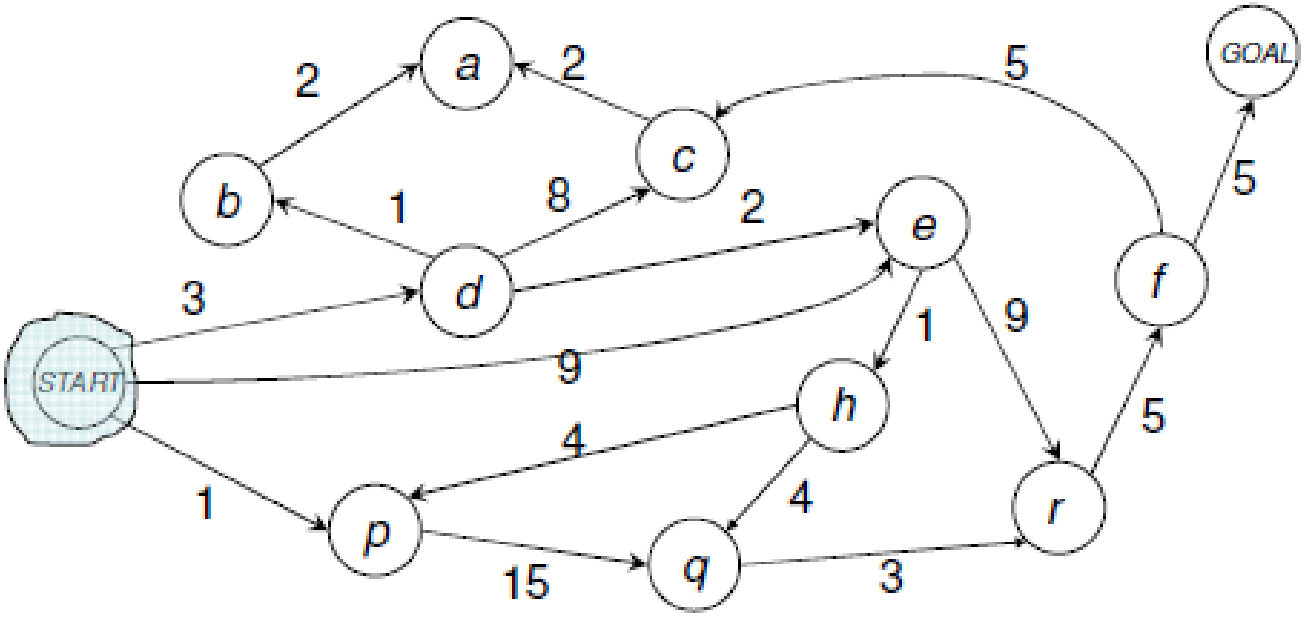


0 1 4 3 2 6 5 7

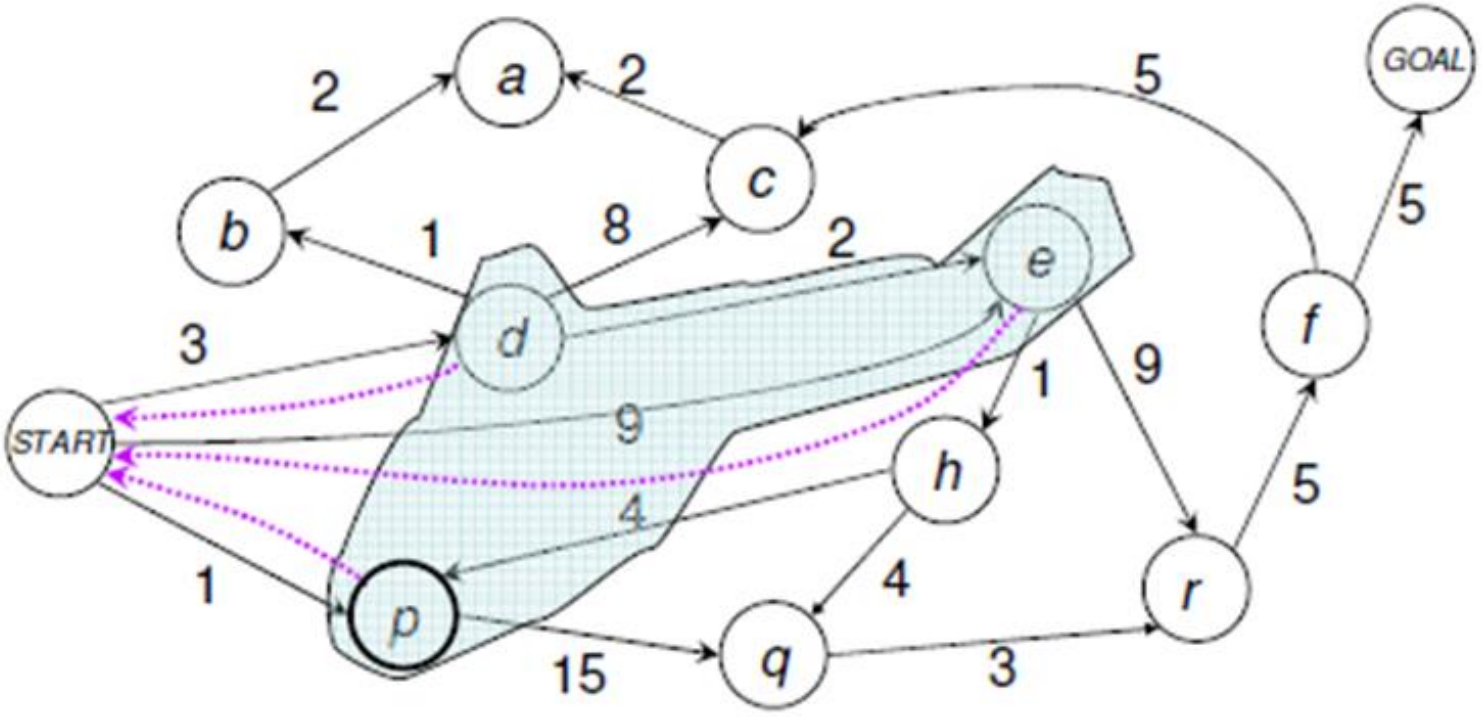
Uniform Cost Search

Uniform-cost is guided by path cost rather than path length, the algorithms starts by expanding the root, then expanding the node with the lowest cost from the root, the search continues in this manner for all nodes.

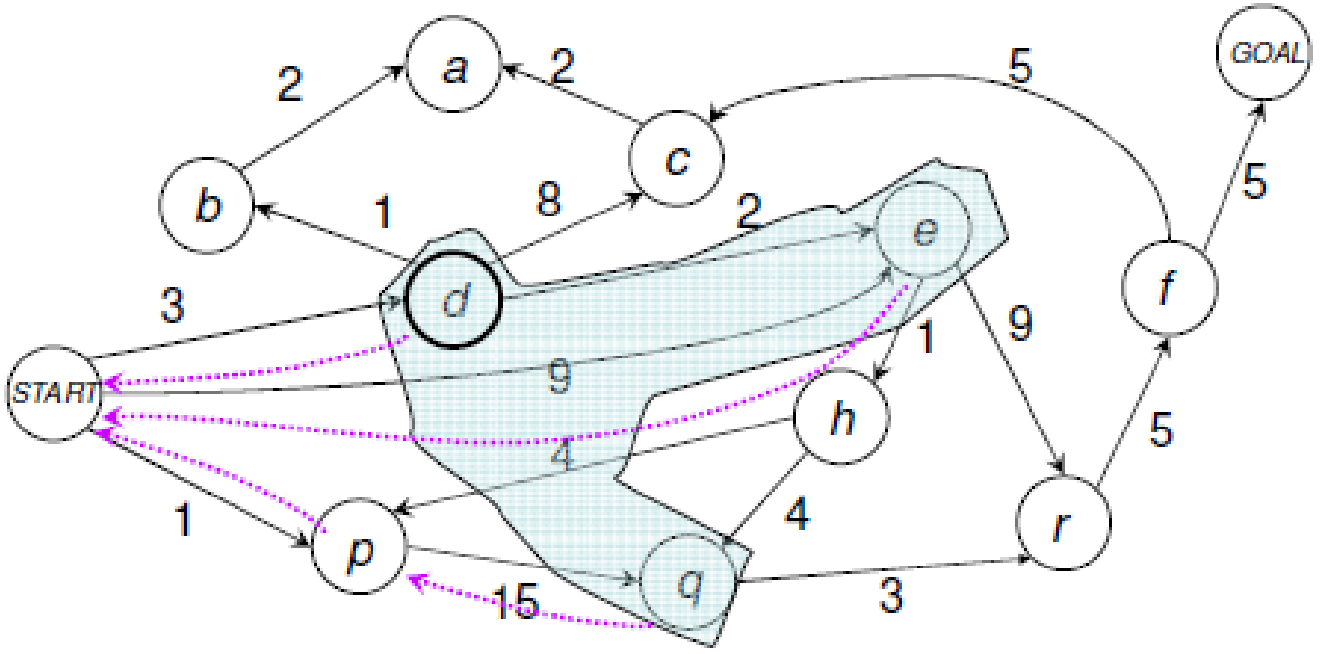
- **Completeness:**
  Complete if the cost of each step exceeds some small positive integer, this to prevent infinite loops.
- **Optimality:**
  Optimal in the sense that the node that it always expands is the node with the least path cost.
- **Time Complexity:** $O(b^{C/e})$.
- **Space Complexity:** $O(b^{C/e})$



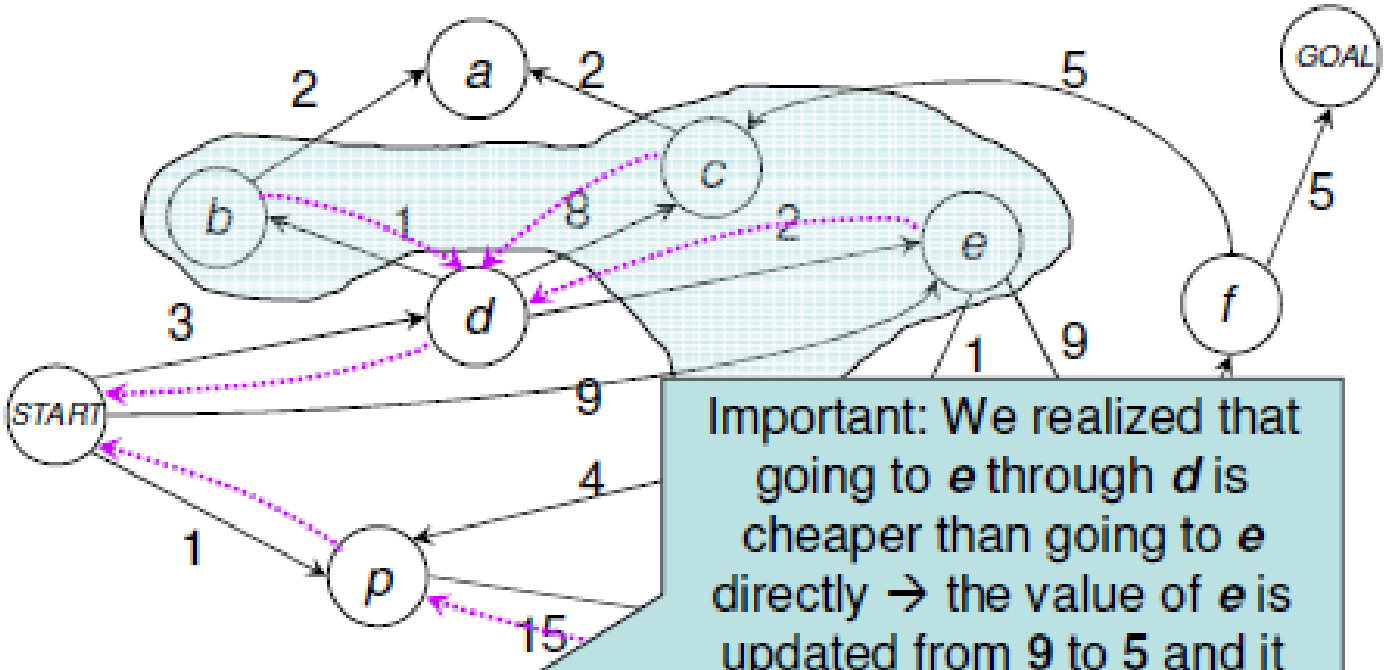
PQ = {(START,0)}



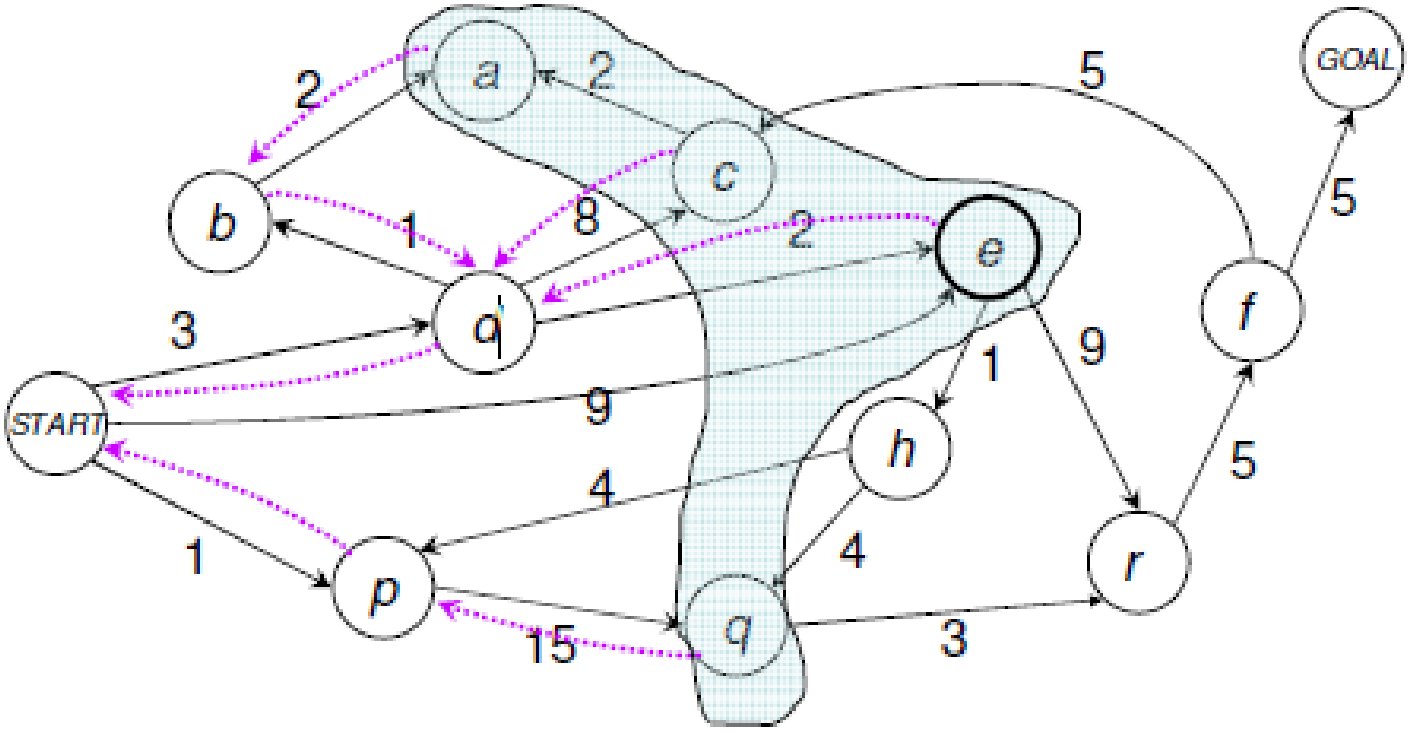
$PQ = \{(p,1)\ (d,3)\ (e,9)\}$



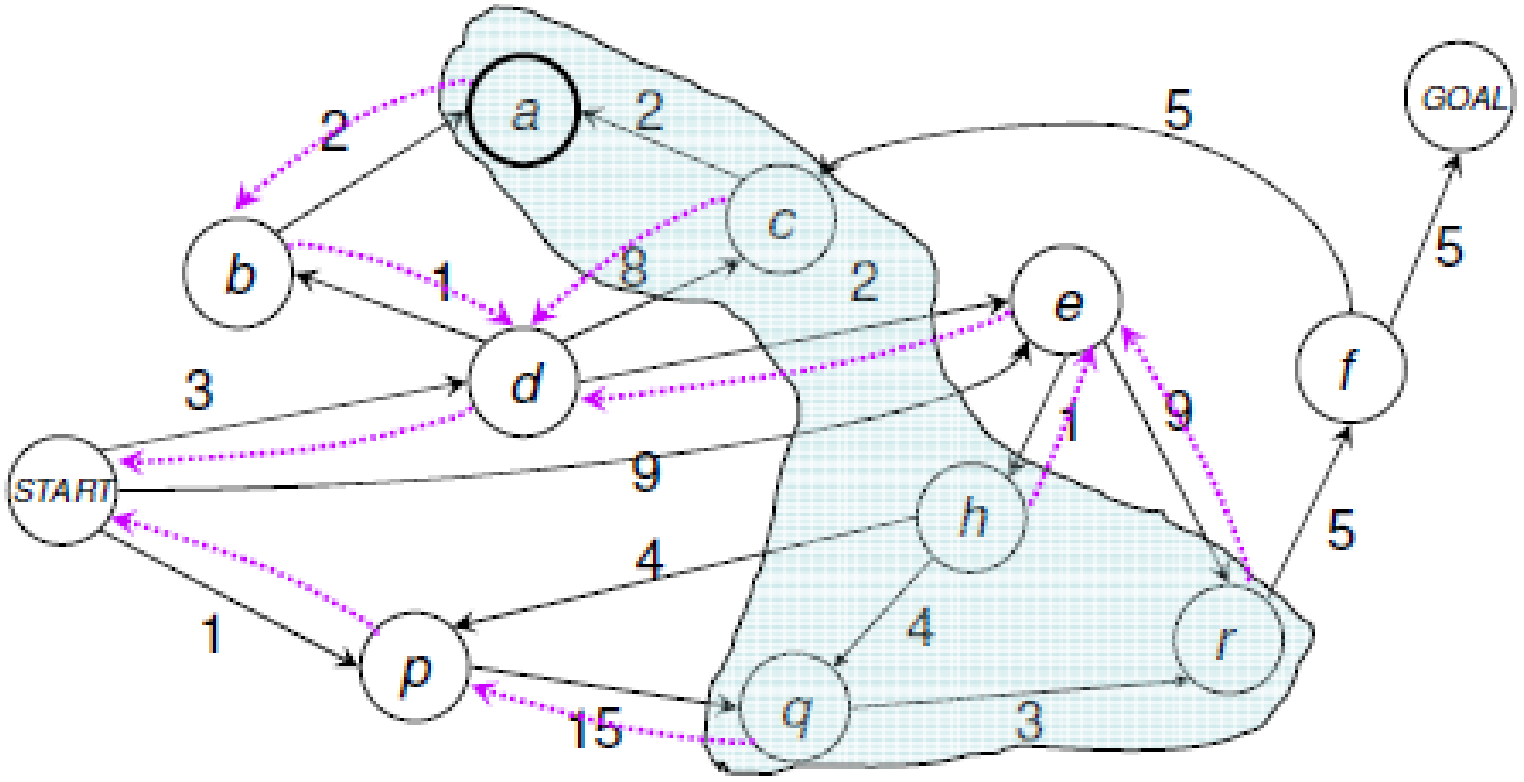
PQ **= {(*d*,3) (*e*,9) (q,16)}**



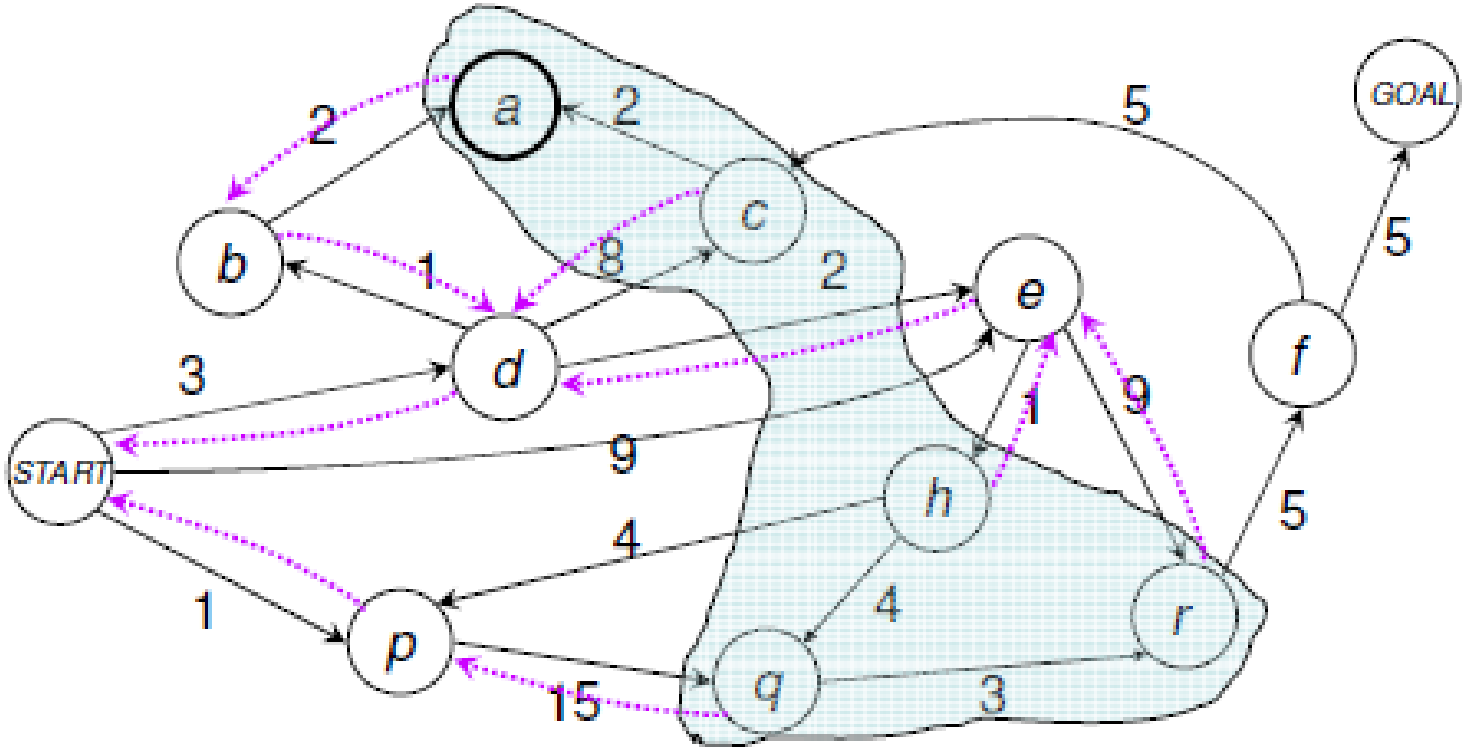
PQ = {(*b*,4) (*e*,5) (*c*,11) (*q*,16)}



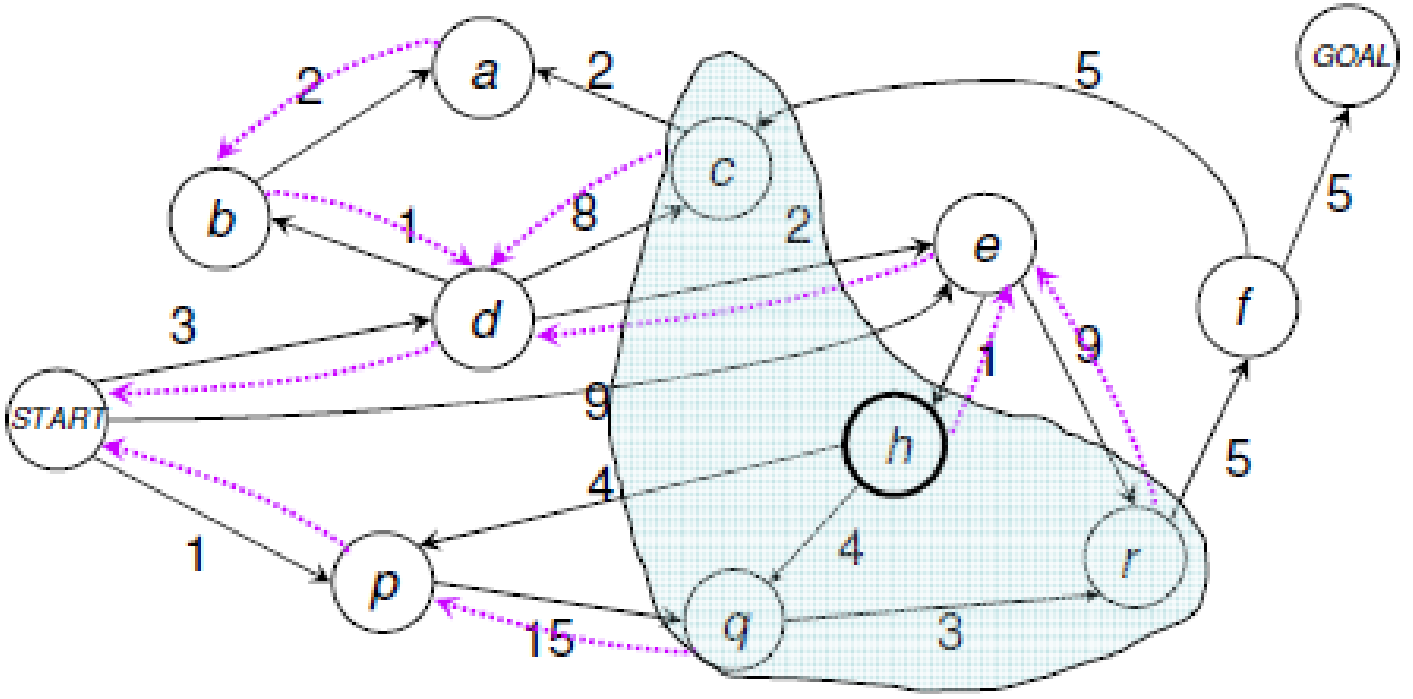
PQ = {(e,5) (a,6) (c,11) (q,16)}



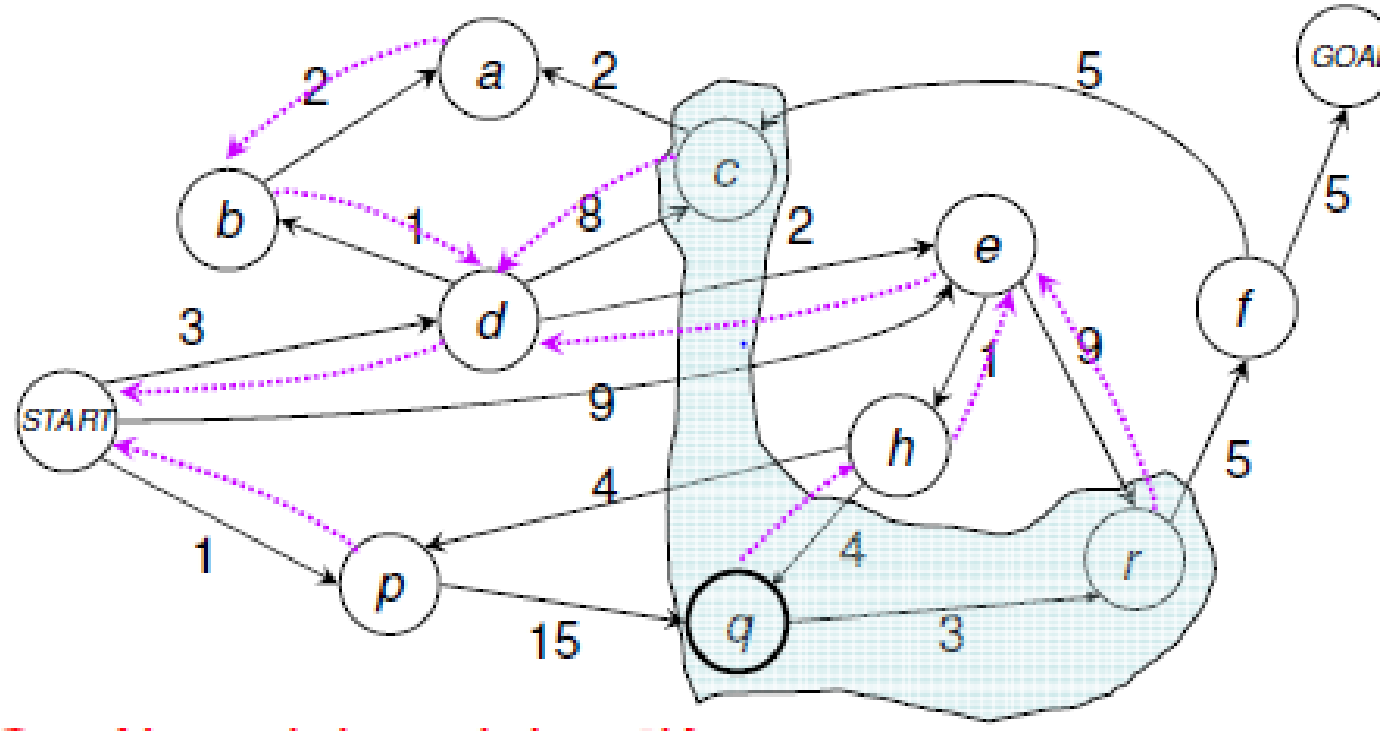
PQ = {(*a*,6) (*h*,6) (*c*,11) (*r*,14) (*q*,16)}



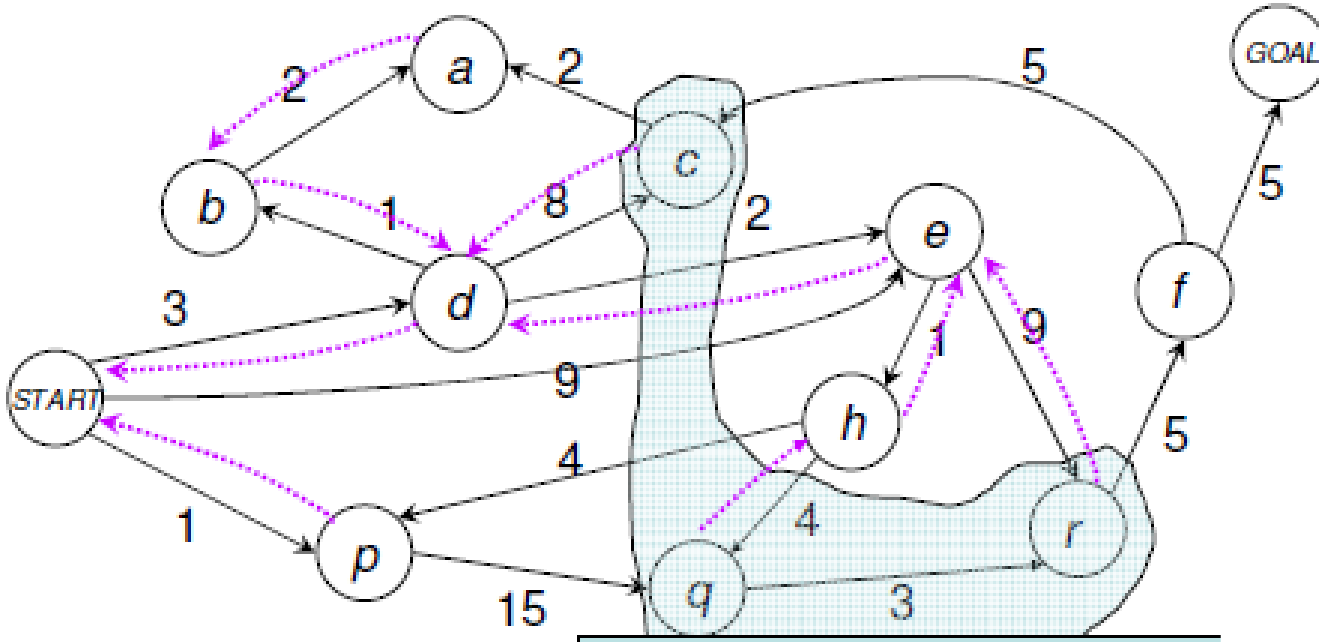
PQ = {(*a*,6) (*h*,6) (*c*,11) (*r*,14) (*q*,16)}



PQ = {(*h*,6) (*c*,11) (*r*,14) (*q*,16)}

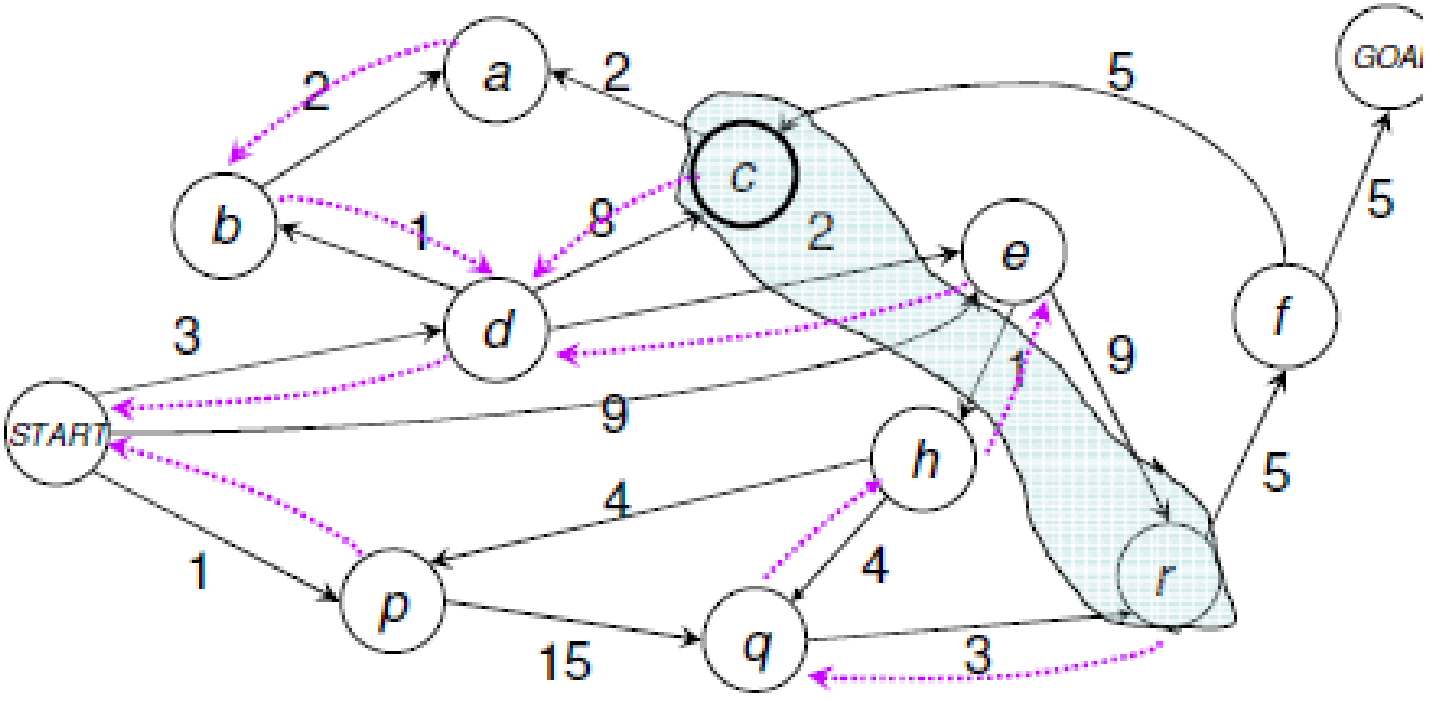


PQ = {(*q*,10) (*c*,11) (*r*,14)}

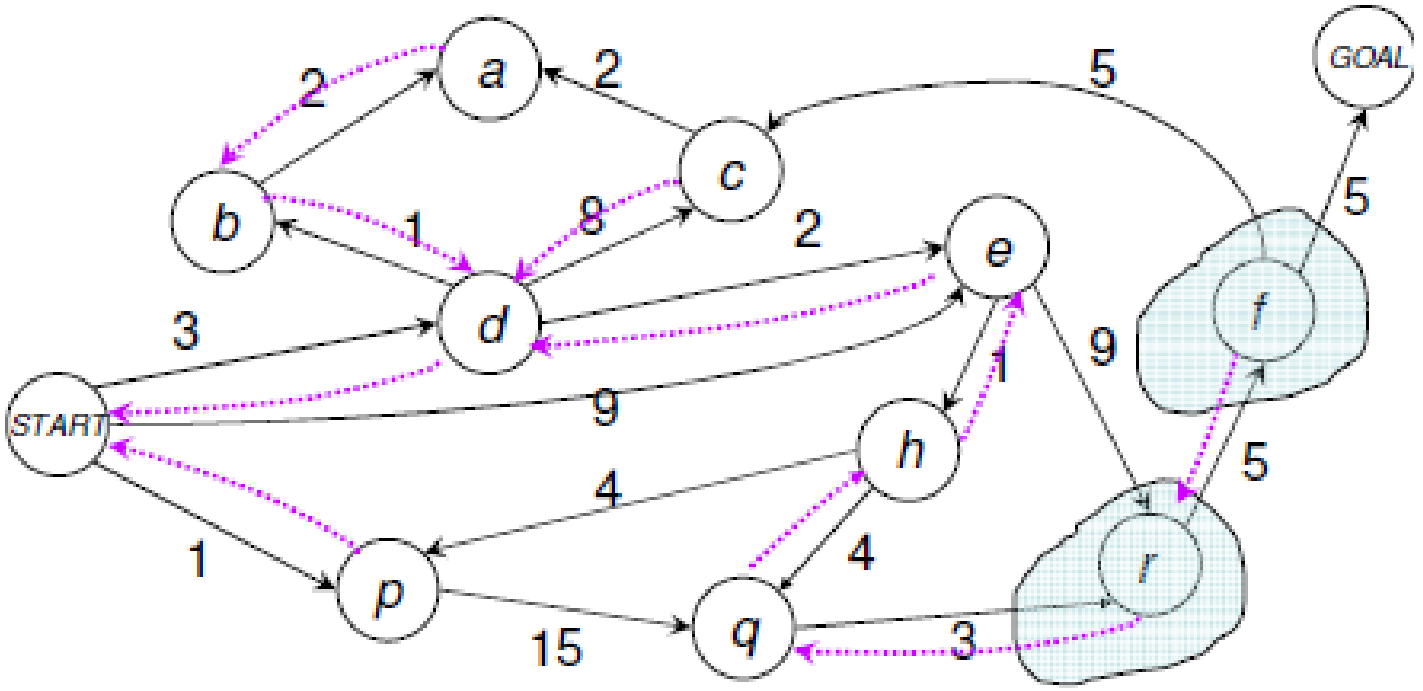


PQ = {(*q*,10) (*c*,11) (

Important: We realized that going to ***q*** through ***h*** is cheaper than going through ***p*** → the value of ***q*** is updated from **16** to **10** and it moves up in PQ

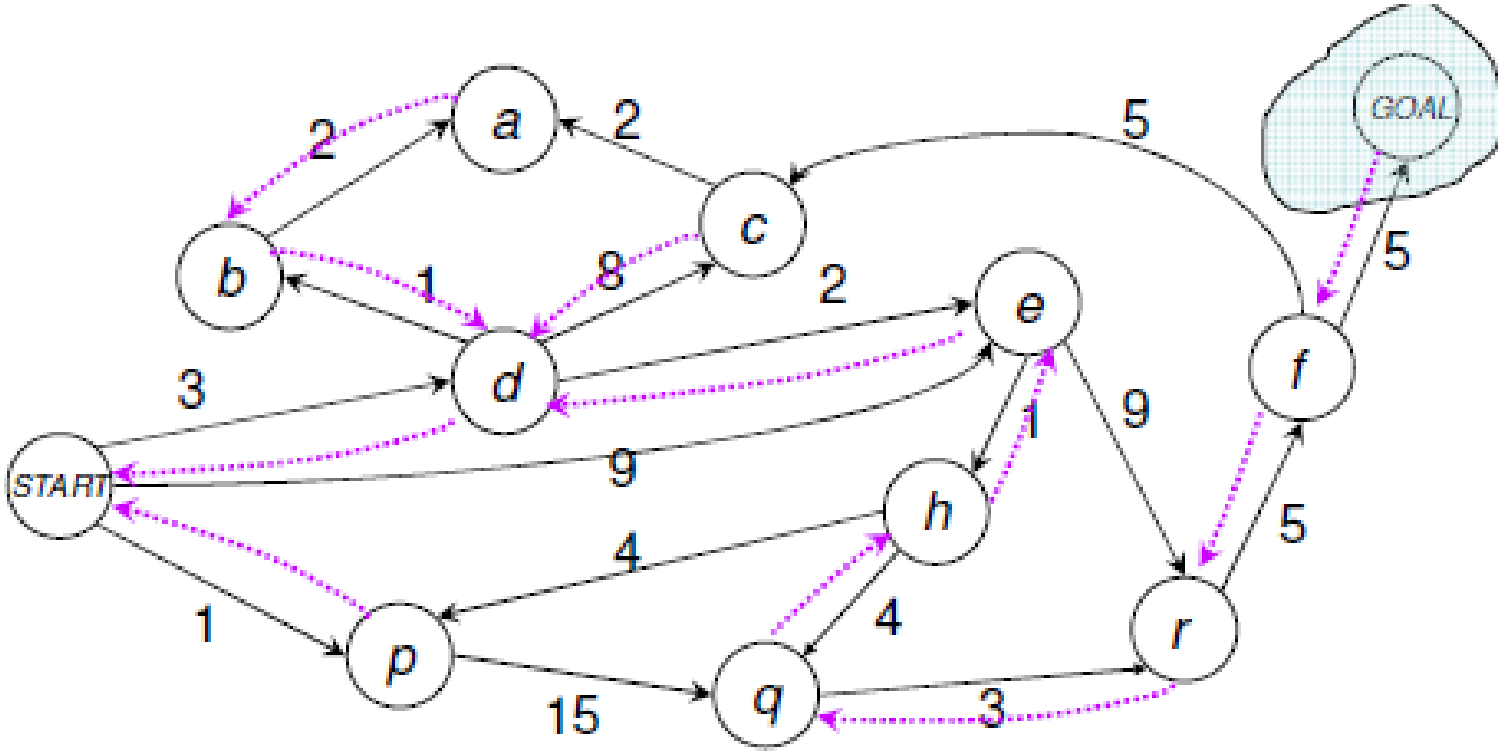


***PQ* = {(*c*,11) (*r*,13)}**

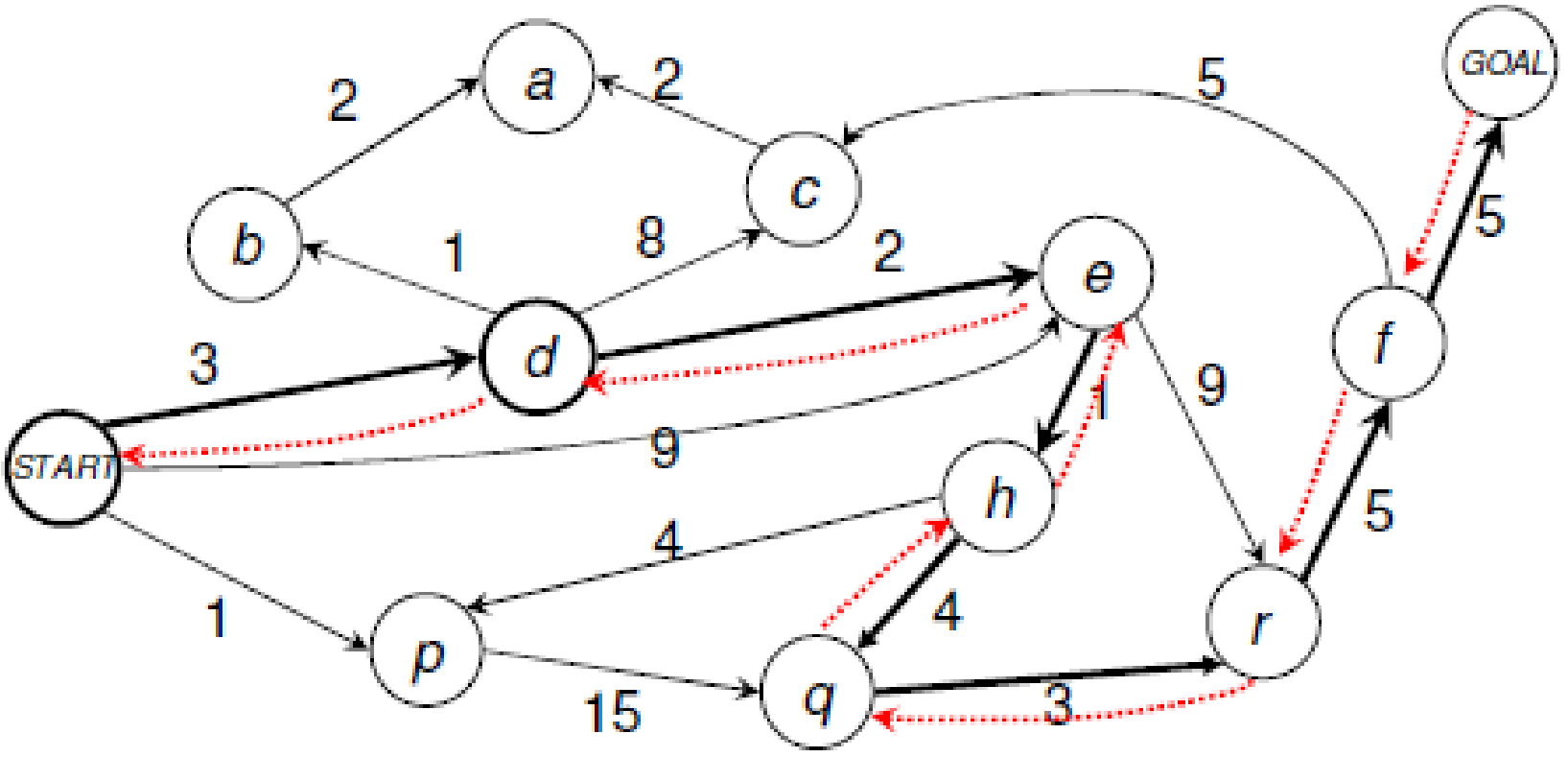


PQ = {(*r*,13)}

PQ = {(*f*,18)}

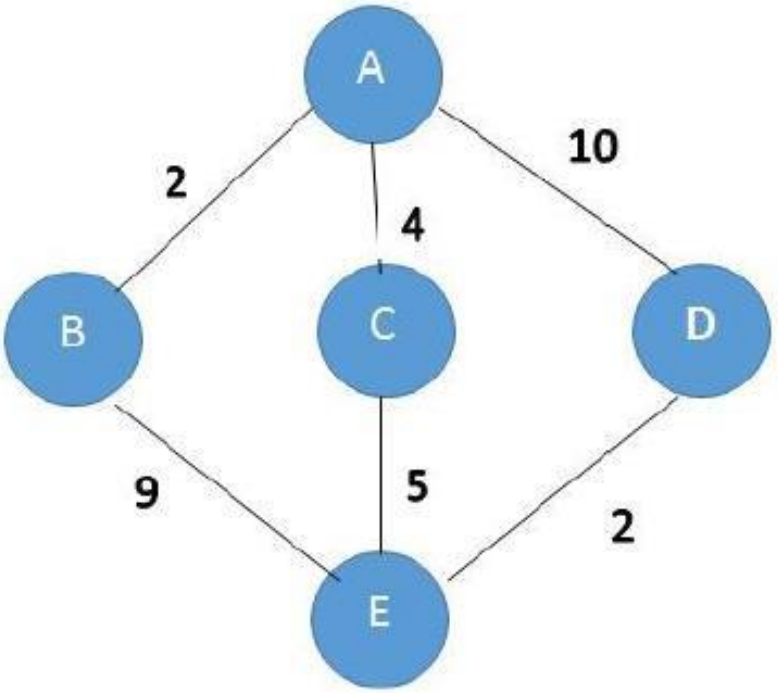


PQ = {(GOAL,23)}



Final path: {*START*, *d*, *e*, *h*, *q*, *r*, *f*, *GOAL*}

Find path from A to E using UCS



Depth Limit Search

- Depth-first search will not find a goal if it searches down a path that has infinite length. So, in general, depth-first search is not guaranteed to find a solution, so it is not complete.
- This problem is eliminated by limiting the depth of the search to some value L. However, this introduces another way of preventing depth-first search from finding the goal: if the goal is deeper than L it will not be found.
- Perform depth first search but only to a pre-specified depth limit L.
- No node on a path that is more than L steps from the initial state

Depth Limit Search

- ***Completeness***:
  Incomplete as solution may be beyond specified depth level
- ***Optimality***:
  not optimal
- ***Space complexity***:
  b as branching factor and L as tree depth level , O(b.L)
- ***Time Complexity***:
  O(bL)

Iterative Deepening Search (IDS)

- Iterative deepening search is a strategy that sidesteps the issue of choosing the best depth limit by trying all possible depth limits.
- Starting at depth limit L = 0, we iteratively increase the depth limit, performing a depth limited search for each depth limit.
- Stop if no solution is found, or if the depth limited search failed without cutting off any nodes because of the depth limit.
- Search is helpful only if the solution is at given depth level.

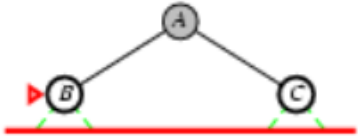
Limit = 0





Limit = 1

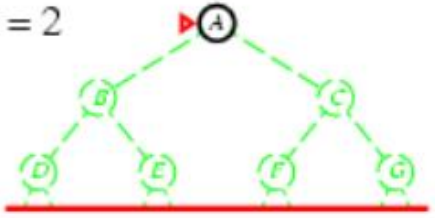


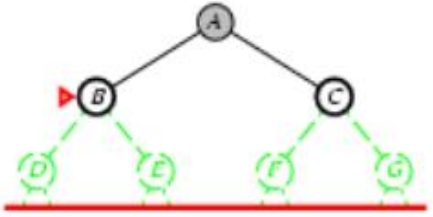


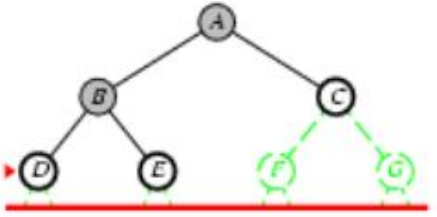


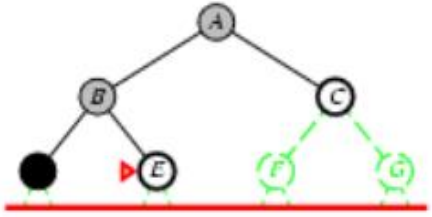


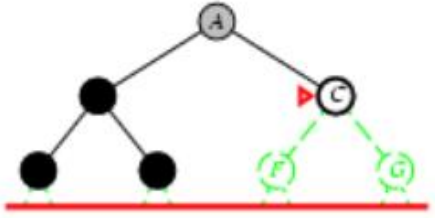
Limit = 2

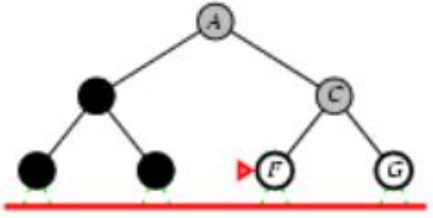


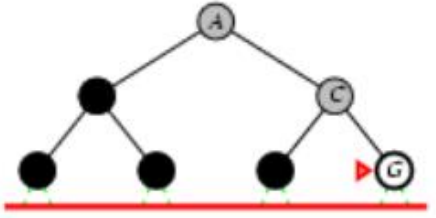


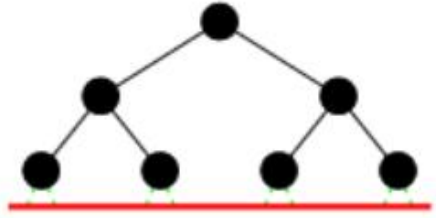




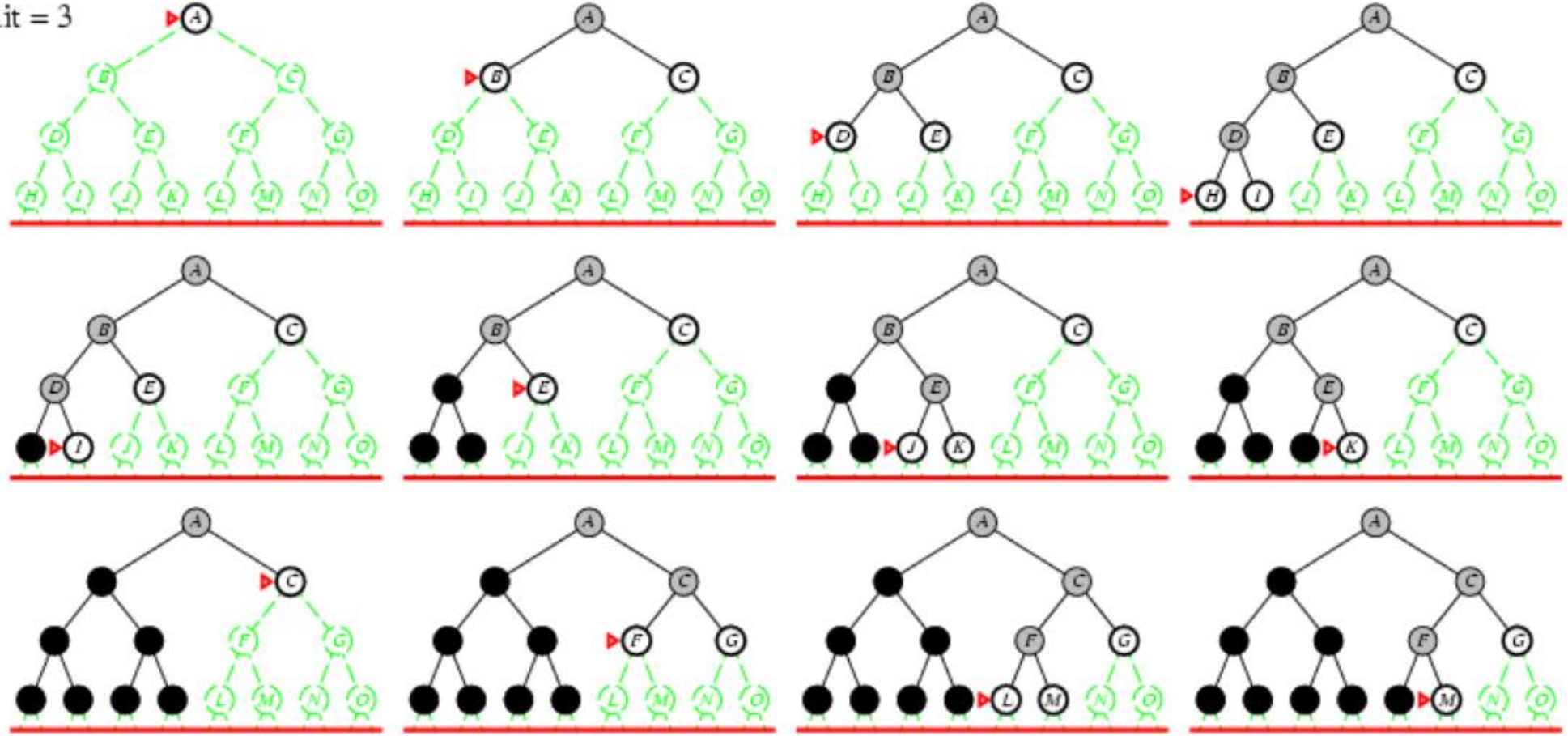








Limit = 3



What is the difference between DLS and Iterative Deepening?

DLS is a search strategy resulting when we limit the depth of the Search and use the modest memory requirements of DFS.
IDS works by looking for the depth d, thus starting with depth limit 0 and make a DFS and if the search failed it increase the depth limit by 1 and try a DFS again with depth 1

Bidirectional Search

- As the name suggests, bidirectional search suggests to run 2 simultaneous searches
- It runs two simultaneous search –
  - Forward search from source/initial vertex toward goal vertex
  - Backward search from goal/target vertex toward source vertex
- Bidirectional search replaces single search graph(which is likely to grow exponentially) with two smaller sub graphs – one starting from initial vertex and other starting from goal vertex. The search terminates when two graphs intersect.

Bidirectional Search

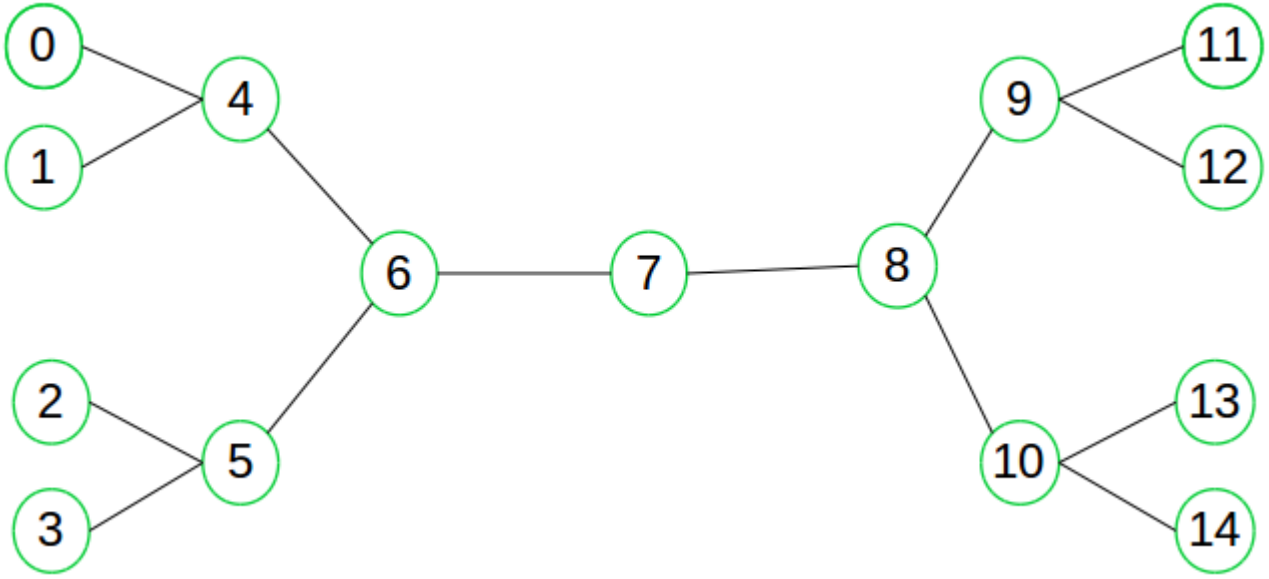
- Completeness:

  Bidirectional search is complete when we use BFS in both searches

- Optimality:

  Like the completeness, bidirectional search is optimal when BFS is used.

- Time/Space Complexity : $O(b^{d/2})$



Suppose we want to find if there exists a path from vertex 0 to vertex 14. Here we can execute two searches, one from vertex 0 and other from vertex 14. When both forward and backward search meet at vertex 7, we know that we have found a path from node 0 to 14 and search can be terminated now

Assignment:

Compare all the Search techniques

Informed Search (Heuristic Search)

- Informed search have problem specific knowledge apart from problem definition.
- They use experimental algorithm which improves efficiency of search process.
- The idea is to develop a domain specific heuristic function h(n) where h(n) guesses the cost of getting to the goal from node n.

Heuristic Function

The heuristic function is a way to inform the search about the direction to a goal. It provides an informed way to guess which neighbor of a node will lead to a goal

The heuristic function is denoted by h(n)

A heuristic function is said to be consistent, or monotone, if it’s estimate is always less than or equal to the estimated distance from any neighboring vertex to the goal.

A heuristic is consistent if for every node n, every successor n' of n if

$$f(n') \geq f(n)$$

Best-First Search (BFS)

- Best-First search is a graph-based heuristic search algorithm
- The name “best-first” refers to the method of exploring the node with the best “score” first.
- An evaluation function is used to assign a score to each candidate node. The evaluation function must represent some estimate of the cost of the path from state to the closest goal state

Algorithm

1. Put the initial node on a list START
2. If START = GOAL or START = EMPTY, then terminate search
3. Assign the next node as START and call this node-A
4. If A = GOAL, terminate the search with success
5. Else-if, node has successor and generate all of them. Find out how far they are from the GOAL node.
6. Sort all the children generated so far by remaining distance from the goal. Name the list as START-1. Replace START = START-1
7. Go to step-2

Types of BFS

- Greedy Best First Search
- A* Search

Greedy Best First Search

- It tries to get as close as it can to the goal.
- It expands the node that appears to be closest to the goal
- It evaluates the node by using heuristic function only.
- Evaluation function f(n) = h(n)

  h(n) - is estimate of cost from n to goal

  - is 0 for goal state

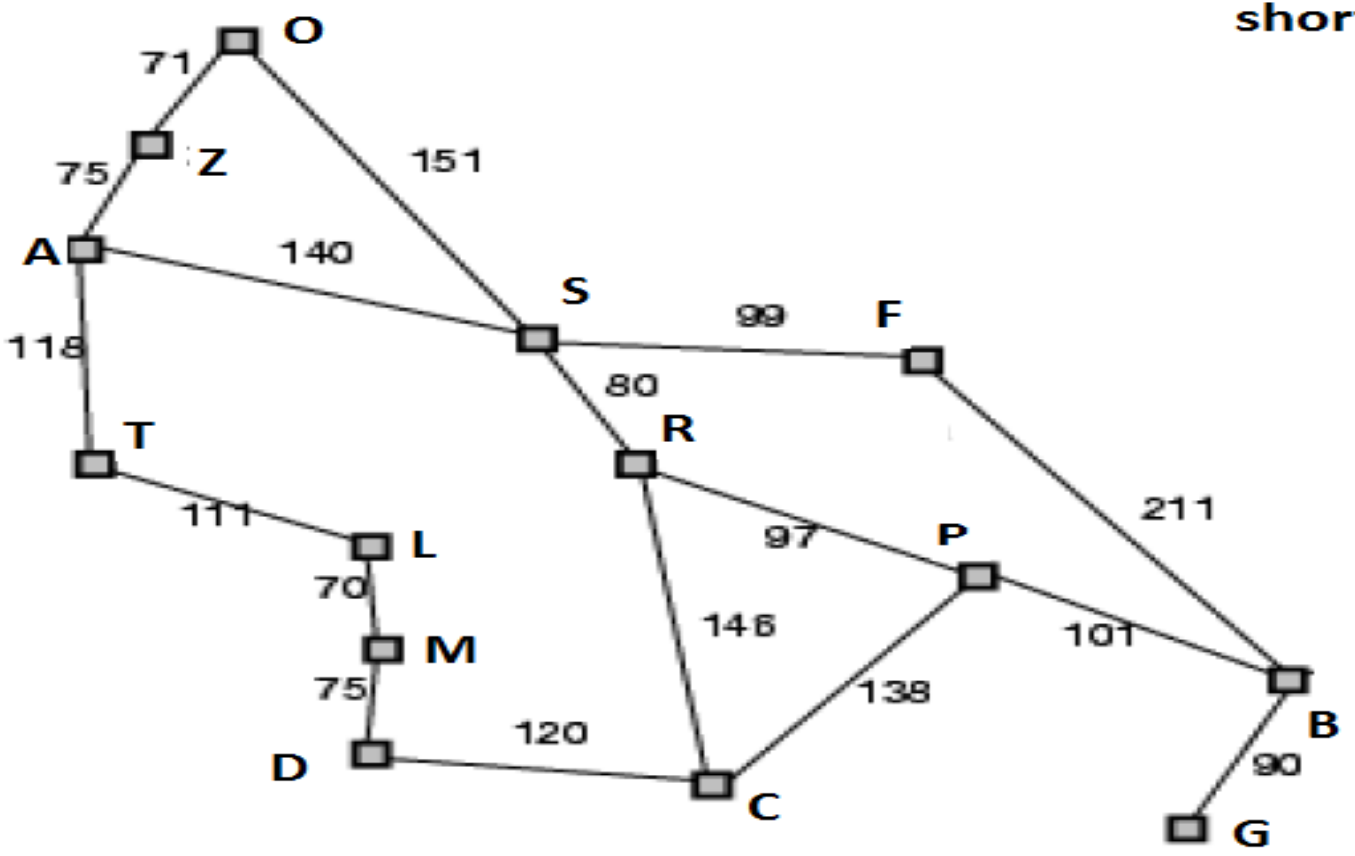
Properties

- ***Completeness:*** No – can get stuck in loops
- ***Time Complexity***: O (b^m), but a good heuristic can give dramatic improvement
- ***Space Complexity***: O (b^m), keeps all nodes in memory
- ***Optimality***: No

Applications:

This algorithm is used in Huffman encoding, minimum spanning tree, Dijkstra's algorithm etc.

Given following graph of cities, starting at City “A”, problem is to reach to the “B”

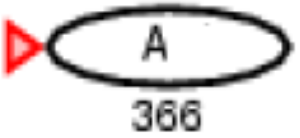


shortest line distance to B

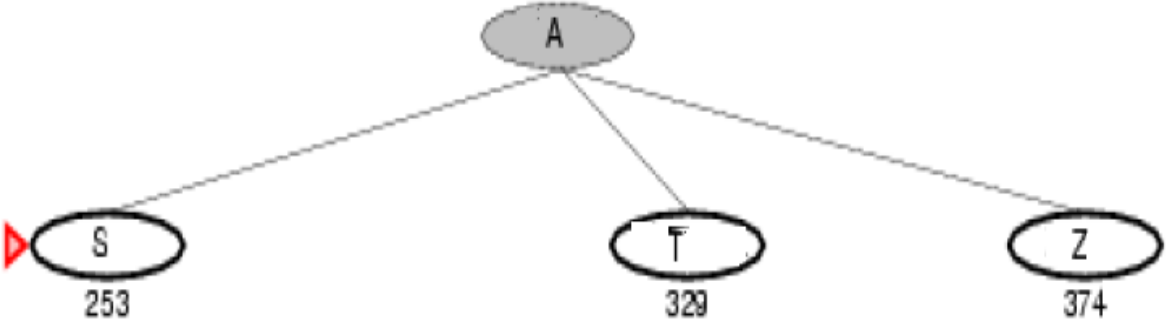
A	366
B	0
C	160
D	242
F	176
G	77
L	244
M	241
O	380
P	10
R	193
S	253
T	329
Z	374

Solution using greedy best first can be as below:

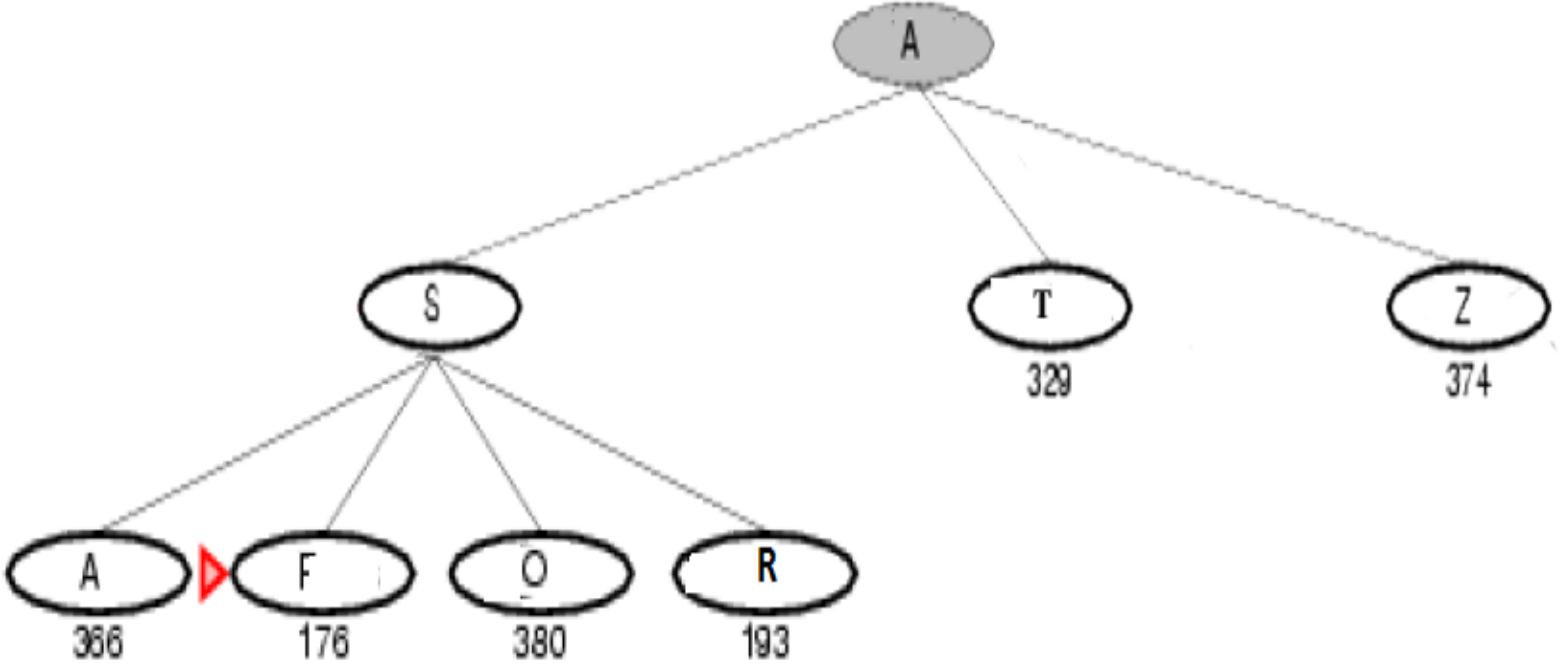
Step 1: Initial State



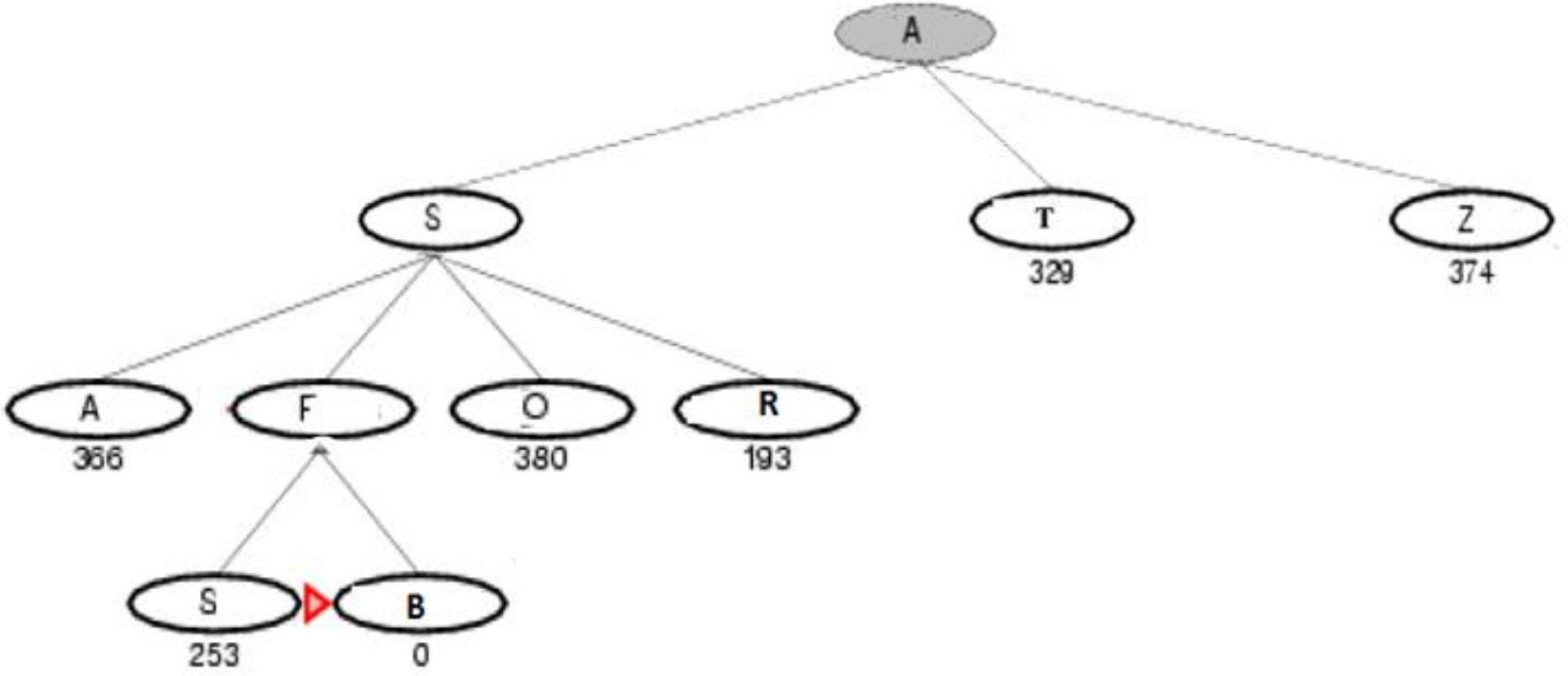
Step 2: After expanding A



Step 3: After expanding S



Step 3: After expanding F



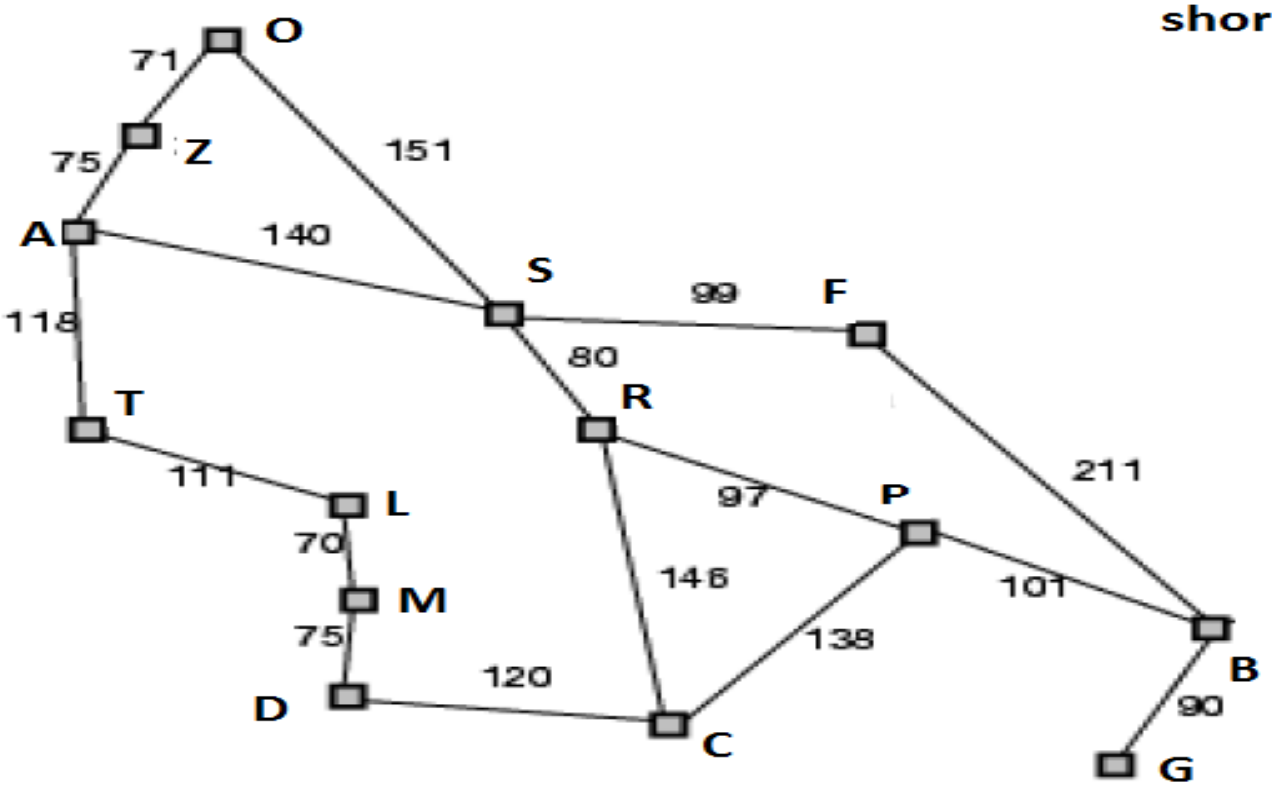
A* Search

- It finds a minimal cost-path joining the start node and a goal node for node n.
- Evaluation function: f(n) = g(n) + h(n)

  Where,
  - g(n) = cost so far to reach n from root
  - h(n) = estimated cost to goal from n

  f(n) = estimated total cost of path through n to goal.
- Avoid expanding paths that are already expensive
- The main drawback of A* algorithm and indeed of any best-first search is its memory requirement.

A* search example (Find path from A to B)



shortest line distance to B

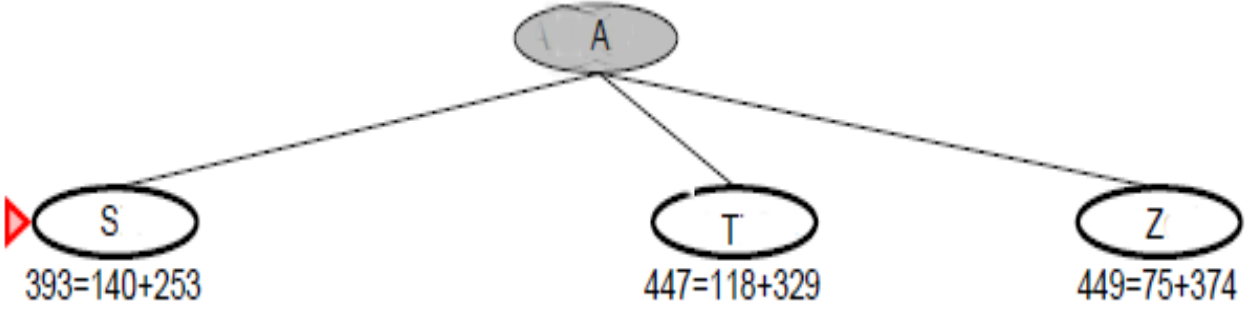
A	366
B	0
C	160
D	242
F	176
G	77
L	244
M	241
O	380
P	10
R	193
S	253
T	329
Z	374

Here, evaluate nodes connected to source. Evaluate f(n)=g(n)+h(n) for each node. Select node with lowest f(n) value.

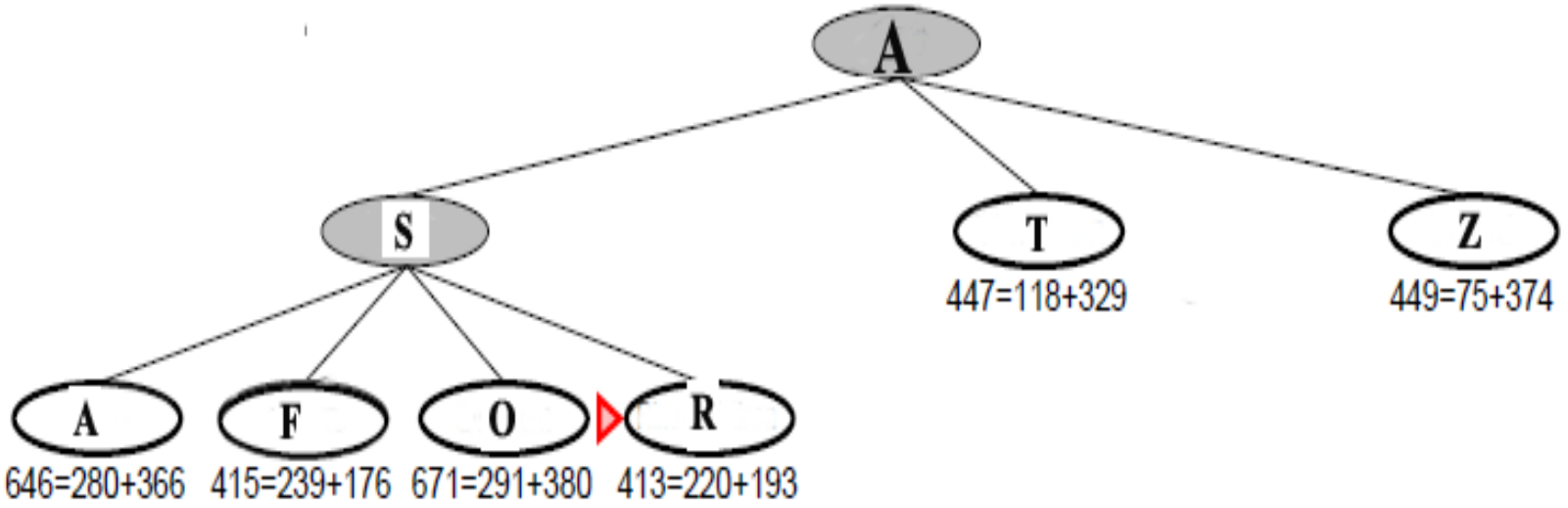
Step 1:



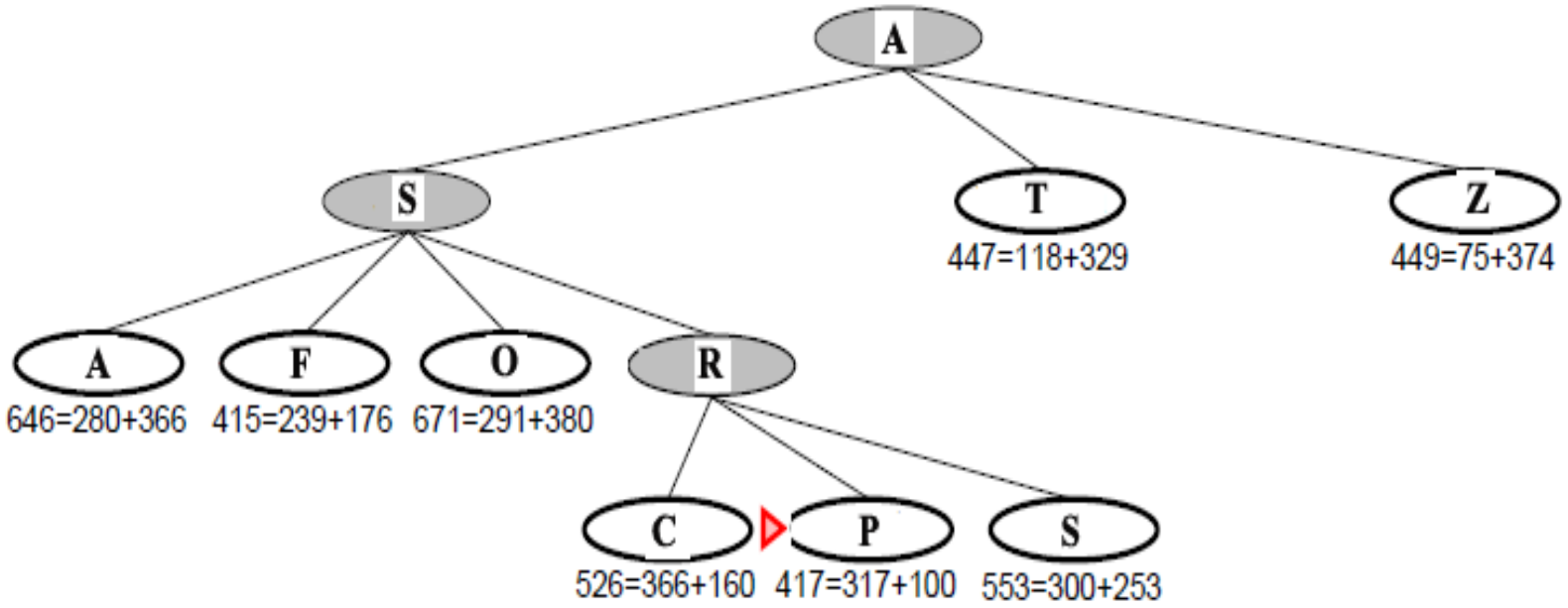
Step 2



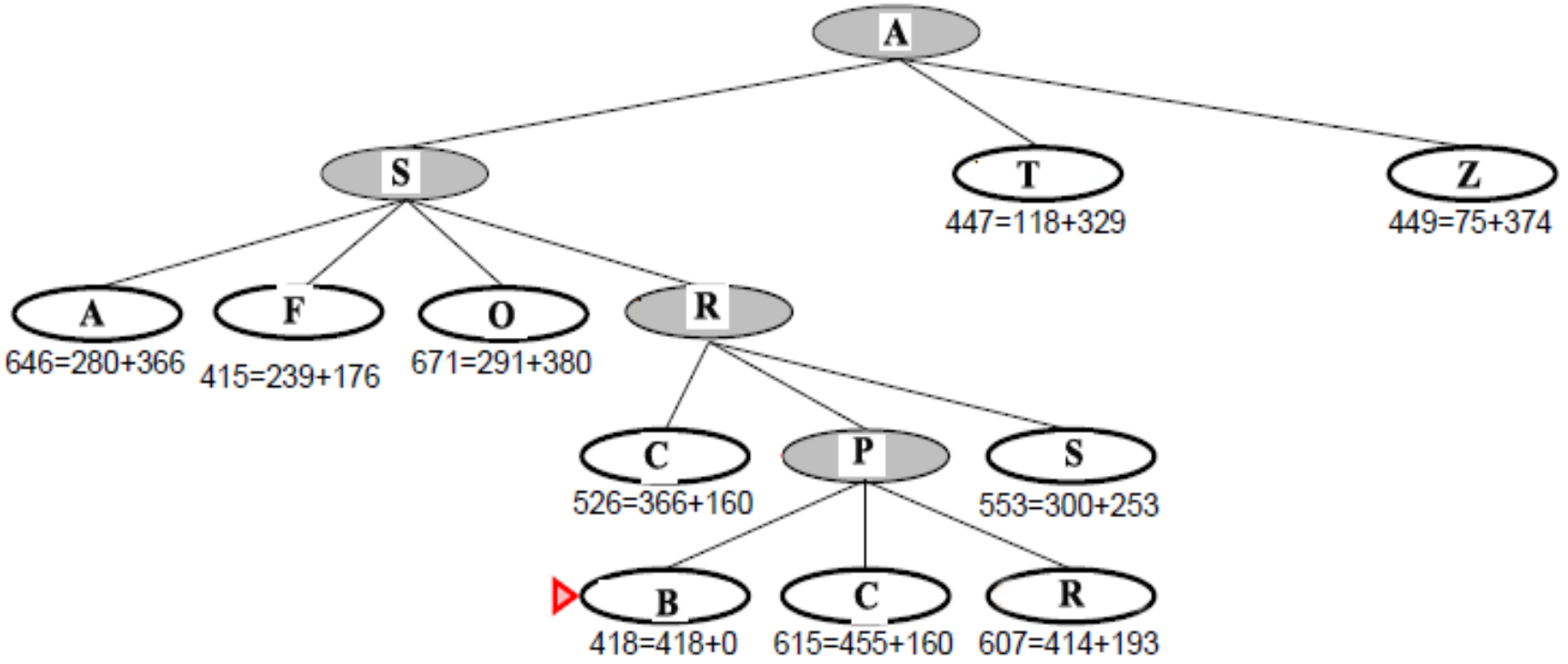
Step 3:


A
S
T
Z
447=118+329
449=75+374
A
F
O
R
646=280+366
415=239+176
671=291+380
413=220+193

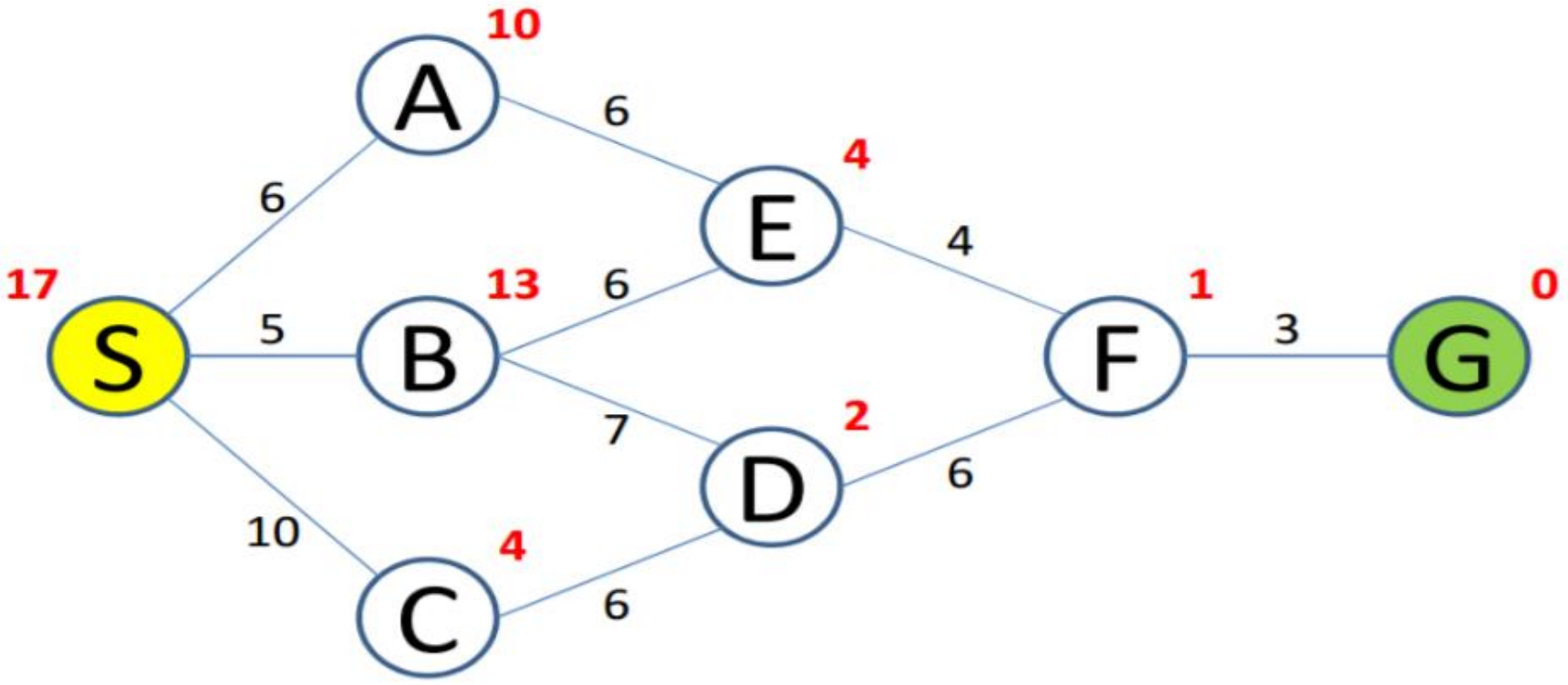
Step 4:


A
S
T
447=118+329
Z
449=75+374
A
646=280+366
F
415=239+176
O
671=291+380
R
C
526=366+160
P
417=317+100
S
553=300+253

Step 5:

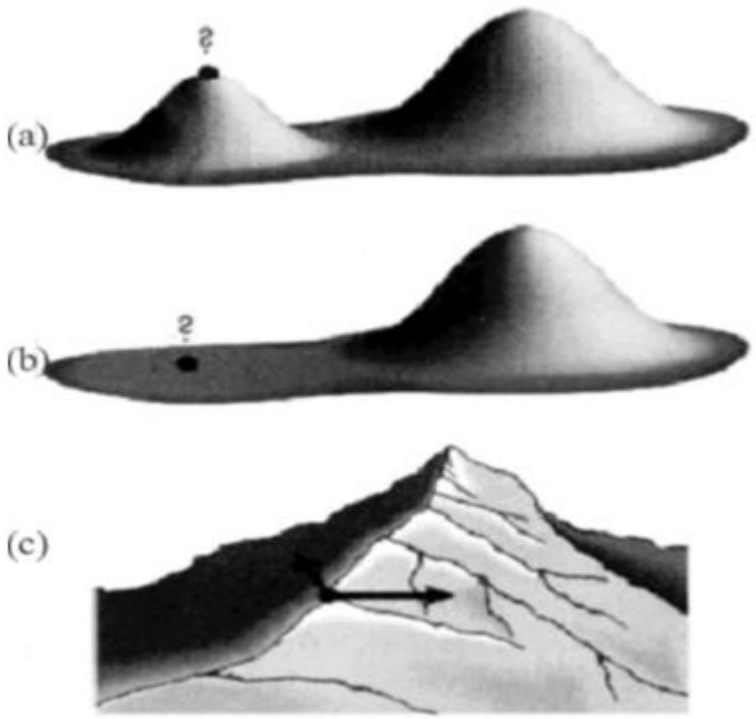

A
S
T
447=118+329
Z
449=75+374
A
646=280+366
F
415=239+176
O
671=291+380
R
C
526=366+160
P
S
553=300+253
B
418=418+0
C
615=455+160
R
607=414+193

Perform Greedy and A* Search



Hill Climbing Search

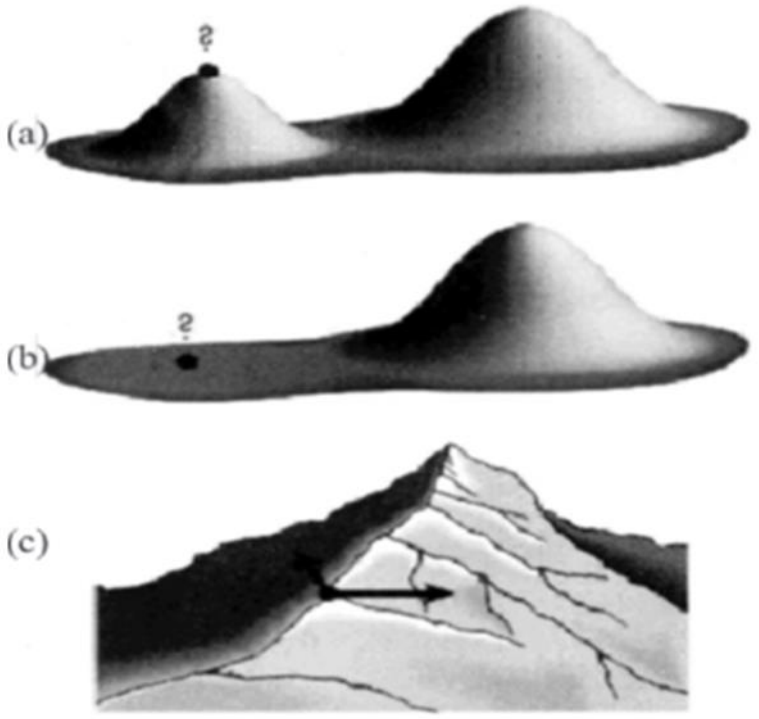
- Hill climbing is an extension of depth-first search which uses some knowledge such as estimates of the distance of each node from the goal to improve the search
- It is simply a loop that continually moves in the direction of increasing value—that is, uphill. It terminates when it reaches a “peak” where no neighbor has a higher value.
- Hill climbing is sometimes called greedy local search because it grabs a good neighbor state without thinking ahead about where to go next



Hill Climbing Search

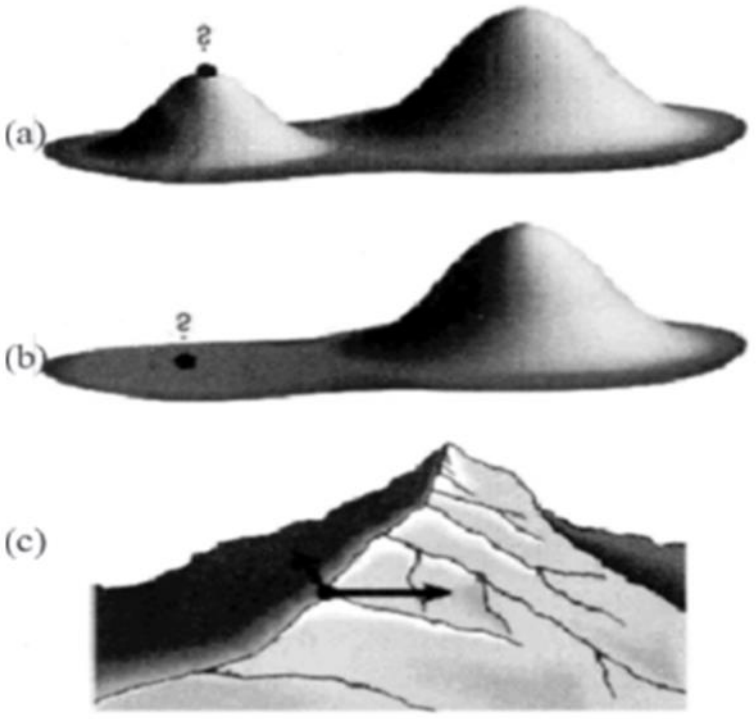
- Drawbacks
  - Local Maxima

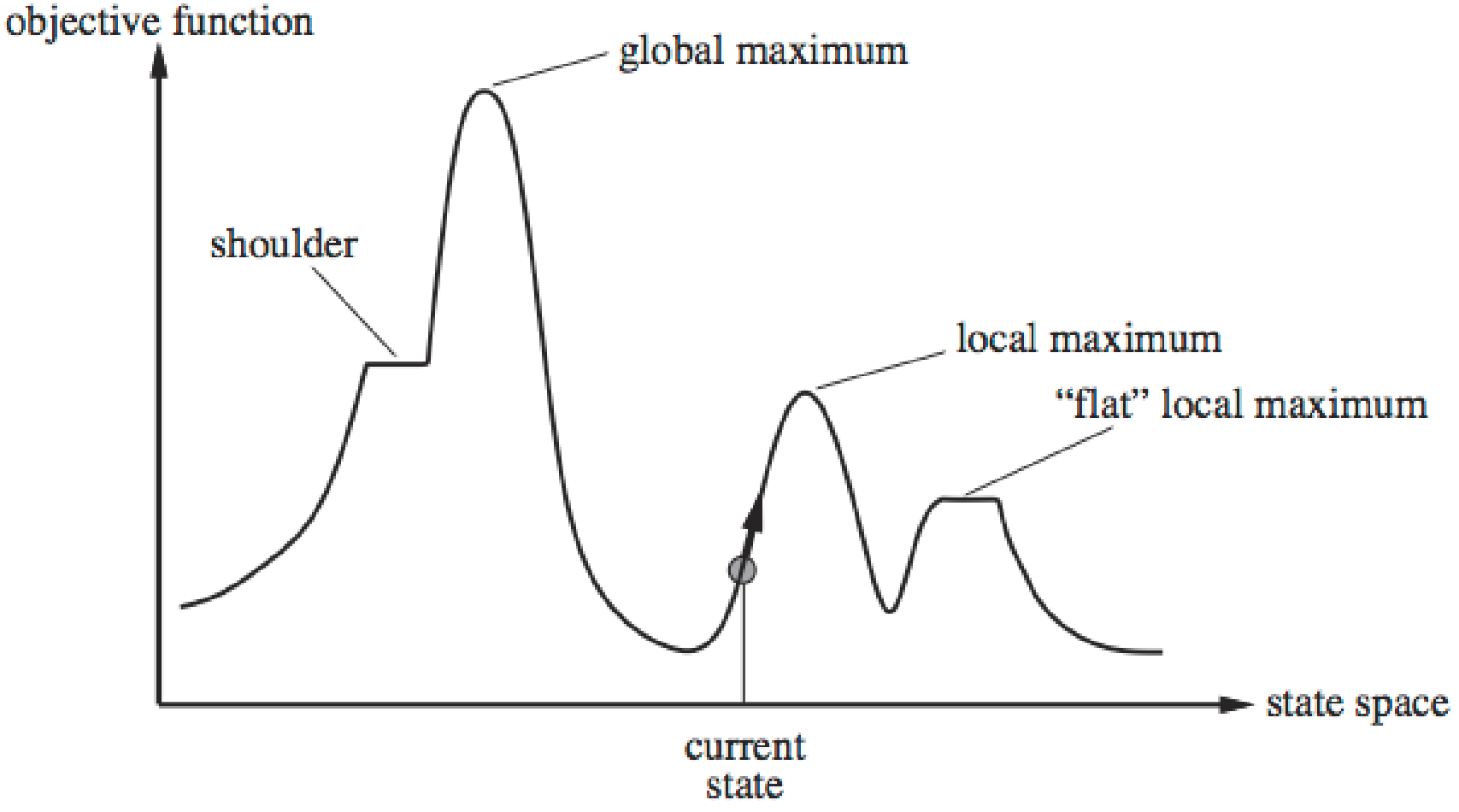
    A local maximum is a peak that is higher than each of its neighboring states but lower than the global maximum.
  - Plateaus: A plateau is an area of the search space where evaluation function is flat, thus requiring random walk
  - Ridges: Where there are steep slopes and the search direction is not towards the top but towards the side



Hill Climbing Search

- **Remedies**
  - **Back tracking for local maximum:**
    The back tracking help in undoing what is been done so far and permit to try totally different part to attain the global peak.
  - **Big Jump:**
    A big jump is the solution to escape from plateaus because all neighbors' points have same value using the greedy approach
  - **Random restart:**
    Keep restarting the search from random locations until a goal is found

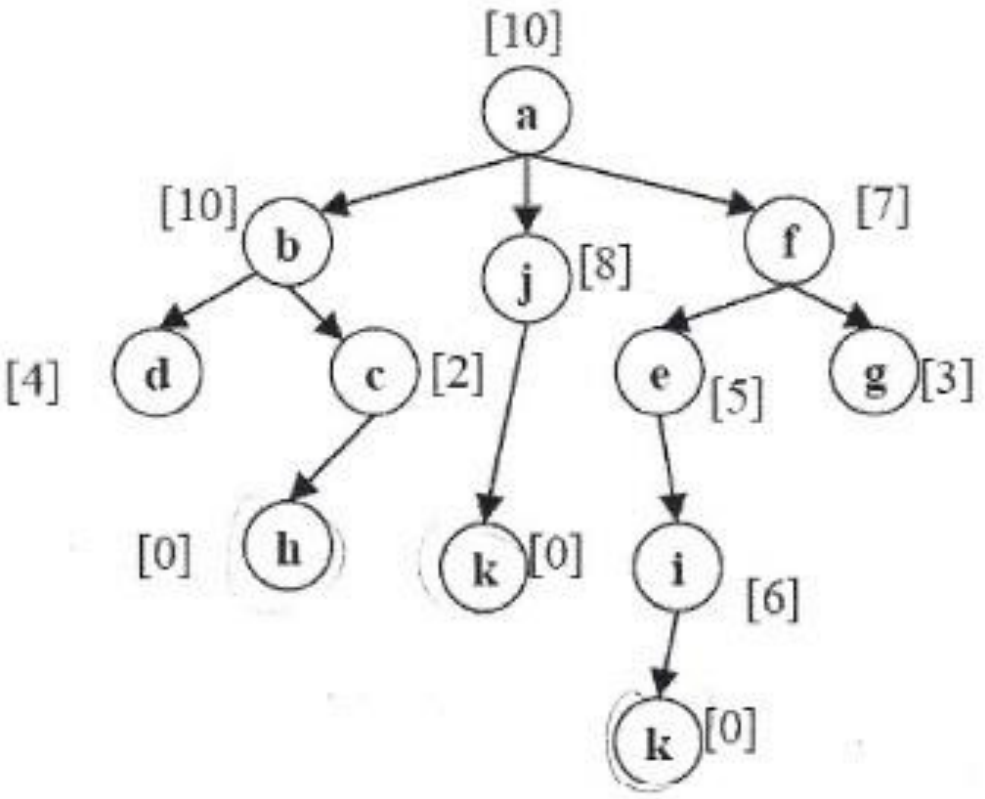



objective function
global maximum
shoulder
local maximum
"flat" local maximum
state space
current
state

Why Hill Climbing is not Complete?

- Hill-climbing always attempts to make changes that improve the current state.
- The main problem that hill climbing can encounter is that of local maxima. This occurs when the algorithm stops making progress towards an optimal solution; mainly due to the lack of immediate improvement in adjacent states.

Problems in Hill Climbing



Here, "a" is initial and h and k are final states

- We start a-> f-> g and then what ??finish(without result)
- A common way to avoid getting stuck in local maxima with Hill Climbing is to use random restarts. In your example if G is a local maxima, the algorithm would stop there and then pick another random node to restart from. So if J or C were picked (or possibly A, B, or D), this would find the global maxima in H or K

Adversarial Search

- Competitive environments in which the agents goals are in conflict, give rise to adversarial (oppositional) search, often known as games.
- In AI, games means fully observable environments in which there are two agents whose actions must alternate and in which utility values at the end of the game are always equal and opposite.

E.g. If first player wins, the other player necessarily loses.

Mini max Algorithm

- It is a recursive algorithm for choosing the next move in a n-player game, usually a two player game
- The value is computed by means of a position evaluation function and it indicates how good it would be for a player to reach the position.
- The player then makes the move that maximizes the minimum value of the position from the opponents possible moves called maximizing player and other player minimize the maximum value of the position called minimizing player.

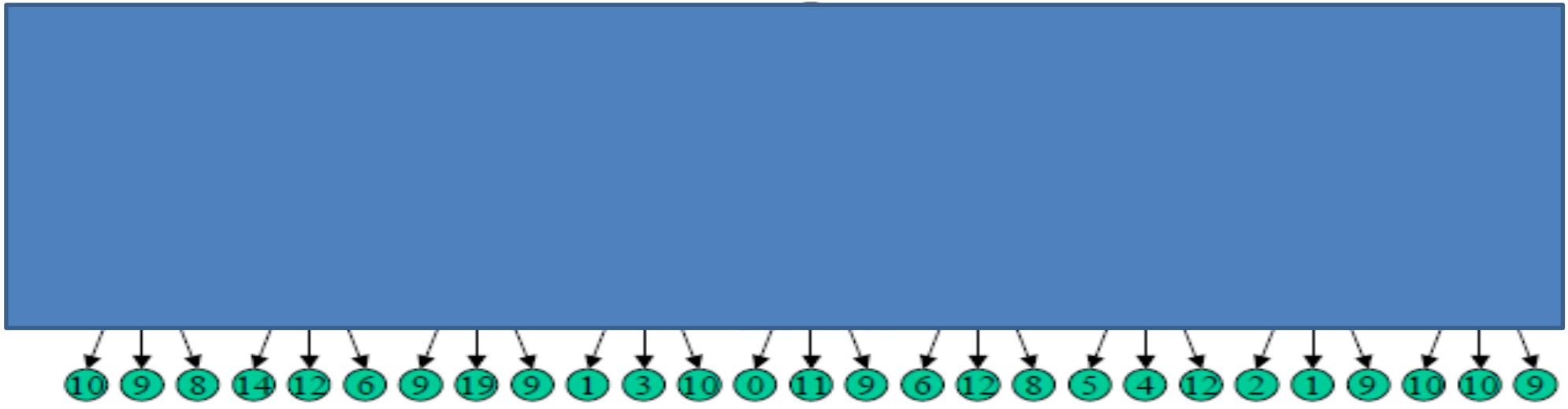
Mini Max Game Search

- It is a Depth-first search with limited depth.
- Assume the opponent will make the best move possible.
- Algorithm
  - *mini max (player, board)*
  - *if(game over in current board position)*
    return winner
  - *if(max's turn)*
    return maximal score of calling minimax
    else (min's turn)
    return minimal score of calling minimax

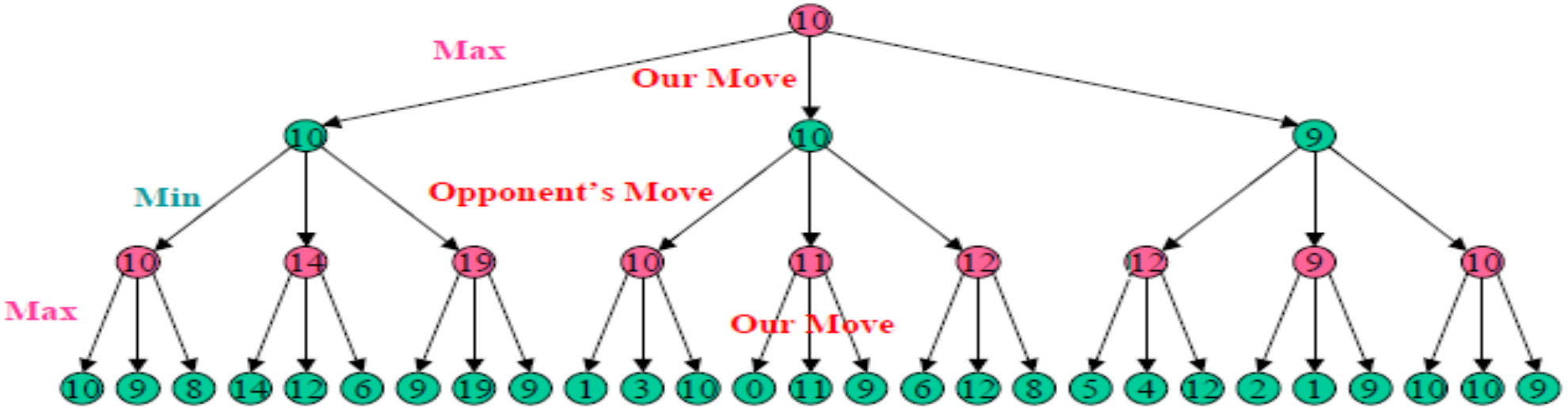
Properties of MiniMax

- ***Complete?*** Yes (if tree is finite)
- ***Optimal?*** Yes (against an optimal opponent)
- ***Time complexity?*** $O(b^m)$
- ***Space complexity?*** $O(b^m)$ (depth-first exploration)

We first consider games with two players; MAX and MIN. MAX moves first, and then they take turns moving until the game is over. Each level of the tree alternates, MAX is trying to maximize score, and MIN is trying to minimize MAX score in order to undermine success



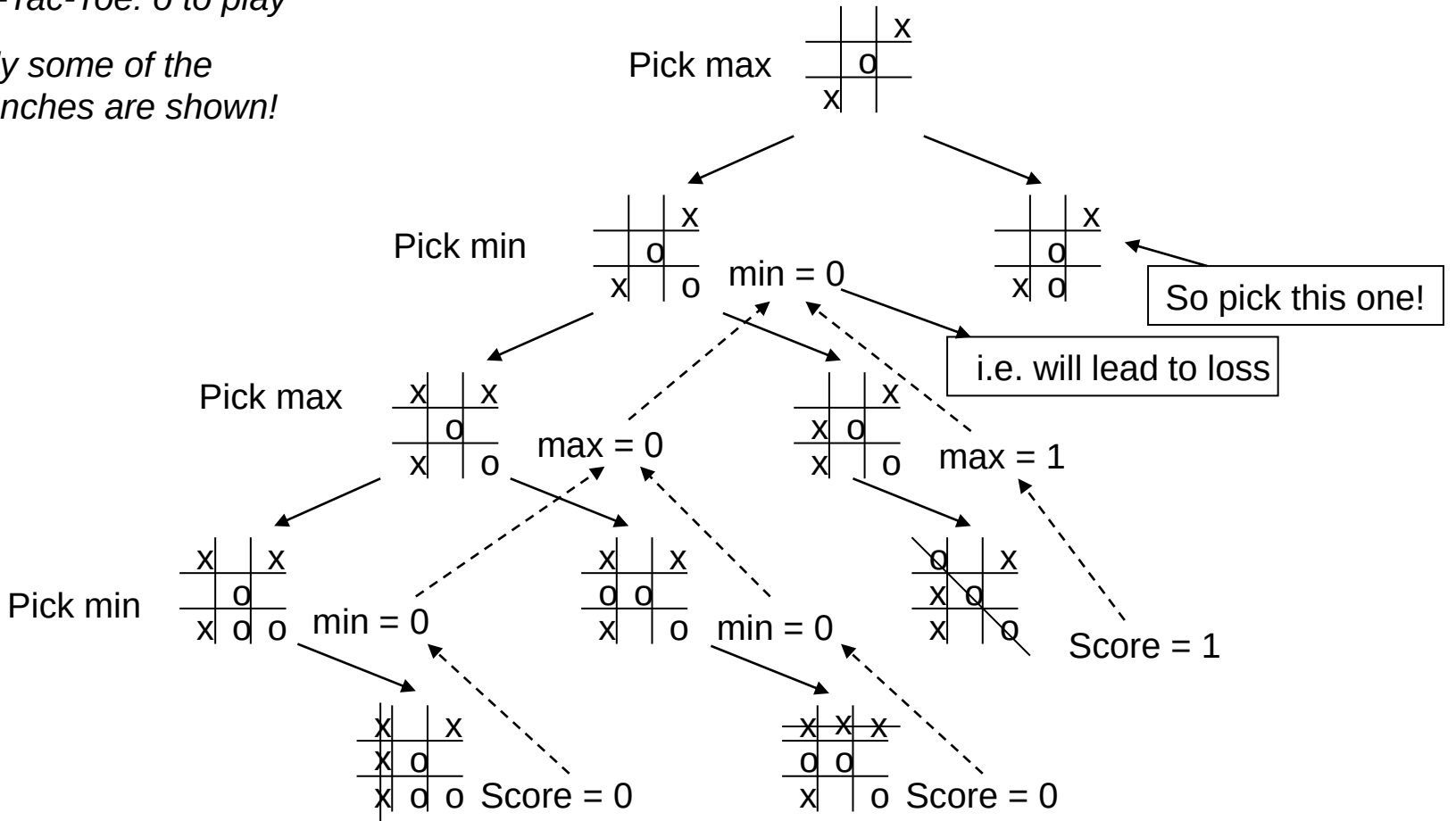
We first consider games with two players; MAX and MIN. MAX moves first, and then they take turns moving until the game is over. Each level of the tree alternates, MAX is trying to maximize score, and MIN is trying to minimize MAX score in order to undermine success



Example Max-Min Strategy

Tic-Tac-Toe: o to play

Only some of the Branches are shown!



The mini max algorithm returns the best move for MAX under the assumption that MIN play optimally. What happens when MIN plays sub optimally ?

Optimality is still well defined, even if the opponent isn't playing well. Moreover, if the game tree is small enough that the agent can fully explore it, then the optimal player really doesn't care what the other one does.

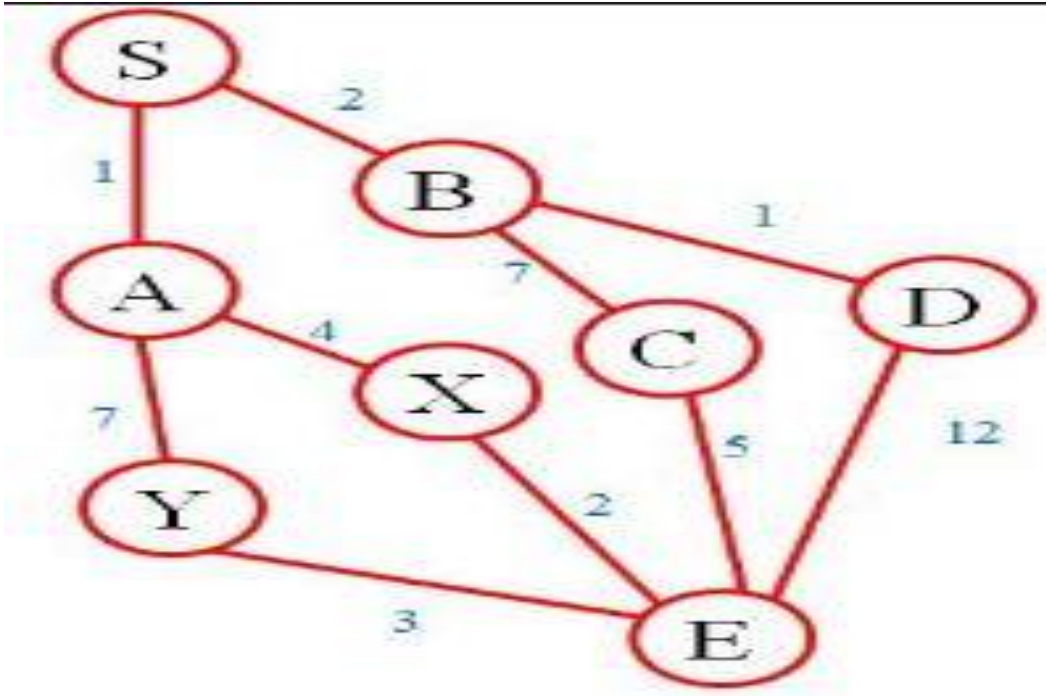
Let's say Max goes first. What will Max do? He will look at every possible game sequence. He will then take the action which guarantees that he will get a score of X. No matter what Min does in subsequent moves, Min can never get a score less than X.

CLASSWORK

There is a farmer who wishes to cross a river but he is not alone. He also has a goat, a wolf, and a cabbage along with him. There is only one boat available which can support the farmer and either of the goat, wolf or the cabbage. So at a time, the boat can have only two objects (farmer and one other).
But the problem is, if the goat and wolf are left alone (either in the boat or onshore), the wolf will eat the goat. Similarly, if the Goat and cabbage are left alone, then goat will eat the cabbage. The farmer wants to cross the river with all three of his belongings: goat, wolf, and cabbage.
What strategy he should use to do so? Represent the state space for the problem.

Solve the following Crypt-Arithmetic problem

```
  LOGIC
 +LOGIC
 ……..
PROLOG
```

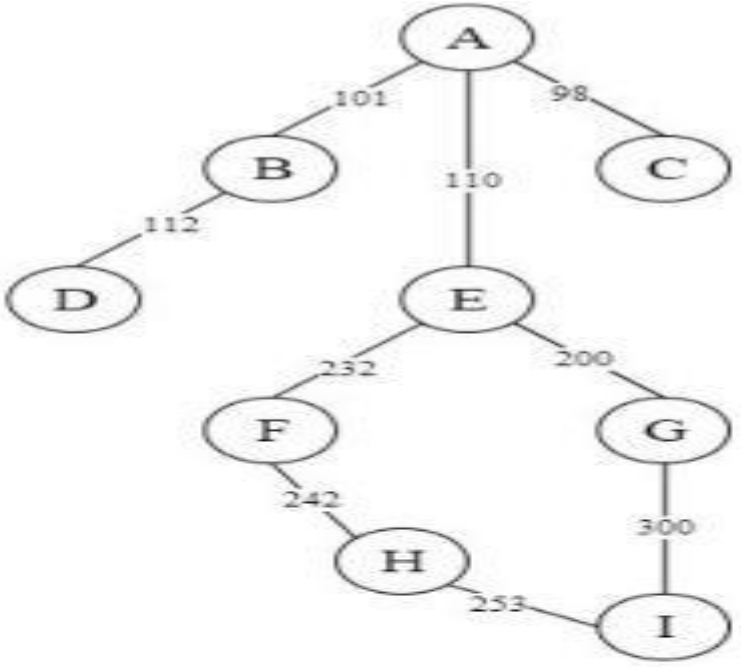


- Values for h:

  A:5, B:6, C:4, D:15, X:5, Y:8

Perform Greedy BFS and A* search from S to E

Use A* search to find path cost from A to I.
Show all possible states.



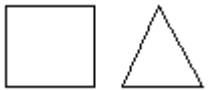
State	Heuristic $h(n)$
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0

Alpha–beta pruning

- Alpha–beta pruning is a search algorithm that seeks to decrease the number of nodes that are evaluated by the mini max algorithm in its search tree.
- It is an adversarial search algorithm used commonly for machine playing of two-player games (Tic-tac-toe, Chess, Go, etc.)
- It stops completely evaluating a move when at least one possibility has been found that proves the move to be worse than a previously examined move. Such moves need not be evaluated further.

β = the value of the best (i.e., highest-value) choice we have found so far at any choice point along the path for MAX.

α = the value of the best (i.e., lowest-value) choice we have found so far at any choice point along the path for MIN.



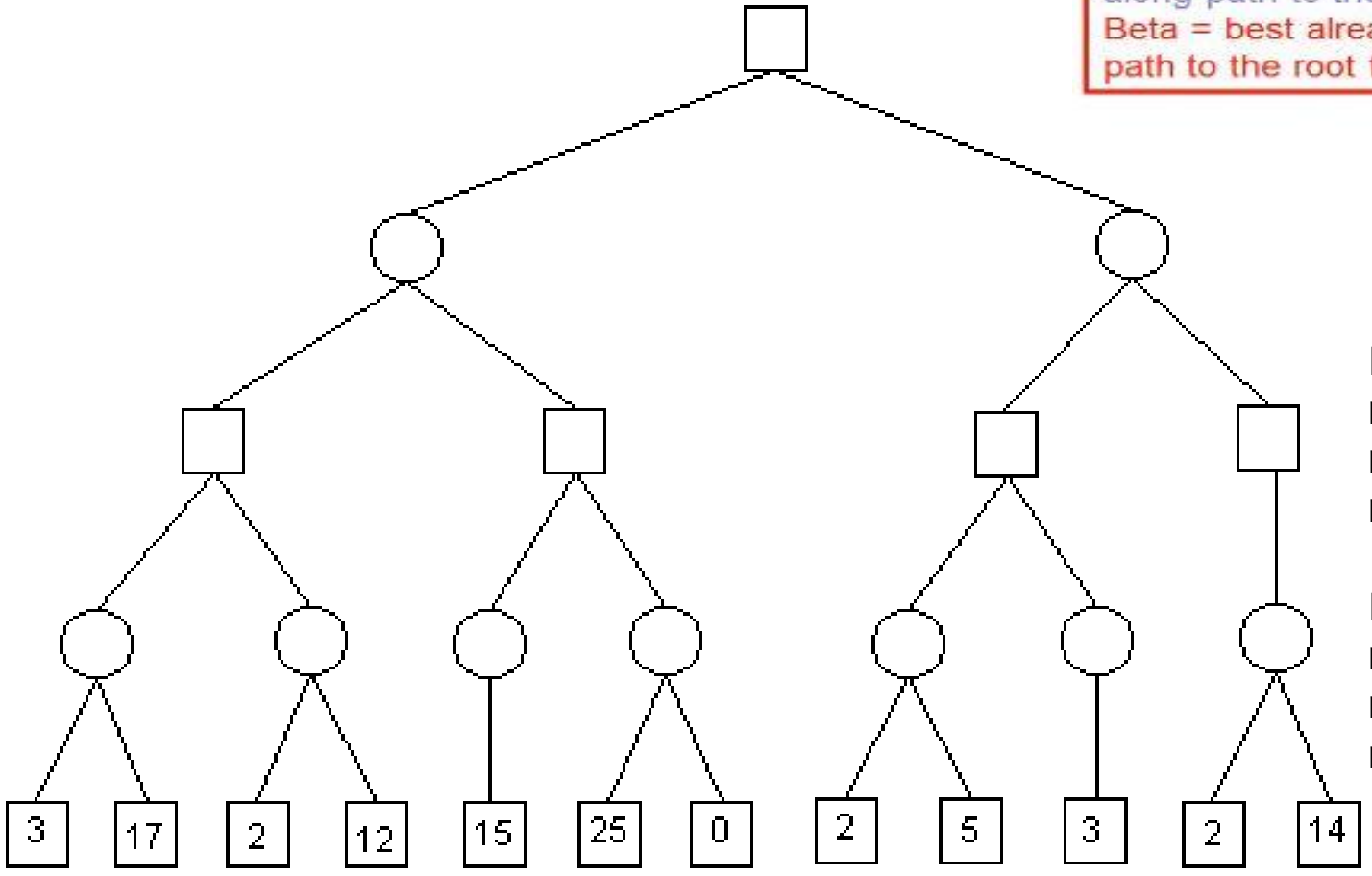
These are also called MAX nodes. The goal at a MAX node is to maximize the value of the subtree rooted at that node. To do this, a MAX node chooses the child with the greatest value, and that becomes the value of the MAX node



These are also called MIN nodes. The goal at a MIN node is to minimize the value of the subtree rooted at that node. To do this, a MIN node chooses the child with the least (smallest) value, and that becomes the value of the MIN node

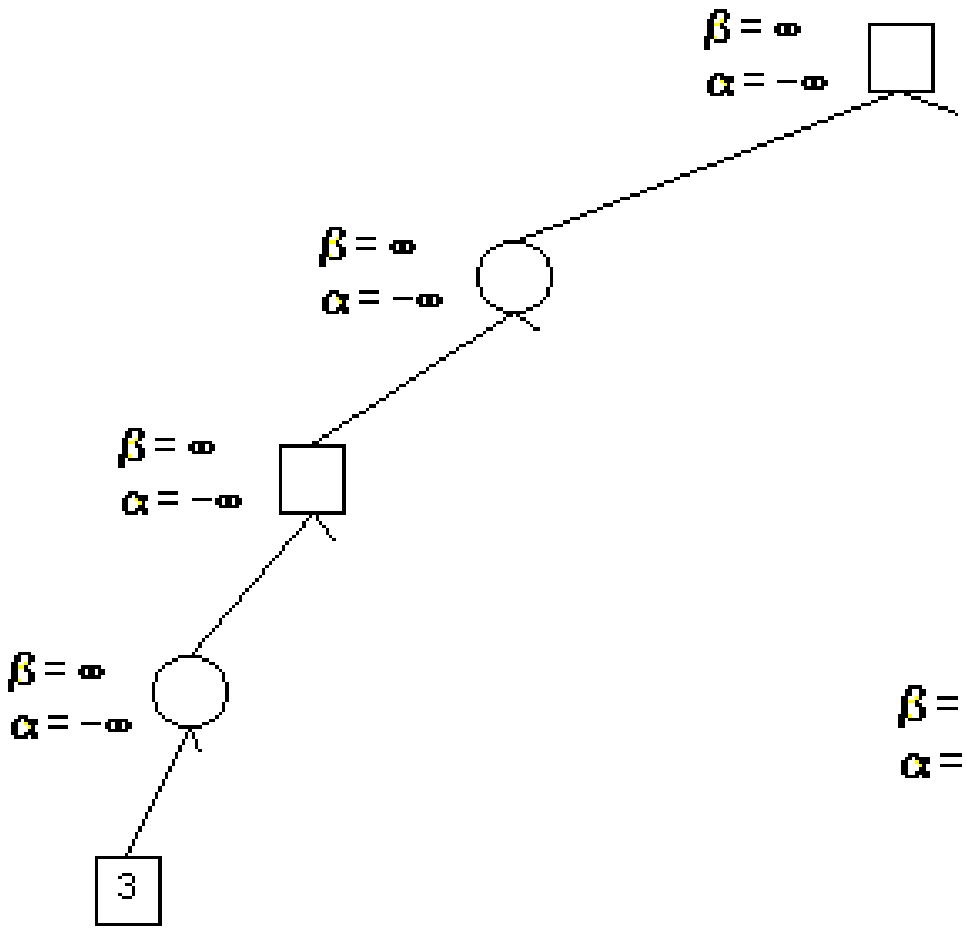
Demonstrate alpha-beta pruning

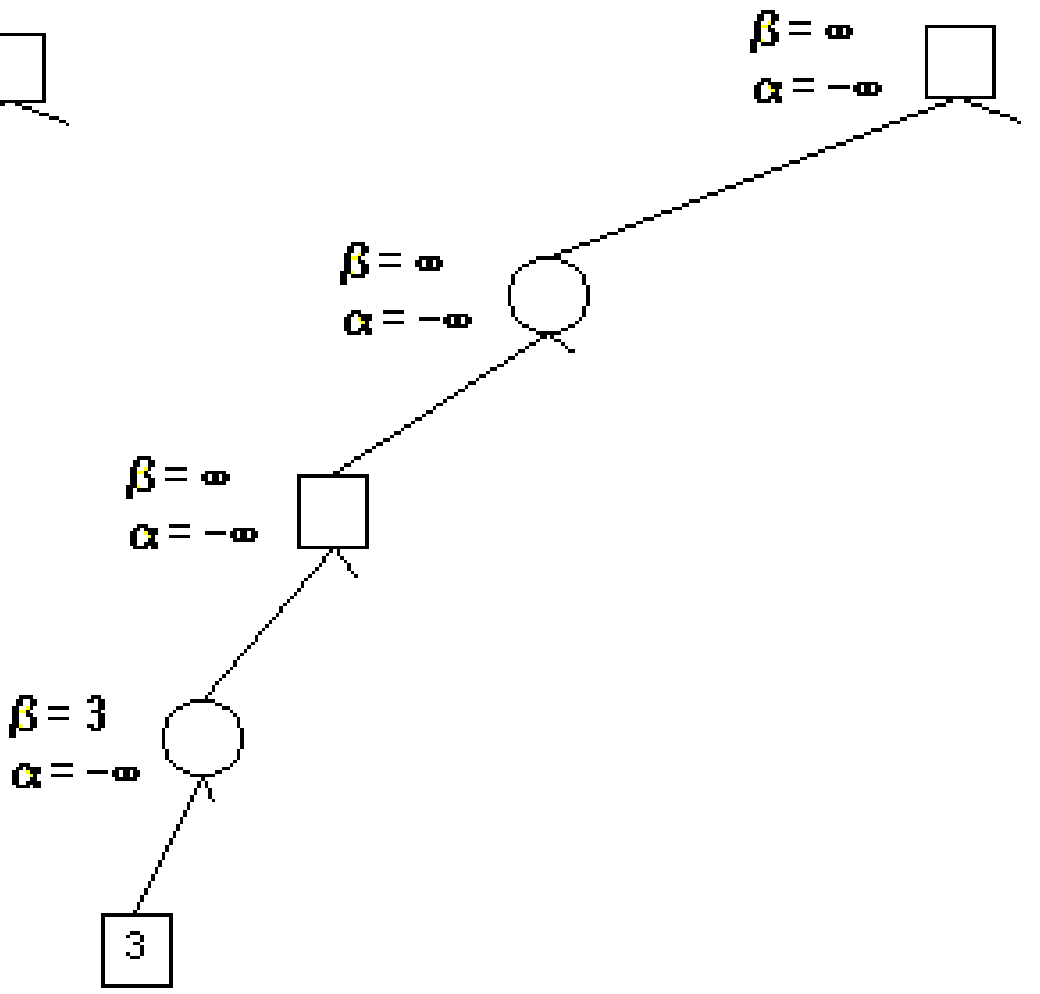
Alpha = best already explored option along path to the root for maximizer
Beta = best already explored option along path to the root for minimizer

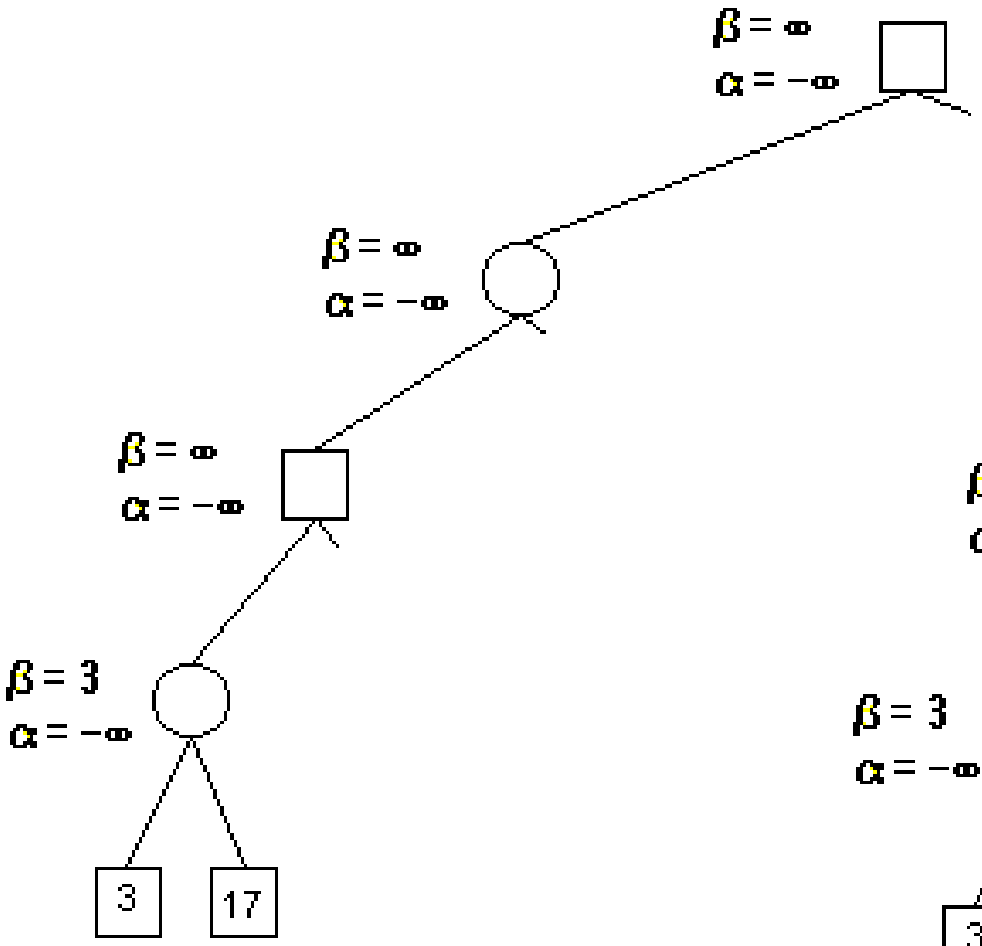


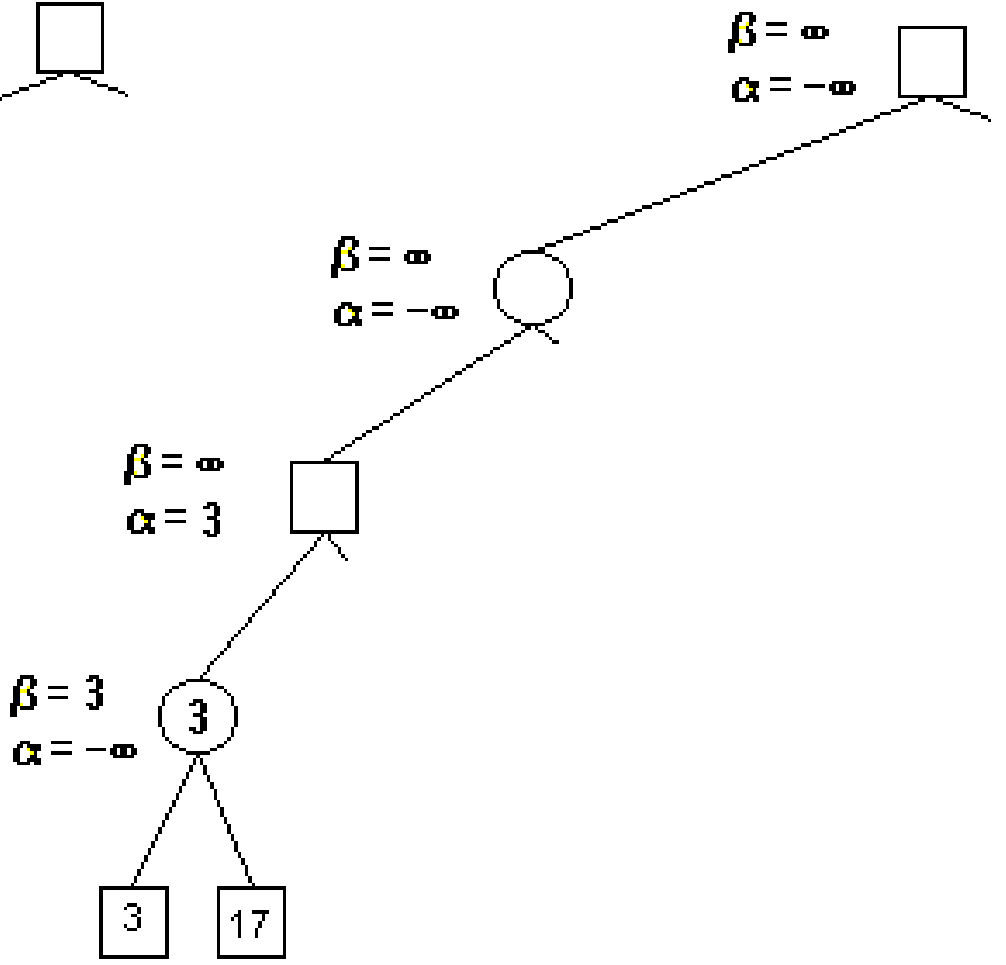
IF V>=beta at maximizer node then no need to explore the next branch in the tree

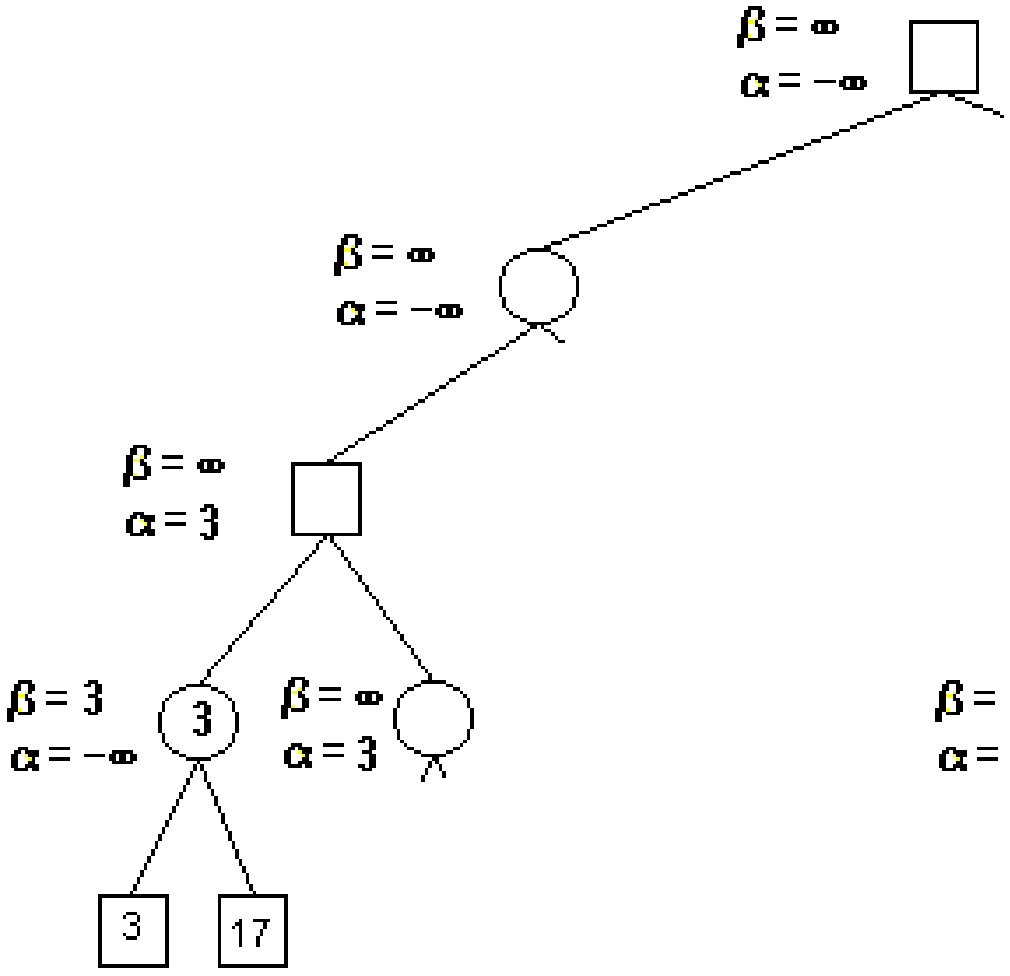
IF V<=alpha at minimizer node then no need to explore the next branch in the tree

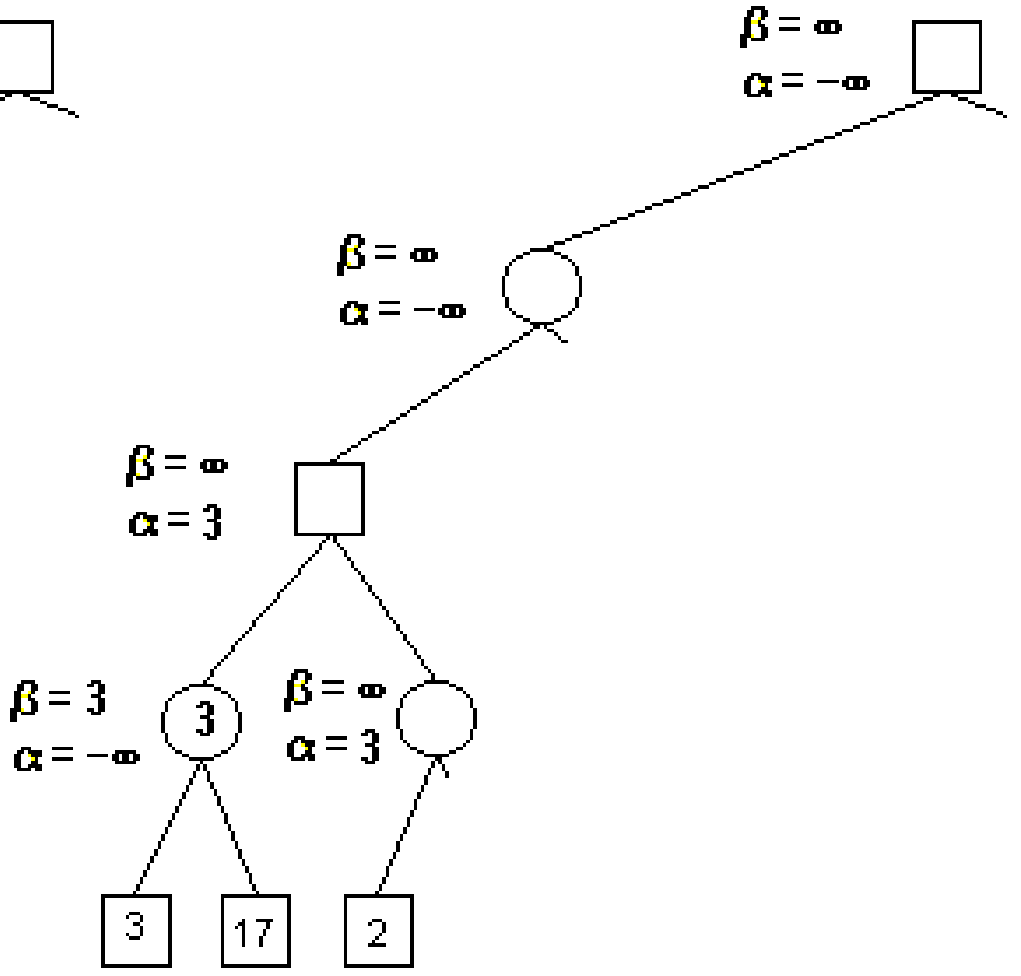

β = ∞
α = −∞
β = ∞
α = −∞
β = ∞
α = −∞
β = ∞
α = −∞
3

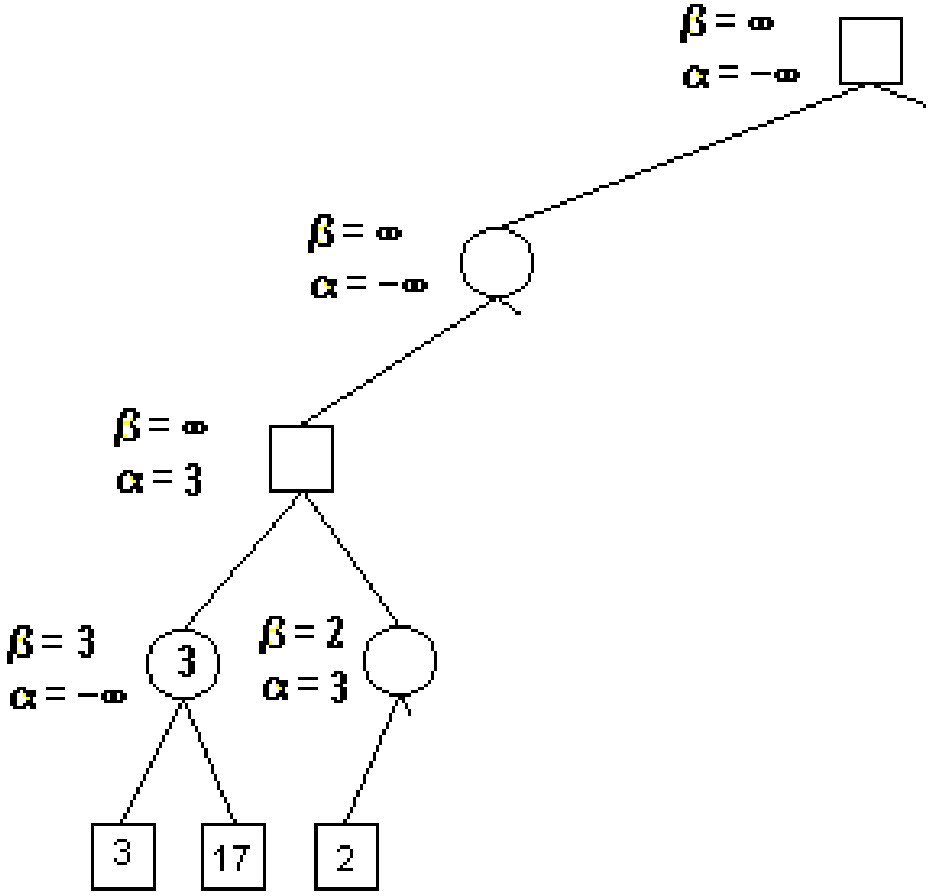

β = ∞
α = −∞
β = ∞
α = −∞
β = ∞
α = −∞
β = 3
α = −∞
3

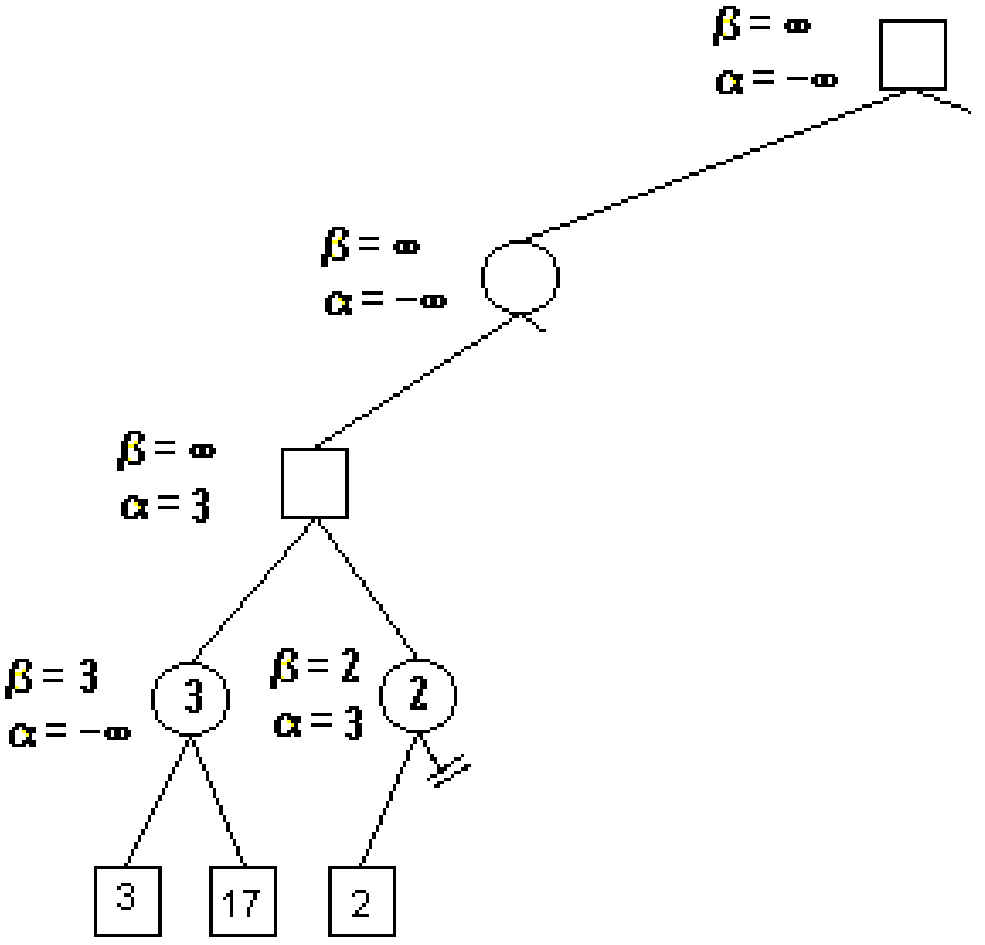

β = ∞
α = −∞
β = ∞
α = −∞
β = ∞
α = −∞
β = 3
α = −∞
3
17

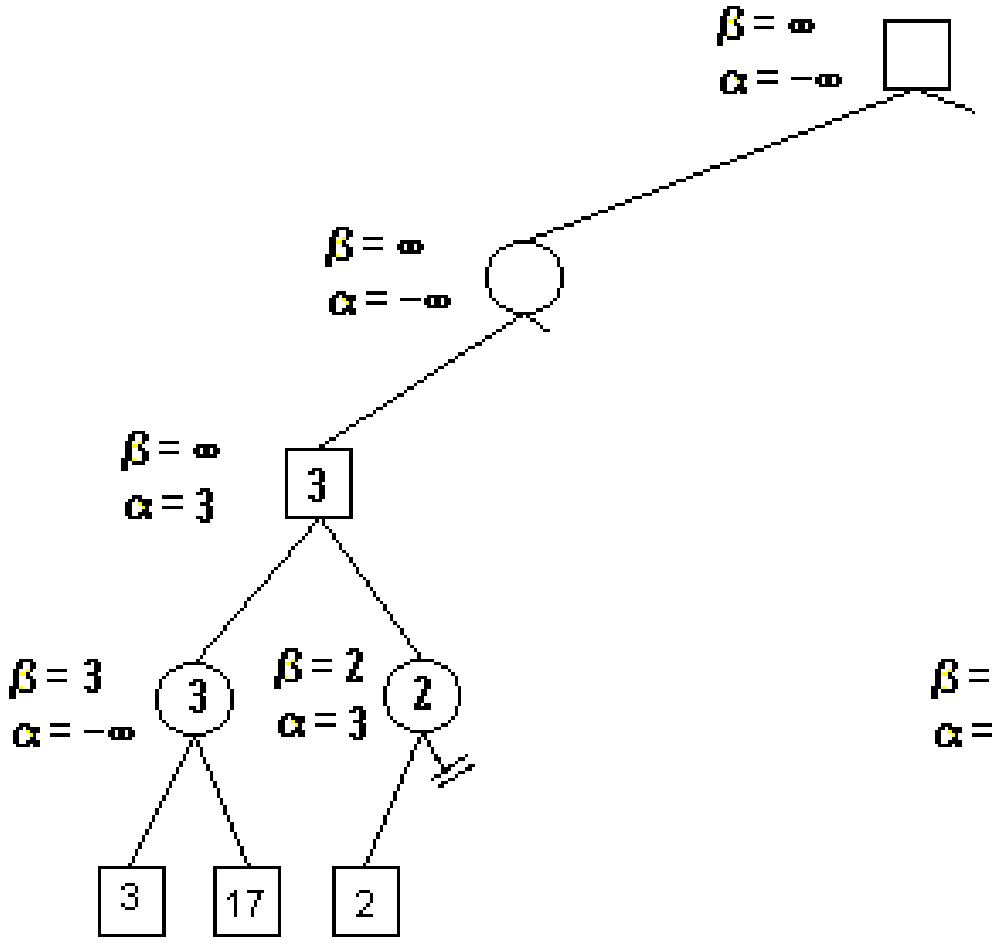

β = ∞
α = −∞
β = ∞
α = −∞
β = ∞
α = 3
β = 3
α = −∞
3
3
17

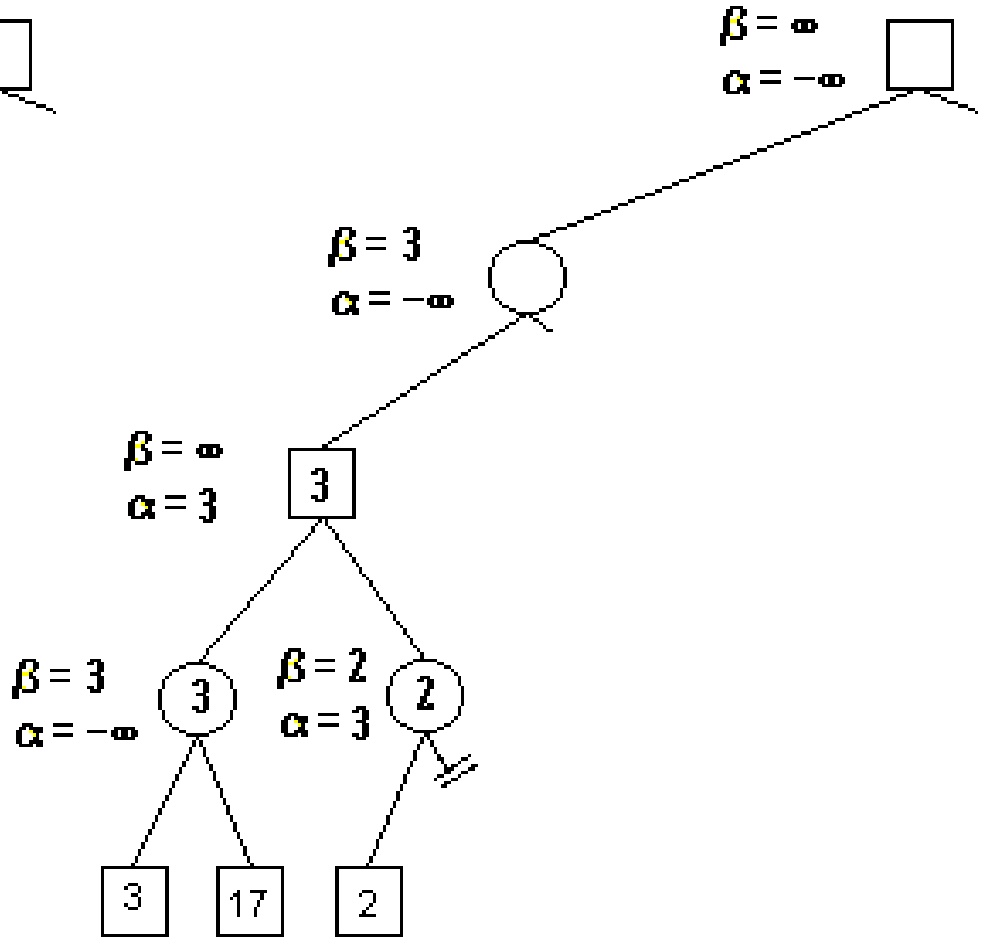

β = ∞
α = −∞
β = ∞
α = −∞
β = ∞
α = 3
β = 3
α = −∞
3
β = ∞
α = 3
3
17

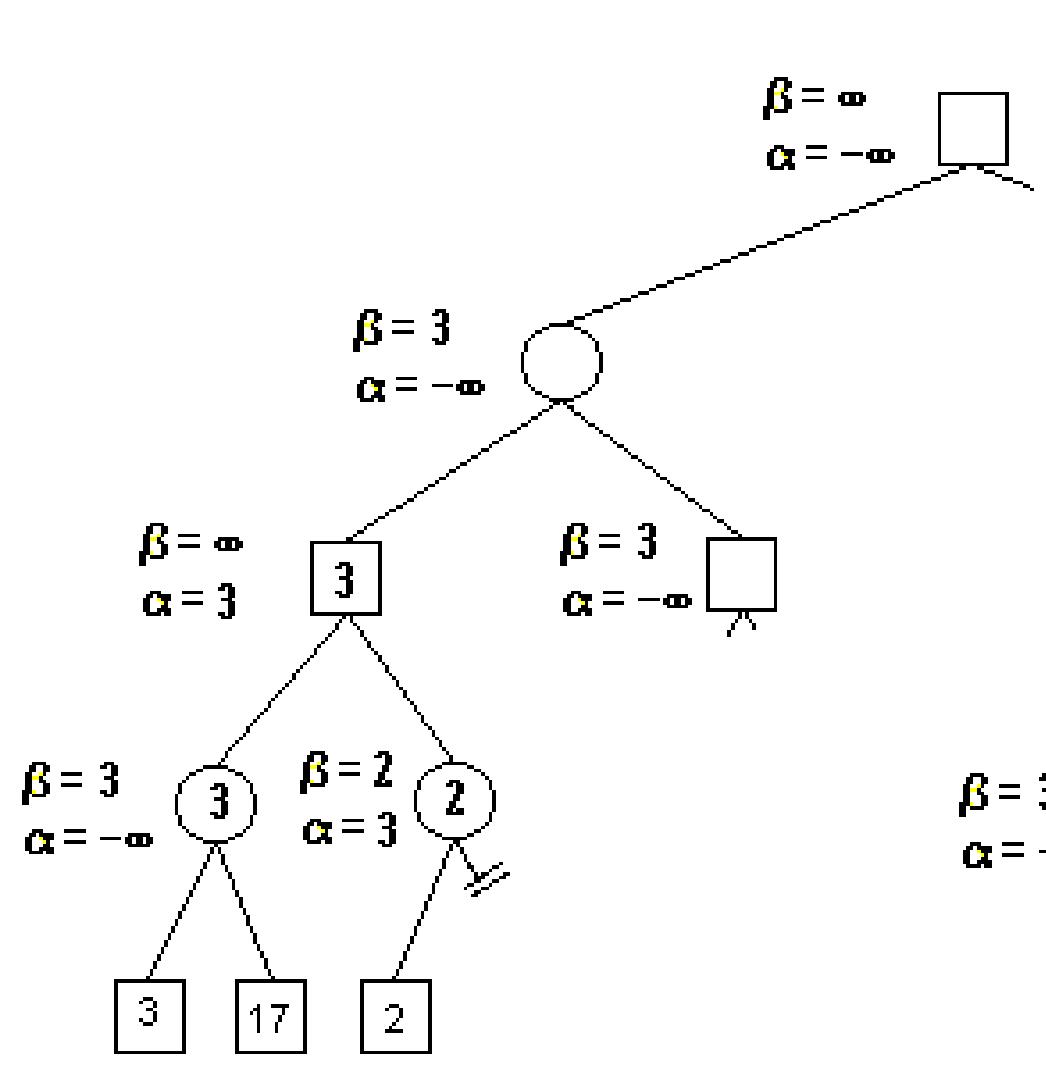

β = ∞
α = −∞
β = ∞
α = −∞
β = ∞
α = 3
β = 3
α = −∞
3
β = ∞
α = 3
3
17
2

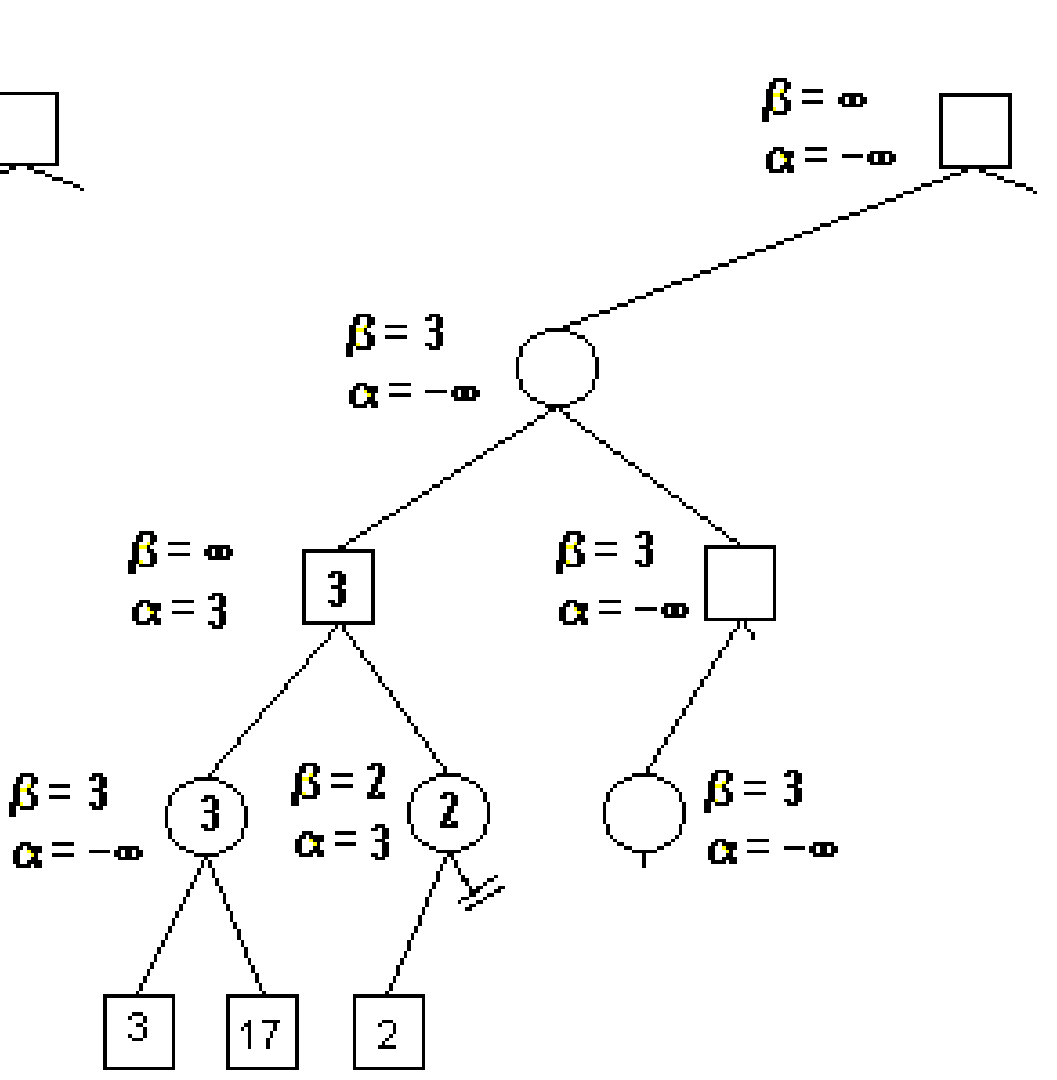

β = ∞
α = −∞
β = ∞
α = −∞
β = ∞
α = 3
β = 3
α = −∞
3
β = 2
α = 3
3
17
2

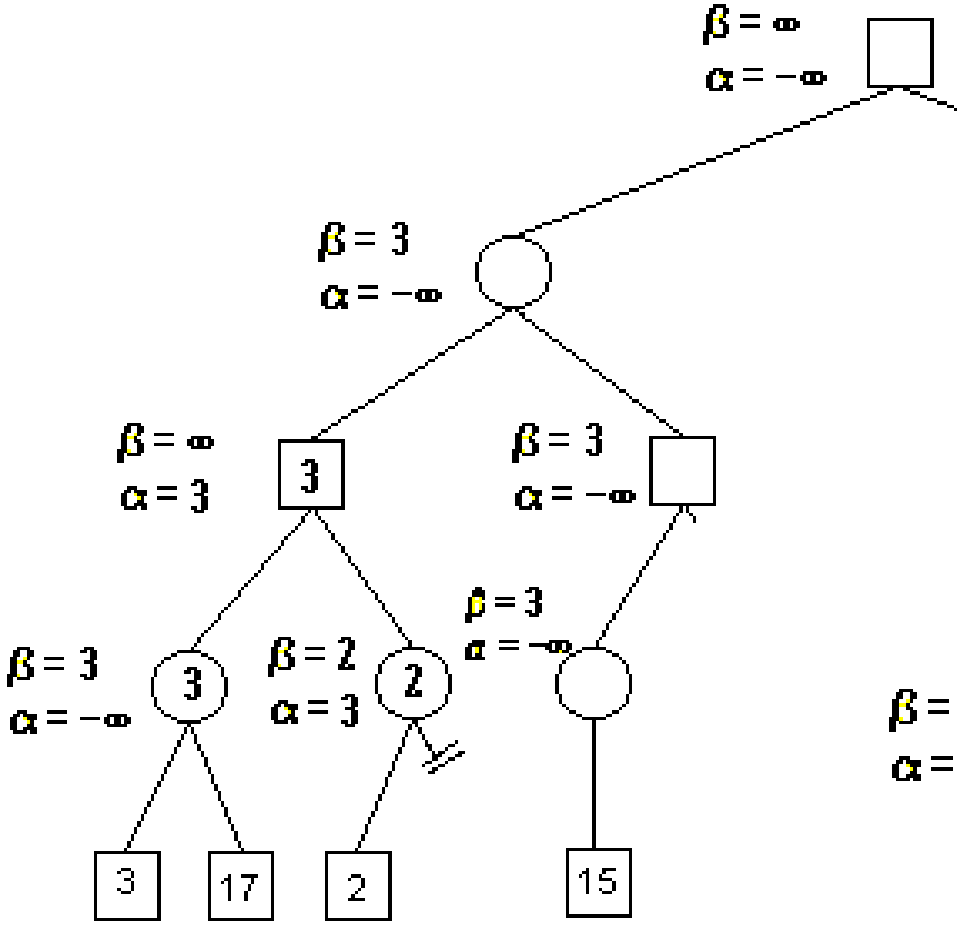

β = ∞
α = −∞
β = ∞
α = −∞
β = ∞
α = 3
β = 3
α = −∞
3
β = 2
α = 3
2
3
17
2

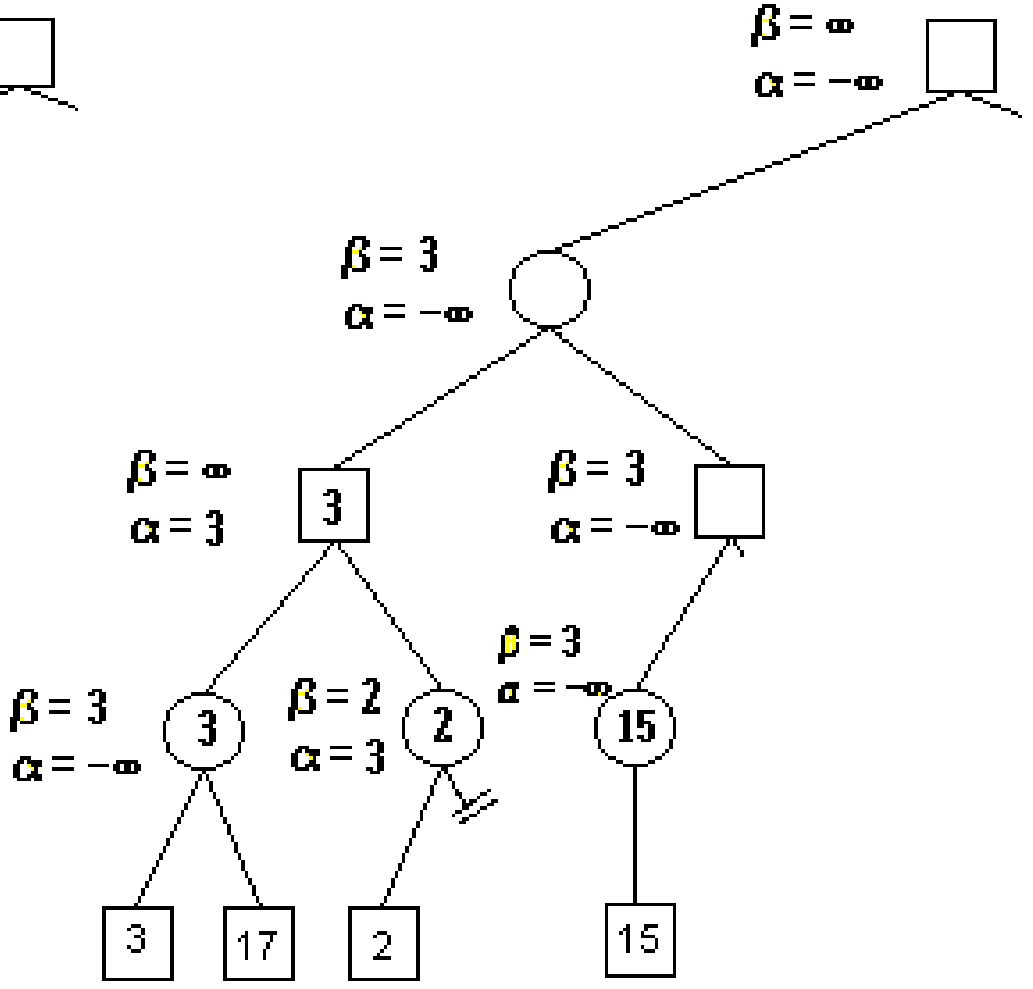

β = ∞
α = −∞
β = ∞
α = −∞
β = ∞
α = 3
3
β = 3
α = −∞
3
β = 2
α = 3
2
3
17
2

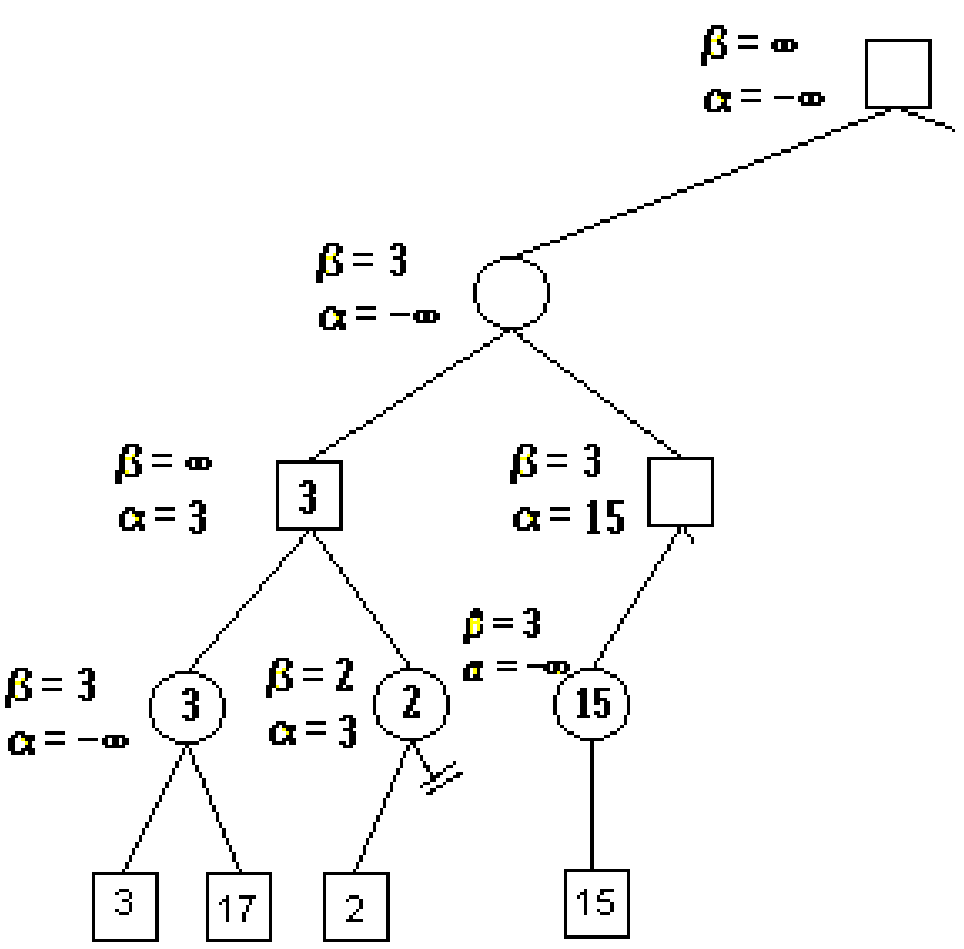

β = ∞
α = −∞
β = 3
α = −∞
β = ∞
α = 3
3
β = 3
α = −∞
3
β = 2
α = 3
2
3
17
2

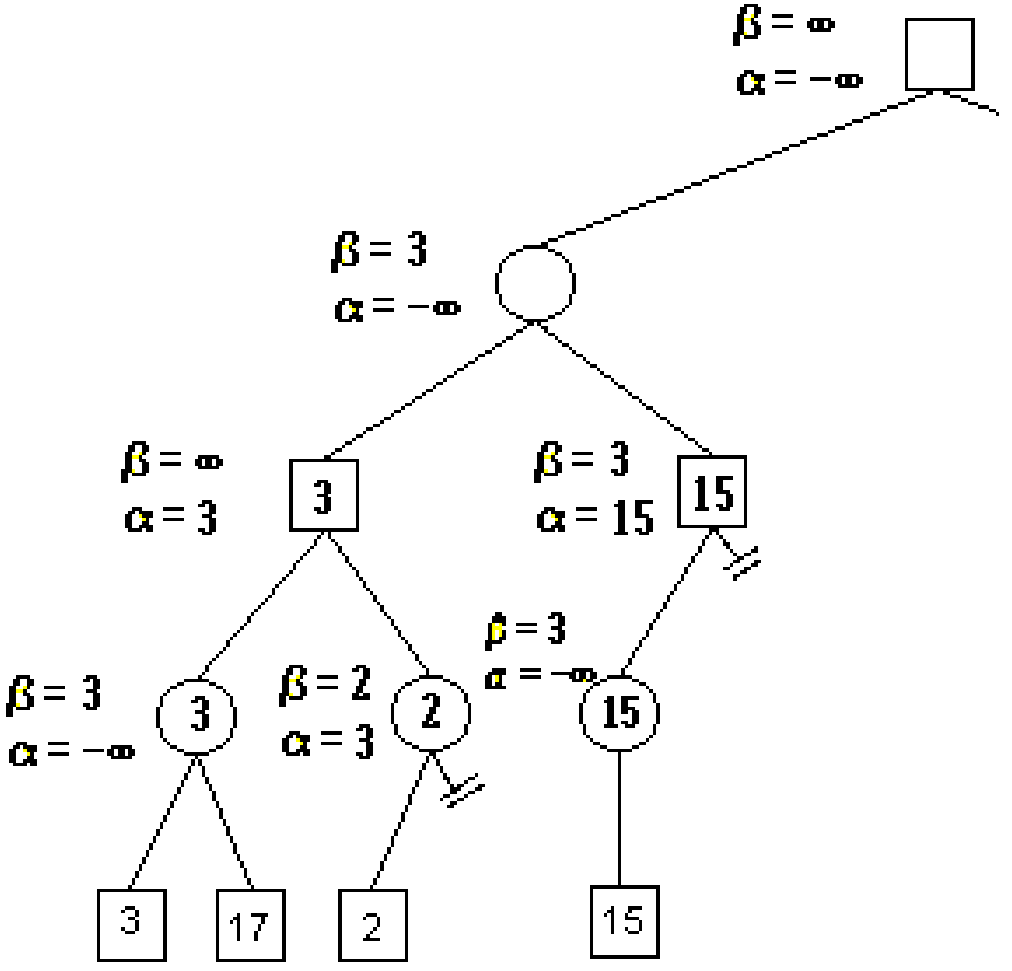

β = ∞
α = −∞
β = 3
α = −∞
β = ∞
α = 3
3
β = 3
α = −∞
β = 3
α = −∞
3
β = 2
α = 3
2
3
17
2

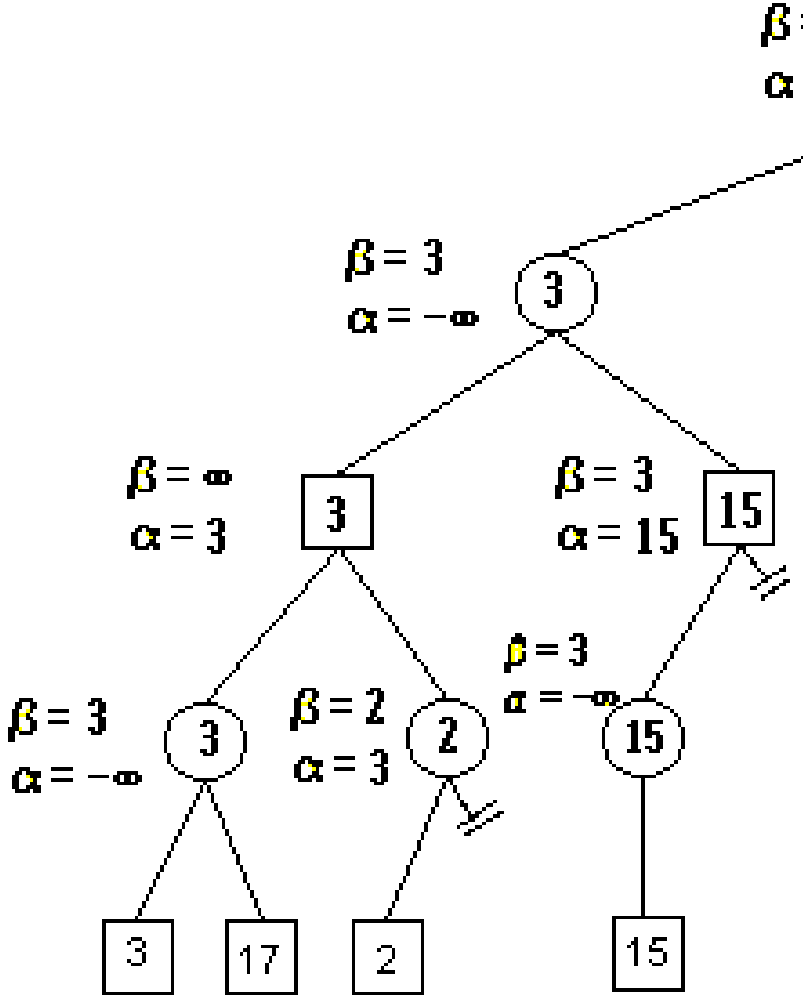

β = ∞
α = −∞
β = 3
α = −∞
β = ∞
α = 3
3
β = 3
α = −∞
β = 3
α = −∞
3
β = 2
α = 3
2
β = 3
α = −∞
3
17
2

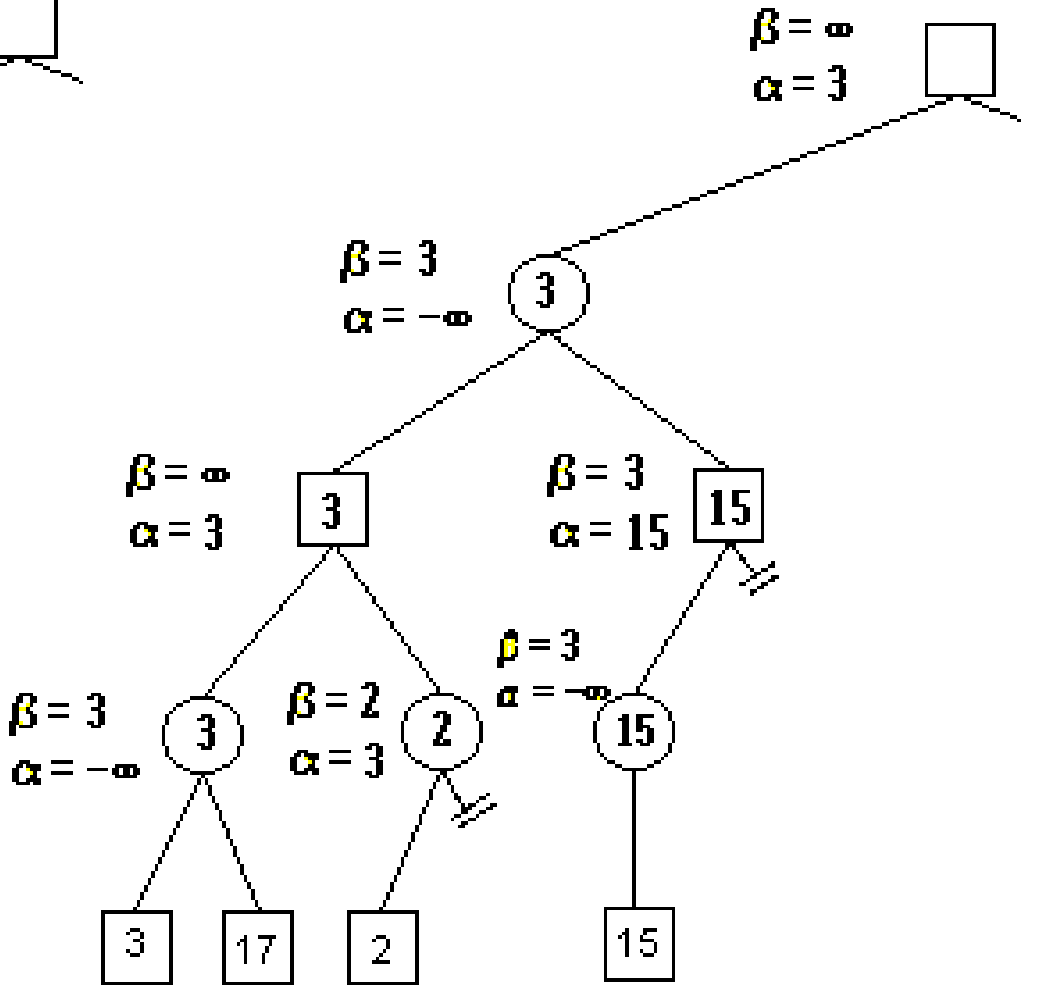

β = ∞
α = −∞
β = 3
α = −∞
β = ∞
α = 3
3
β = 3
α = −∞
β = 3
α = −∞
3
β = 2
α = 3
2
β = 3
α = −∞
3
17
2
15

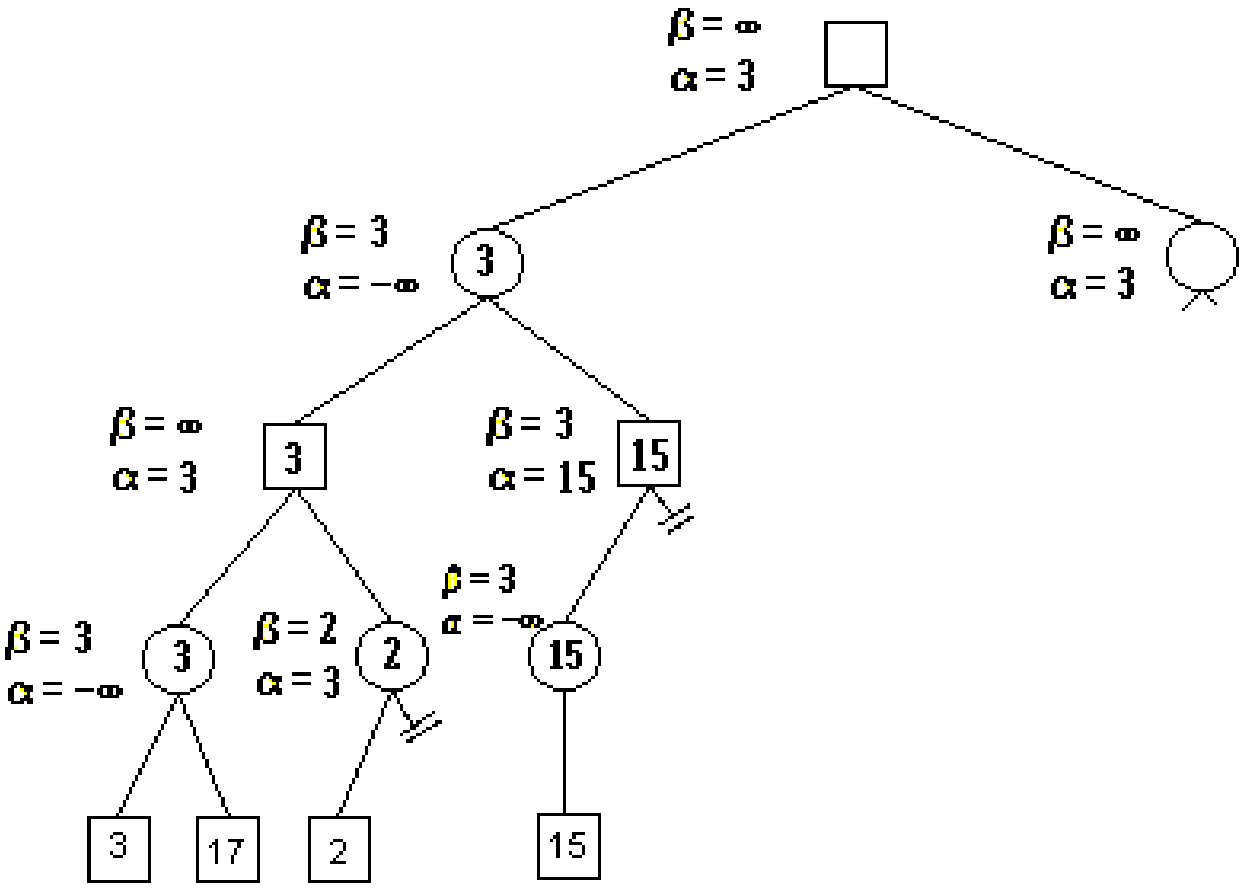

β = ∞
α = −∞
β = 3
α = −∞
β = ∞
α = 3
3
β = 3
α = −∞
β = 3
α = −∞
3
β = 2
α = 3
2
β = 3
α = −∞
15
3
17
2
15

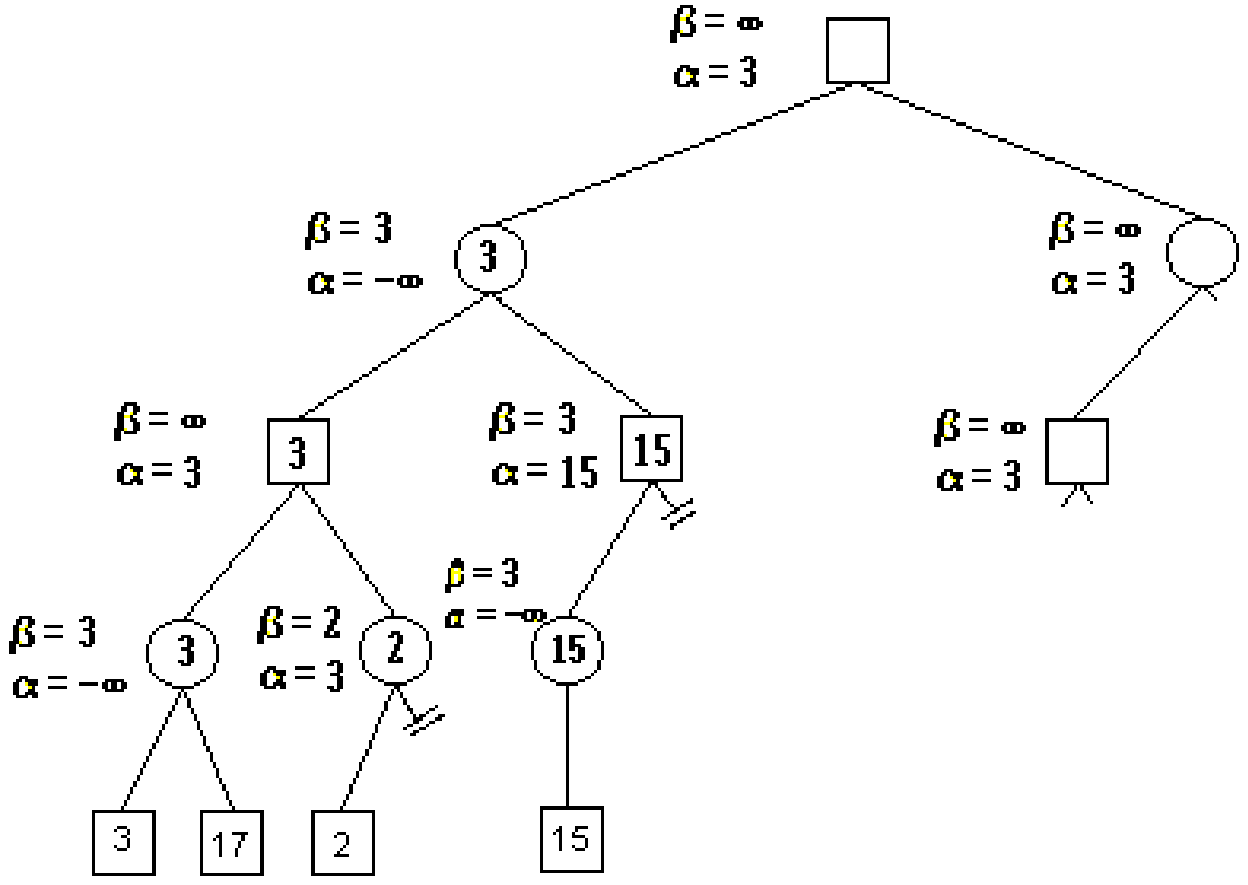

β = ∞
α = −∞
β = 3
α = −∞
β = ∞
α = 3
3
β = 3
α = 15
β = 3
α = −∞
3
β = 2
α = 3
2
β = 3
α = −∞
15
3
17
2
15

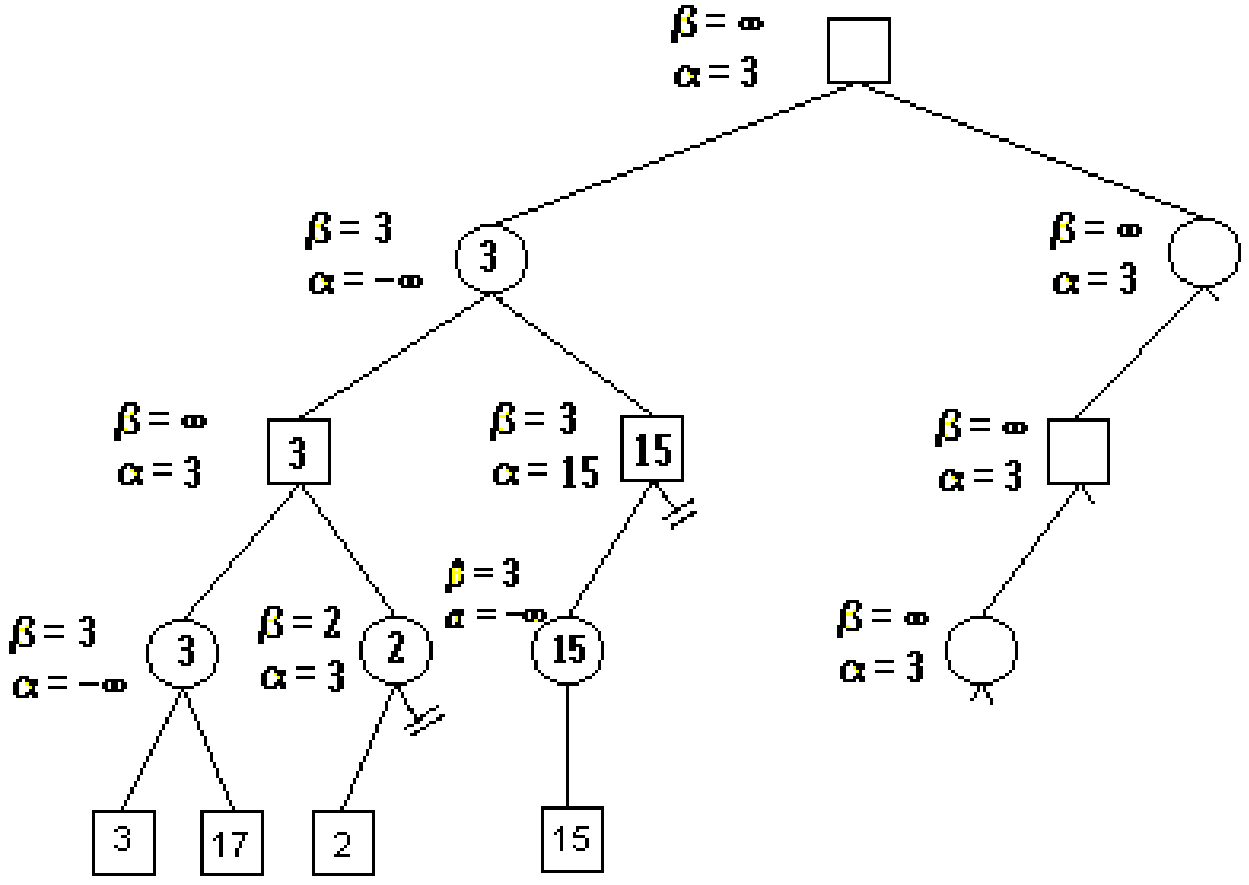

β = ∞
α = −∞
β = 3
α = −∞
β = ∞
α = 3
3
β = 3
α = 15
15
β = 3
α = −∞
3
β = 2
α = 3
2
β = 3
α = −∞
15
3
17
2
15

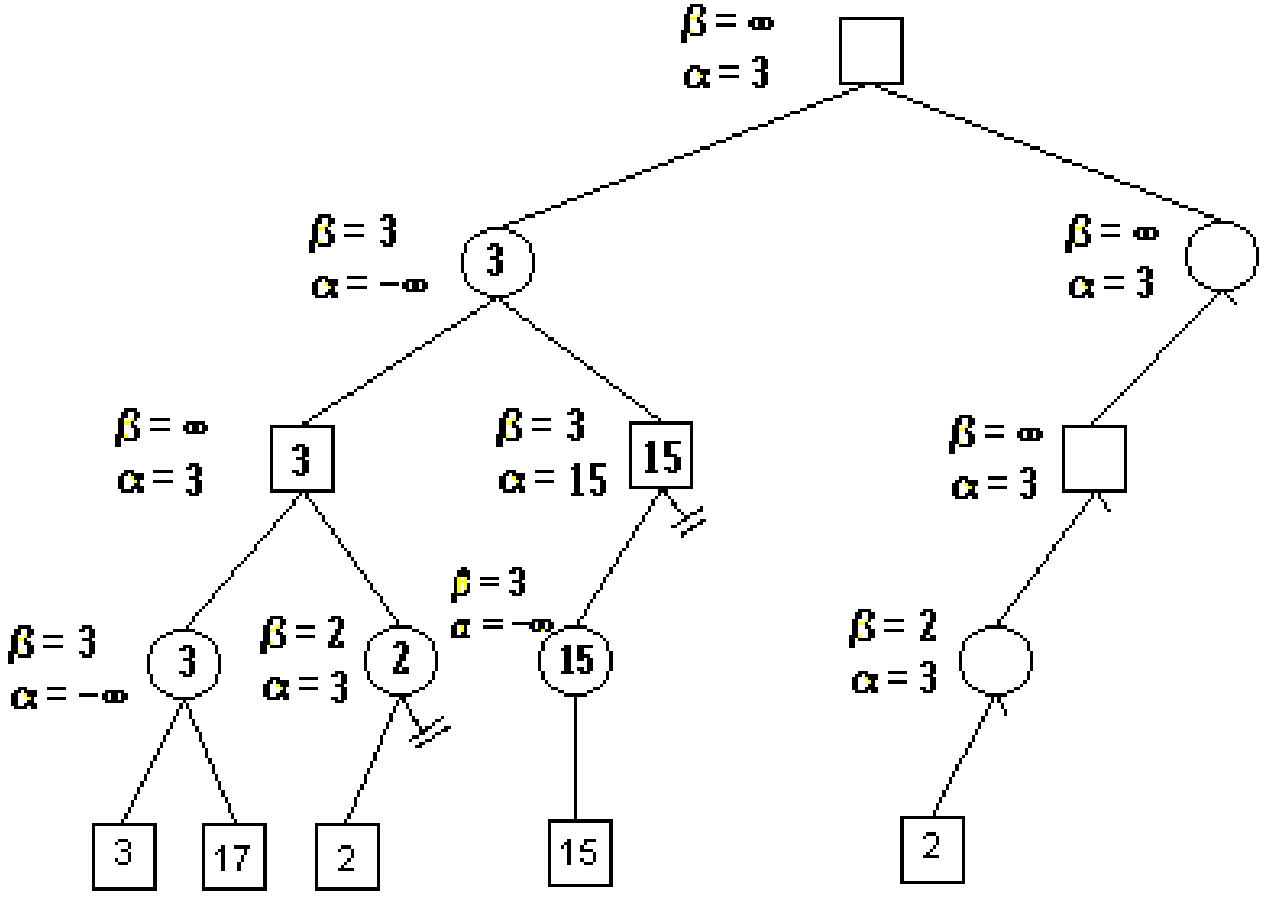

β = ∞
α = −∞
β = 3
α = −∞
3
β = ∞
α = 3
3
β = 3
α = 15
15
β = 3
α = −∞
3
β = 2
α = 3
2
β = 3
α = −∞
15
3
17
2
15

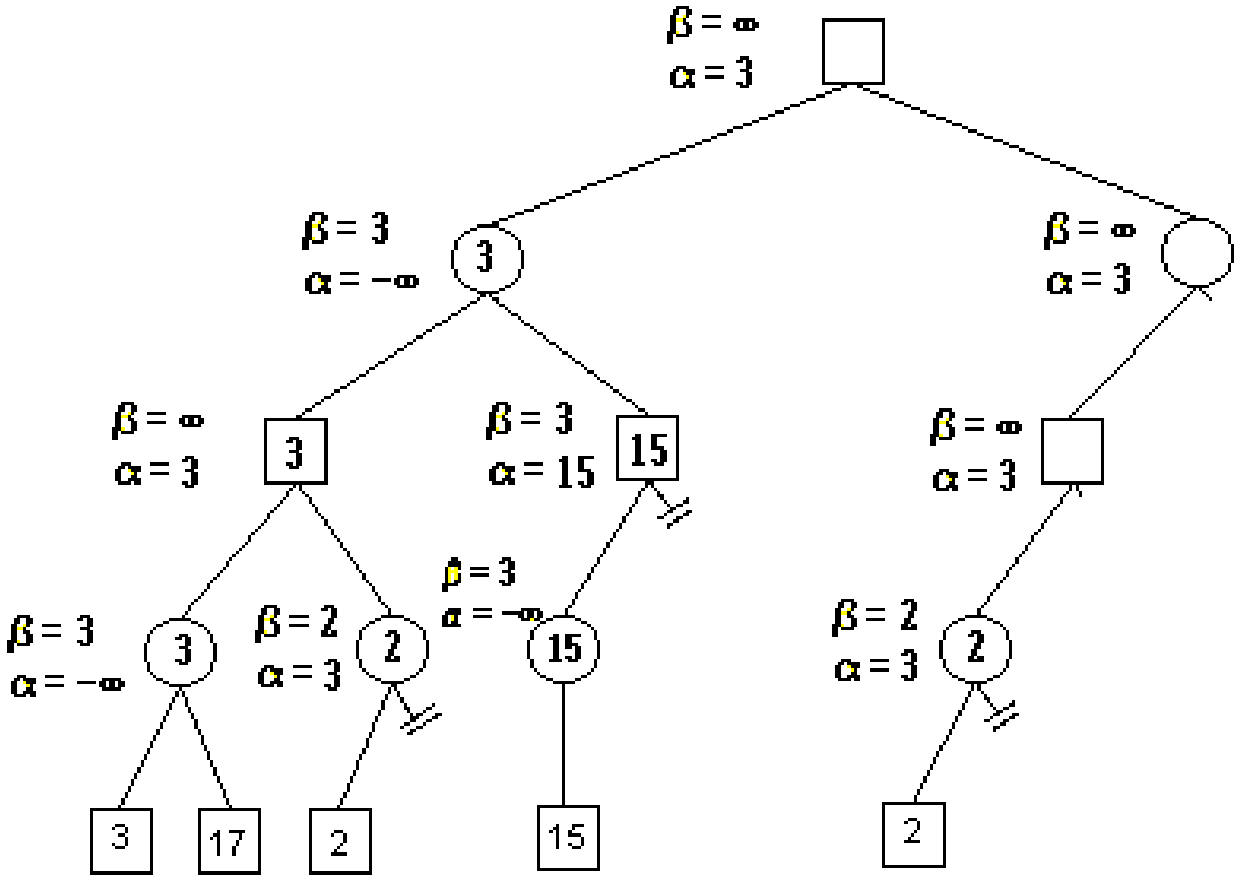

β = ∞
α = 3
β = 3
α = −∞
3
β = ∞
α = 3
3
β = 3
α = 15
15
β = 3
α = −∞
3
β = 2
α = 3
2
β = 3
α = −∞
15
3
17
2
15

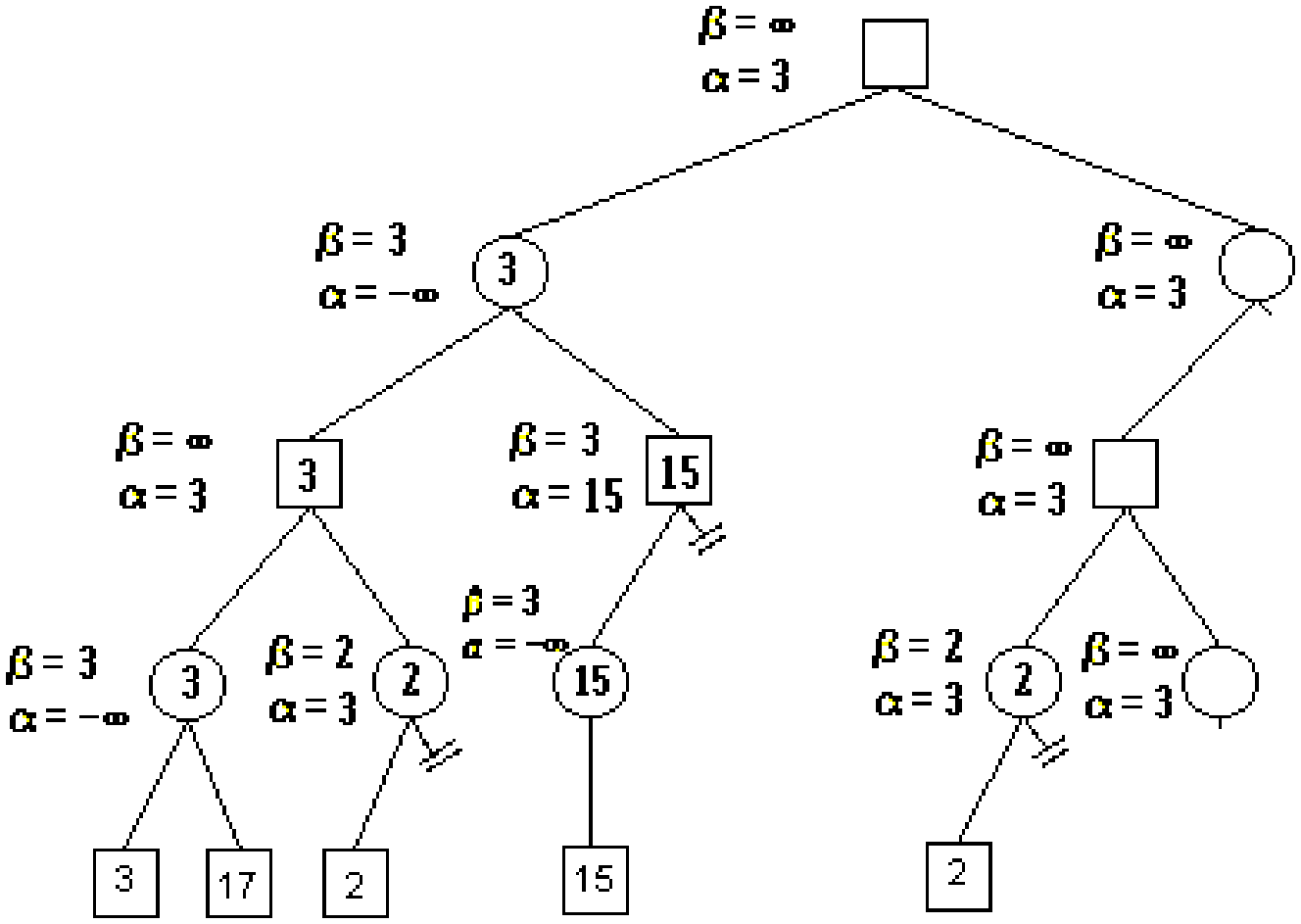

β = ∞
α = 3
β = 3
α = −∞
3
β = ∞
α = 3
β = ∞
α = 3
3
β = 3
α = 15
15
β = 3
α = −∞
β = 3
α = −∞
3
β = 2
α = 3
2
15
3
17
2
15

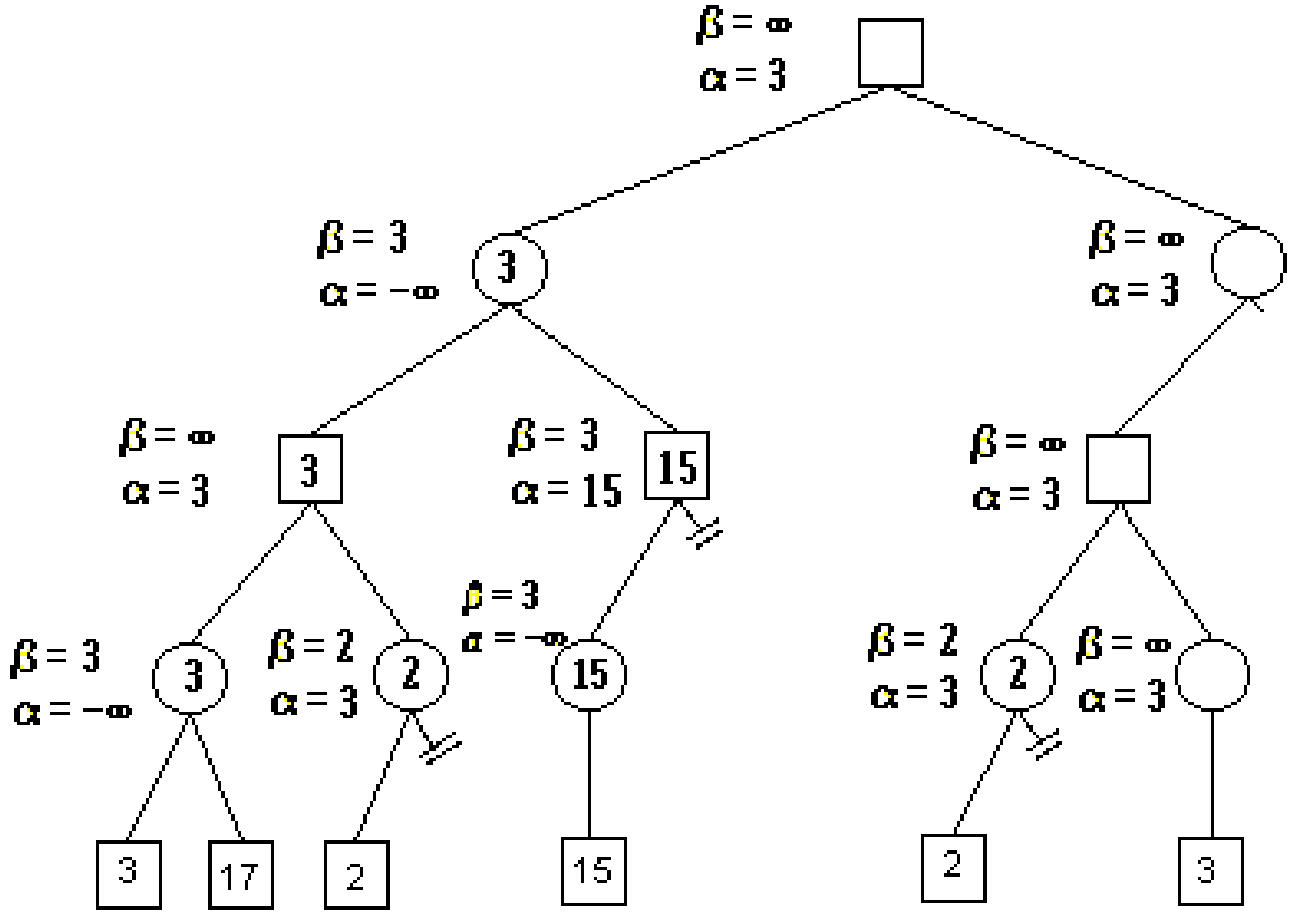

β = ∞
α = 3
β = 3
α = −∞
3
β = ∞
α = 3
β = ∞
α = 3
3
β = 3
α = 15
15
β = ∞
α = 3
β = 3
α = −∞
3
β = 2
α = 3
2
β = 3
α = −∞
15
3
17
2
15

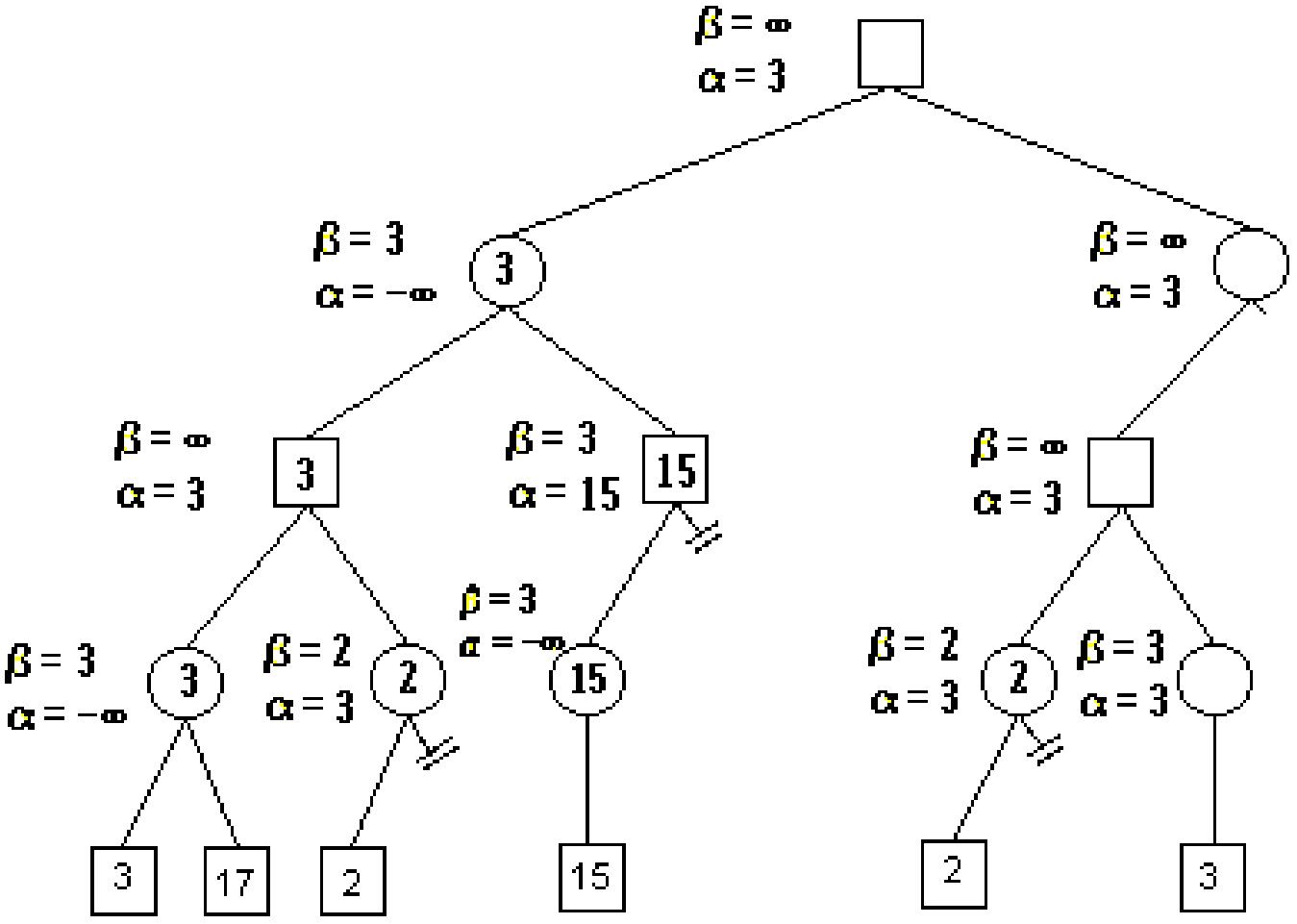

β = ∞
α = 3
β = 3
α = −∞
3
β = ∞
α = 3
β = ∞
α = 3
3
β = 3
α = 15
15
β = ∞
α = 3
β = 3
α = −∞
3
β = 2
α = 3
2
β = 3
α = −∞
15
β = ∞
α = 3
3
17
2
15

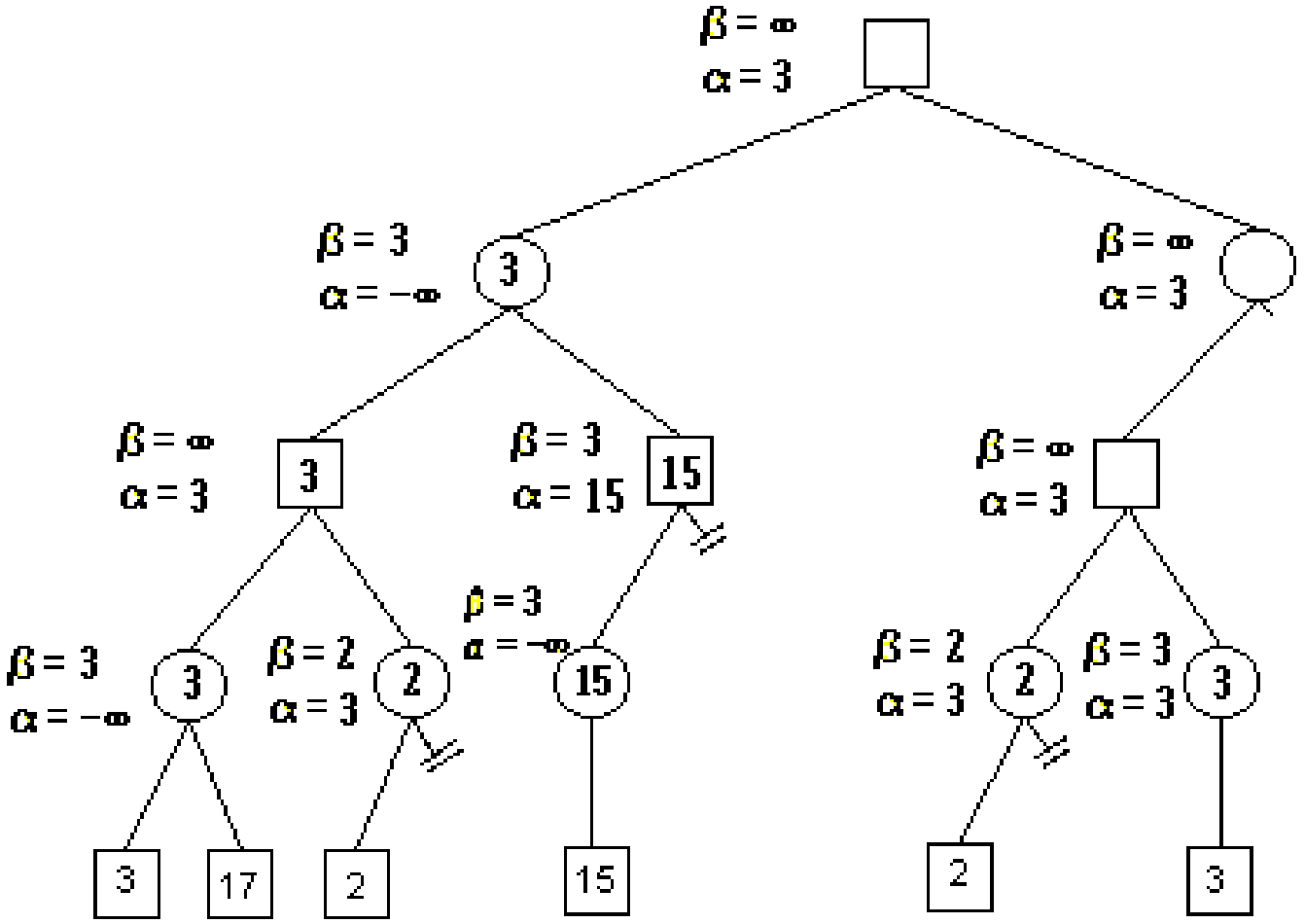

β = ∞
α = 3
β = 3
α = −∞
3
β = ∞
α = 3
β = ∞
α = 3
3
β = 3
α = 15
15
β = ∞
α = 3
β = 3
α = −∞
3
β = 2
α = 3
2
β = 3
α = −∞
15
β = 2
α = 3
3
17
2
15
2

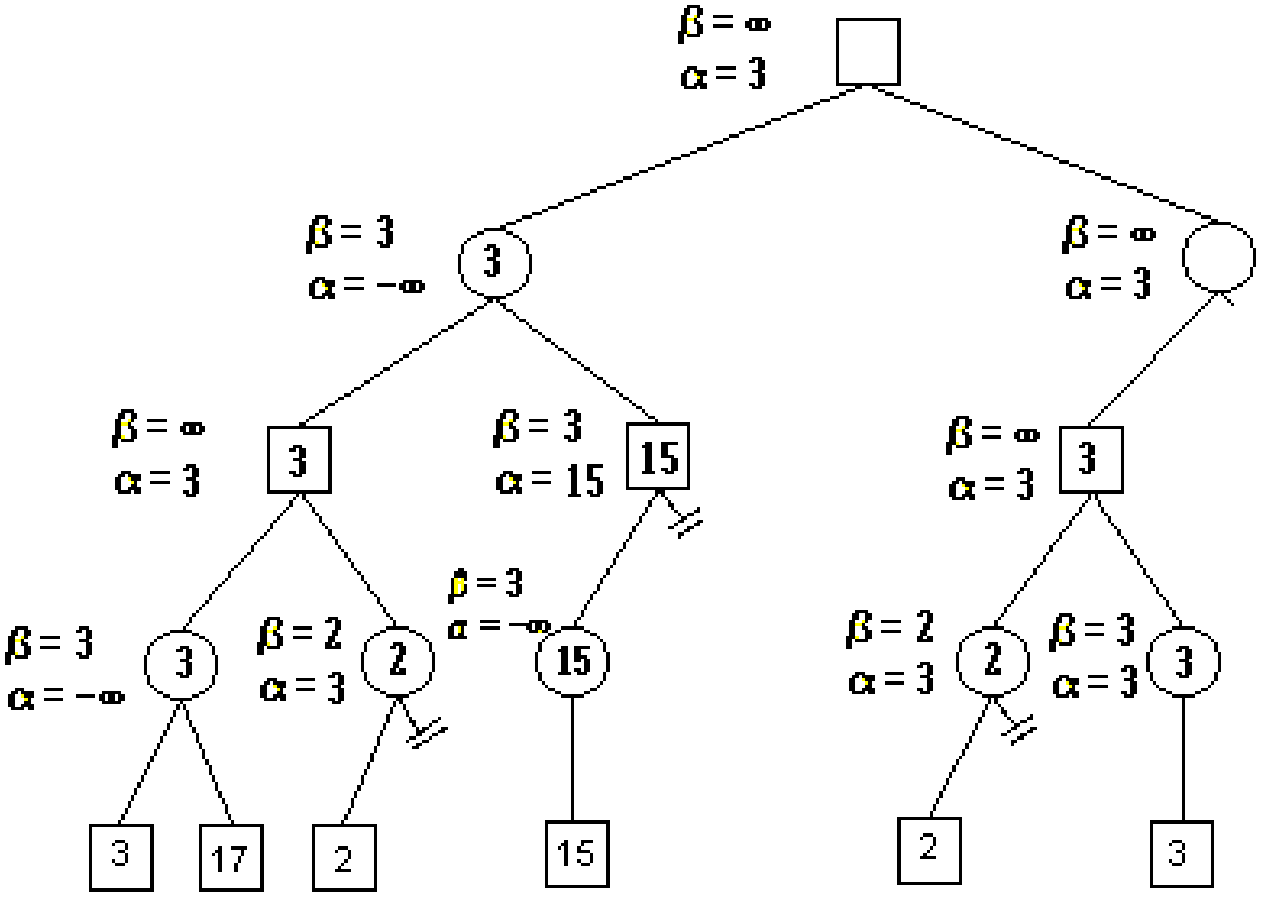

β = ∞
α = 3
β = 3
α = −∞
3
β = ∞
α = 3
β = ∞
α = 3
3
β = 3
α = 15
15
β = ∞
α = 3
β = 3
α = −∞
3
β = 2
α = 3
2
β = 3
α = −∞
15
β = 2
α = 3
2
3
17
2
15
2

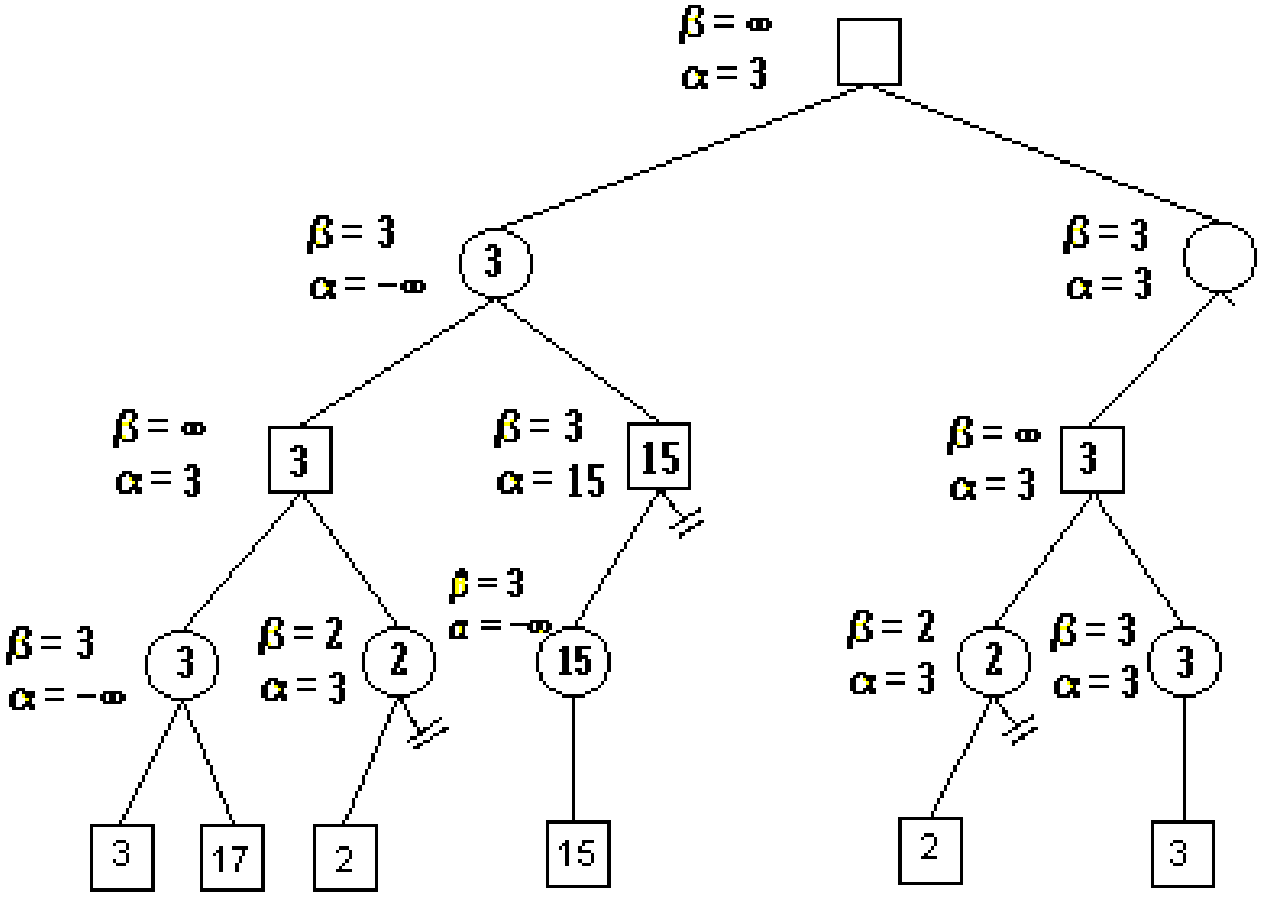

β = ∞
α = 3
β = 3
α = −∞
3
β = ∞
α = 3
β = ∞
α = 3
3
β = 3
α = 15
15
β = ∞
α = 3
β = 3
α = −∞
3
β = 2
α = 3
2
β = 3
α = −∞
15
β = 2
α = 3
2
β = ∞
α = 3
3
17
2
15
2


β = ∞
α = 3
β = 3
α = −∞
3
β = ∞
α = 3
β = ∞
α = 3
3
β = 3
α = 15
15
β = ∞
α = 3
β = 3
α = −∞
3
β = 2
α = 3
2
β = 3
α = −∞
15
β = 2
α = 3
2
β = ∞
α = 3
3
17
2
15
2
3


β = ∞
α = 3
β = 3
α = −∞
3
β = ∞
α = 3
β = ∞
α = 3
3
β = 3
α = 15
15
β = ∞
α = 3
β = 3
α = −∞
3
β = 2
α = 3
2
β = 3
α = −∞
15
β = 2
α = 3
2
β = 3
α = 3
3
17
2
15
2
3


β = ∞
α = 3
β = 3
α = −∞
3
β = ∞
α = 3
β = ∞
α = 3
3
β = 3
α = 15
15
β = ∞
α = 3
β = 3
α = −∞
3
β = 2
α = 3
2
β = 3
α = −∞
15
β = 2
α = 3
2
β = 3
α = 3
3
3
17
2
15
2
3


β = ∞
α = 3
β = 3
α = −∞
3
β = ∞
α = 3
β = ∞
α = 3
3
β = 3
α = 15
15
β = ∞
α = 3
3
β = 3
α = −∞
3
β = 2
α = 3
2
β = 3
α = −∞
15
β = 2
α = 3
2
β = 3
α = 3
3
3
17
2
15
2
3


β = ∞
α = 3
β = 3
α = −∞
3
β = 3
α = 3
β = ∞
α = 3
3
β = 3
α = 15
15
β = ∞
α = 3
3
β = 3
α = −∞
3
β = 2
α = 3
2
β = 3
α = −∞
15
β = 2
α = 3
2
β = 3
α = 3
3
3
17
2
15
2
3

Q. What is the difference between MiniMax and Alphabeta Prunning?

Mini max := Search game space

Alpha beta := Improve mini max by eliminating the nodes

Perform Min Max and Alphabeta

